

【多易教育】Flink 入门到精通

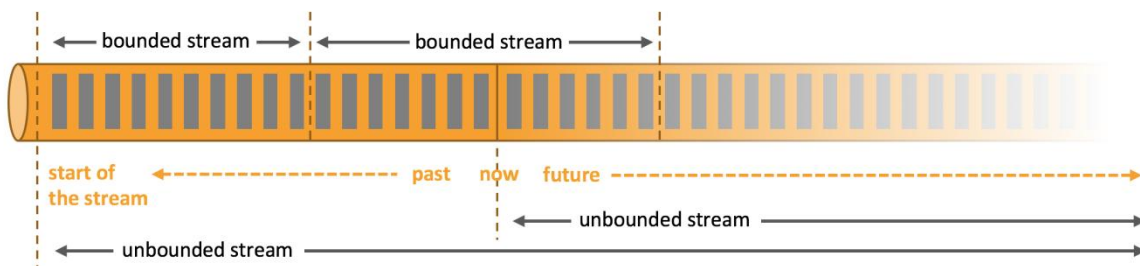


JUST DO IT
多易教育

1. 快速认识 flink

1.1. 离线批计算与实时流式计算

批计算与流式计算，本质上就是对有界流和无界流的计算



- 批计算

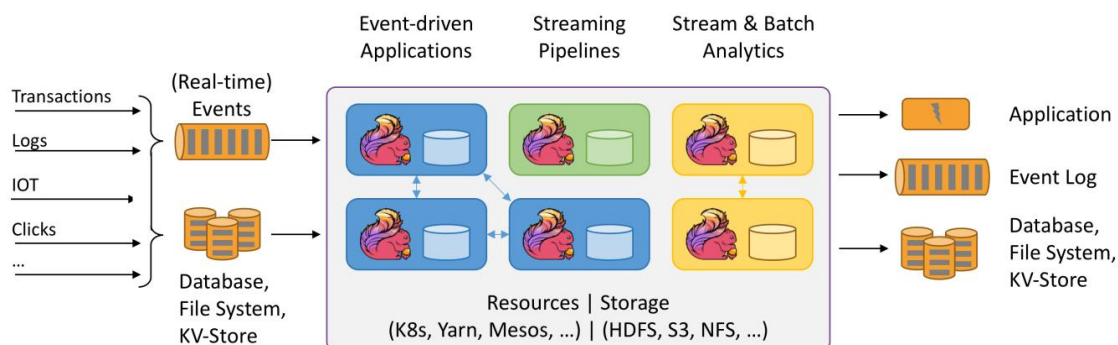
针对有界流；由于在产出计算结果前可以看到整个（完整）数据集，因而如下计算都可以实现：对数据排序，计算全局统计值，对输入数据的整体产出最终汇总聚合报表；

- 流计算

针对无界流；由于永远无法看到输入数据的整体（数据的输入永远无法结束），只能每逢数据到达就进行计算，并输出"当时"的计算结果（因而计算结果也不会是一个一次性的最终结果，而是源源不断的无界的结果流）；

1.2. Flink 基本概念

一个分布式、有状态的实时流式处理系统（编程框架）

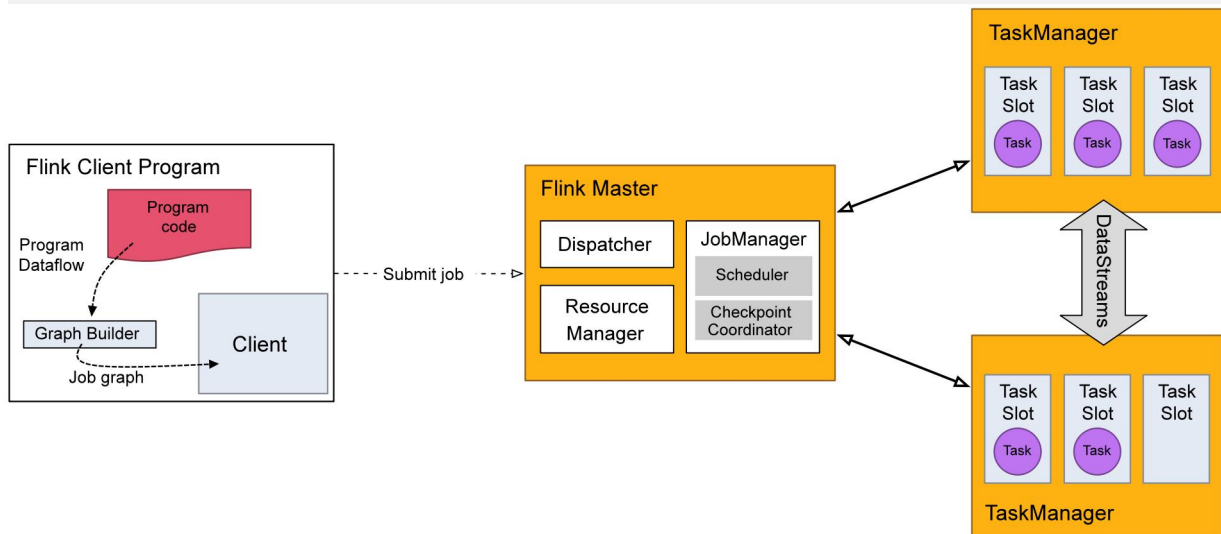


flink，主要使用java 语言开发而成，但对用户提供了java、scala、python 编程api

flink，以流处理方式作为基础的世界观，并通过引入有界流来实现批计算，从而实现流批一体

1.3. Flink 的运行架构

核心架构角色



Flink 集群采取 Master - Slave 架构：

- Master 的角色为 JobManager，负责集群和作业管理；
- Slave 的角色是 TaskManager，负责执行计算任务；
- 客户端 Client 负责管理集群和提交任务，JobManager 和 TaskManager 是集群的进程；

各角色主要职责说明

（1）Client

Flink 客户端是 Flink 提供的 CLI 命令行工具，用来提交 Flink 作业到 Flink 集群，在客户端中负责 StreamGraph (流图)和 Job Graph (作业图)的构建。

（2）JobManager

JobManager 根据并行度将 Flink 客户端提交的 Flink 应用分解为子任务，从资源管理器 ResourceManager 申请所需的计算资源，资源具备之后，开始分发任务到 TaskManager 执行 Task，并负责应用容错，跟踪作业的执行状态，发现异常则恢复作业等。

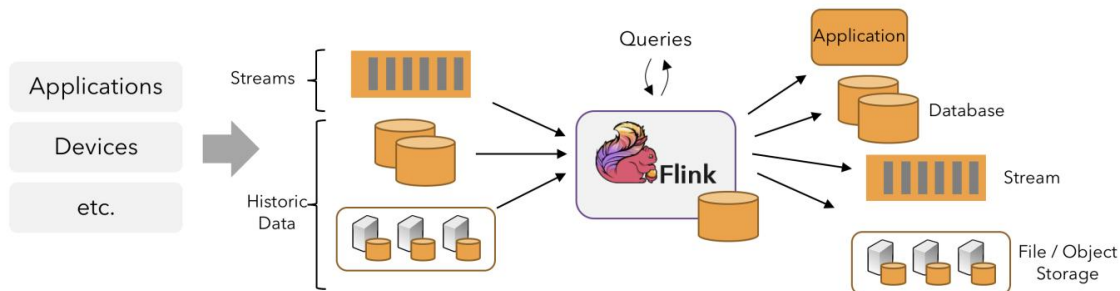
（3）TaskManager

TaskManager 接收 JobManager 分发的子任务，根据自身的资源情况 管理子任务的启动、停止、销毁、异常恢复等生命周期阶段。Flink 程序中必须有一个 TaskManager。

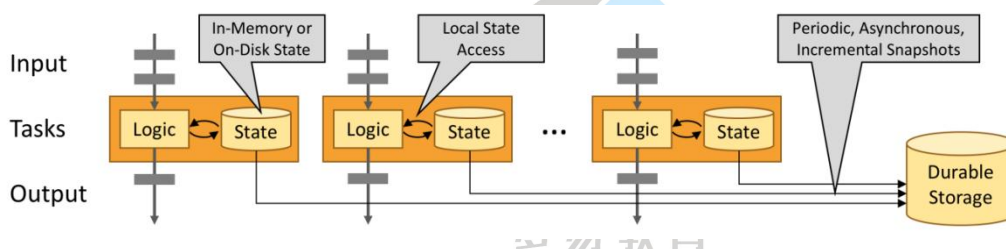
1.4. Flink 的特性

适用于几乎所有的流式数据处理场景

- 事件驱动型应用
- 流、批数据分析
- 数据管道及 ETL



自带状态管理机制

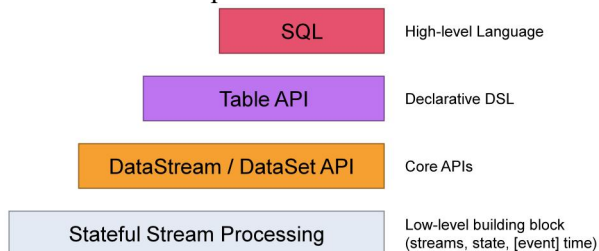


强大的准确性保证

- exactly-once 状态一致性
- 事件时间处理
- 专业的迟到数据处理

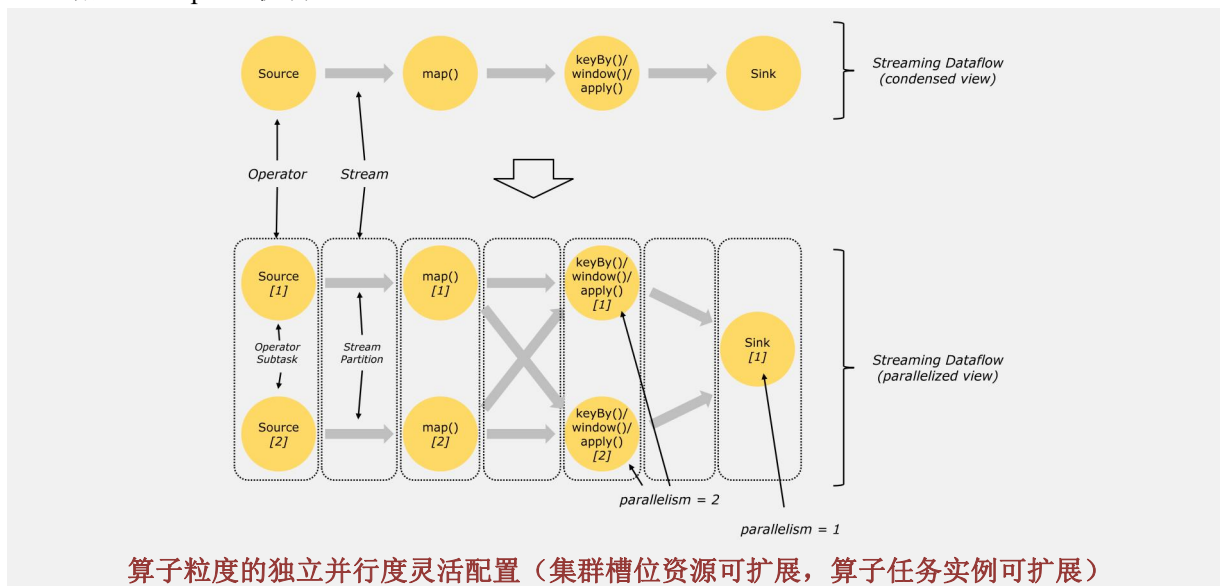
灵活丰富的多层 api

- 流、批数据之上的 SQL 查询
- 流、批数据之上的 TableApi
- datastream 流处理算子 api 、 dataset 批处理算子 api
- 精细可控的 processFunction



规模弹性扩展

- 可扩展的分布式架构（集群级别的资源规模灵活配置，算子粒度的独立并行度灵活配置）
- 支持超大状态管理
- 增量 checkpoint 机制



强大的运维能力

- 弹性实施部署机制
- 高可用配置
- 保存点恢复机制

优秀的性能

- 低延迟
- 高吞吐
- 内存计算

2. Flink 编程基础 api

2.1. 需要引入的基本依赖

```
<dependency>

    <groupId>org.apache.flink</groupId>

    <artifactId>flink-java</artifactId>

    <version>1.14.4</version>

</dependency>

<dependency>

    <groupId>org.apache.flink</groupId>

    <artifactId>flink-streaming-java_2.12</artifactId>

    <version>1.14.4</version>

</dependency>

<dependency>

    <groupId>org.apache.flink</groupId>

    <artifactId>flink-clients_2.12</artifactId>

    <version>1.14.4</version>

</dependency>
```

如需使用 *scala API*，则替换上面的：

flink-java 为 *flink-scala_2.12*

flink-streaming-java_2.12 为 *flink-streaming-scala_2.12*



2.2. flink 的 DataStream 抽象

- DataStream 代表一个数据流，它可以是无界的，也可以是有界的；
- DataStream 类似于 spark 的 rdd，它是不可变的（immutable）；
- 无法对一个 datastream 进行自由的添加或删除或修改元素；
- 只能通过算子对 datastream 中的数据进行转换，将一个 datastream 转成另一个 datastream；
- datastream 可以通过 source 算子加载、映射外部数据而来；或者从已存在的 datastream 转换而来

2.3. flink 编程模板

无论简单与复杂，flink 程序都由如下几个部分组成

- 获取一个编程、执行入口环境 env
- 通过数据源组件，加载、创建 datastream
- 对 datastream 调用各种处理算子表达计算逻辑
- 通过 sink 算子指定计算结果的输出方式
- 在 env 上触发程序提交运行

以下是一个 flink 程序的完整代码<wordcount>示例；

<测试时，需要用 nc -lk 9999(linux)或者 nc -L -p 9999(windows)在本机开启一个 9999 的 socket 服务>

- (java 版)

```
import org.apache.flink.api.common.functions.FlatMapFunction;

import org.apache.flink.api.java.tuple.Tuple2;

import org.apache.flink.streaming.api.datastream.DataStream;

import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

import org.apache.flink.streaming.api.windowing.time.Time;

import org.apache.flink.util.Collector;

public class WindowWordCount {
```

```
public static void main(String[] args) throws Exception {

    StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

    DataStream<Tuple2<String, Integer>> dataStream = env

        .socketTextStream("localhost", 9999)

        .flatMap(new Splitter())

        .keyBy(value -> value.f0)

        .window(TumblingProcessingTimeWindows.of(Time.seconds(5)))

        .sum(1);

    dataStream.print();

    env.execute("Window WordCount");

}

public static class Splitter implements FlatMapFunction<String, Tuple2<String, Integer>> {

    @Override

    public void flatMap(String sentence, Collector<Tuple2<String, Integer>> out) throws Exception {

        for (String word: sentence.split(" ")) {

            out.collect(new Tuple2<String, Integer>(word, 1));

        }

    }

}
```

```
}
```

● (scala 版)

```
import org.apache.flink.streaming.api.scala._

import org.apache.flink.streaming.api.windowing.time.Time

object WindowWordCount {

  def main(args: Array[String]) {

    val env = StreamExecutionEnvironment.getExecutionEnvironment

    val text = env.socketTextStream("localhost", 9999)

    val counts = text.flatMap { _.toLowerCase.split("\\W+") filter { _.nonEmpty } }

      .map { (_, 1) }

      .keyBy(_._1)

      .window(TumblingProcessingTimeWindows.of(Time.seconds(5)))

      .sum(1)

    counts.print()

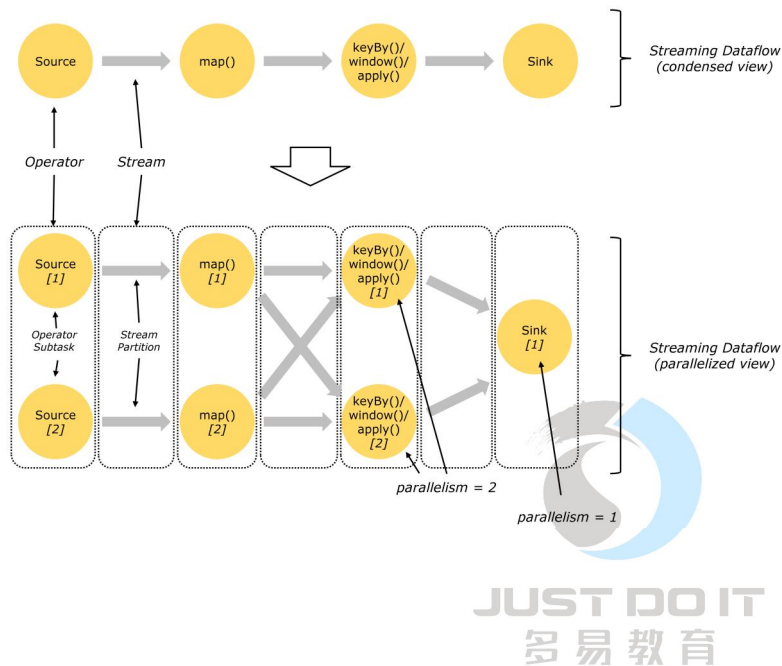
    env.execute("Window Stream WordCount")

  }

}
```

2.4. flink 程序的并行概念

- flink 程序中，每一个算子都可以成为一个独立任务（task）；
- flink 程序中，视上下游算子间数据分发规则、并行度、共享槽位设置，可组成算子链成为一个 task
- 每个任务在运行时都可拥有多个并行的运行实例（subTask）；
- 且每个算子任务的并行度都可以在代码中显式设置；



2.5. flink 编程入口

批处理计算入口

```
ExecutionEnvironment env = ExecutionEnvironment.getExecutionEnvironment();
```

流式计算入口

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
```

流批一体入口

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
```

```
// 为 env 设置环境参数

ExecutionConfig config = env.getConfig();

// 设置为批处理模式

config.setExecutionMode(ExecutionMode.BATCH);
```

开启 webui 的本地运行环境

```
Configuration conf = new Configuration();

conf.setInteger("rest.port",8081);

StreamExecutionEnvironment env = StreamExecutionEnvironment.createLocalEnvironmentWithWebUI(conf);
```

要开启本地 webui 功能，需要添加依赖

```
<dependency>

    <groupId>org.apache.flink</groupId>

    <artifactId>flink-runtime-web_2.12</artifactId>

    <version>${flink.version}</version>

</dependency>
```

2.6. 基本 source 算子

source 是用来获取外部数据的算子，按照获取数据的方式，可以分为：

- 基于集合的 Source
- 基于 Socket 网络端口的 Source
- 基于文件的 Source
- 第三方 Connector Source

- 自定义 Source 五种。

从并行度的角度，source 又可以分为非并行的 source 和并行的 source。

- 非并行 source：并行度只能为 1，即只有一个运行时实例，在读取大量数据时效率比较低，通常是用来做一些实验或测试，例如 Socket Source；
- 并行 Source：并行度可以是 1 到多个，在计算资源足够的前提下，并行度越大，效率越高。例如 Kafka Source；

2.6.1. 基于集合的 Source（测试用）

可将一个普通的 Java 集合、迭代器或者可变参数转换成一个分布式数据流 `DataStream`；

fromElements

非并行的 Source，可以将一到多个数据作为可变参数传入到该方法中，返回 `DataStreamSource`。

```
public class FromElementDemo {

    public static void main(String[] args) throws Exception {

        //创建流计算执行上下文环境

        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

        //指定多个相同类型的数据创建 DataStream

        DataStreamSource<String> words = env.fromElements("flink", "hadoop", "flink");

        //调用 Sink 将数据在控制台打印

        words.print();

        //执行

        env.execute("FromElementDemo");

    }

}
```

fromCollection

非并行的 Source，可以将一个 Collection 作为参数传入到该方法中，返回一个 DataSource。

```
//创建一个 List

List<String> wordList = Arrays.asList("flink", "spark", "hadoop", "flink");

//将 List 并行化成 DataSource

DataSource<String> words = env.fromCollection(wordList);

//从 ParallelCollection 创建并行的生成数据的 DataSource

fromParallelCollection(SplittableIterator, Class) 方法是一个并行的 Source（并行度可以使用 env 的
setParallelism 来设置），该方法需要传入两个参数，第一个是继承 SplittableIterator 的实现类的迭代器，第二个是迭代
器中数据的类型。

StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();env.setParallelism(3); //设置并行度为 3//调用 env 的
fromParallelCollection 创建并行的生成数据的 DataSource

DataSource<Long> numbers = env.fromParallelCollection(

    new NumberSequenceIterator(1L, 100L), // 生成数组的 range

    Long.class //输出数据的类型

);
```

generateSequence

并行的 Source（并行度也可以通过调用该方法后，再调用 setParallelism 来设置）通过指定的起始值和结束值来生成数据序列流；

```
//调用 env 的 generateSequence 生成并行的 DataSource，输出的数字是 1 到 100

DataSource<Long> numbers = env.generateSequence(1L, 100L).setParallelism(3);
```

2.6.2. 基于 Socket 的 Source（测试用）

非并行的 Source，通过 socket 通信来获取数据得到数据流；

该方法还有多个重载的方法，如：

```
socketTextStream(String hostname, int port, String delimiter, Long maxRetry)
```

可以指定行分隔符和最大重新连接次数。

```
//调用 env 的 socketTextStream 方法，从指定的 Socket 地址和端口创建 DataStream
```

```
DataStreamSource<String> lines = env.socketTextStream("localhost", 8888);
```

提示：socketSource 是一个非并行 source，如果使用 socketTextStream 读取数据，在启动 Flink 程序之前，必须先启动一个 Socket 服务，为了方便，Mac 或 Linux 用户可以在命令行终端输入 nc -lk 8888 启动一个 Socket 服务并在命令行中向该 Socket 服务发送数据。

2.6.3. 基于文件的 Source

基于文件的 Source，本质上就是使用指定的 FileInputFormat 组件读取数据，可以指定 TextInputFormat、CsvInputFormat、BinaryInputFormat 等格式；

底层都是 ContinuousFileMonitoringFunction，这个类继承了 RichSourceFunction，都是非并行的 Source；

readFile

readFile(FileInputFormat inputFormat, String filePath) 方法可以指定读取文件的 FileInputFormat 格式，参数 FileProcessingMode，可取值：

- PROCESS_ONCE，只读取文件中的数据一次，读取完成后，程序退出
- PROCESS_CONTINUOUSLY，会一直监听指定的文件，文件的内容发生变化后，会将以前的内容和新的内容全部都读取出来，进而造成数据重复读取。

```
String path = "file:///Users/xing/Desktop/a.txt";
```

```
//PROCESS_CONTINUOUSLY 模式是一直监听指定的文件或目录，2 秒钟检测一次文件是否发生变化
```

```
DataStreamSource<String> lines = env.readFile(new TextInputFormat(null), path,
```

```
FileProcessingMode.PROCESS_CONTINUOUSLY, 2000);
```

readTextFile

`readTextFile(String filePath)` 可以从指定的目录或文件读取数据，默认使用的是 `TextInputFormat` 格式读取数据，还有一个重载的方法 `readTextFile(String filePath, String charsetName)` 可以传入读取文件指定的字符集，默认是 UTF-8 编码。该方法是一个有限的数据源，数据读完后，程序就会退出，不能一直运行。该方法底层调用的是 `readFile` 方法，`FileProcessingMode` 为 `PROCESS_ONCE`

```
DataStreamSource<String> lines = env.readTextFile(path);
```

2.6.4. Kafka Source（生产常用）

在实际生产环境中，为了保证 flink 可以高效地读取数据源中的数据，通常是跟一些分布式消息中间件结合使用，例如 Apache Kafka。Kafka 的特点是分布式、多副本、高可用、高吞吐、可以记录偏移量等。Flink 和 Kafka 整合可以高效的读取数据，并且可以保证 `Exactly Once`（精确一次性语义）。

首先在 maven 项目的 `pom.xml` 文件中导入 Flink 跟 Kafka 整合的依赖

```
<dependency>

    <groupId>org.apache.flink</groupId>

    <artifactId>flink-connector-kafka_2.12</artifactId>

    <version>1.14.4</version>

</dependency>
```

老版本 API

核心类 `FlinkKafkaConsumer`，需指定三个参数：

- topic 名称
- 反序列化 Schema，`SimpleStringSchema` 指的是读取 Kafka 中的数据反序列化成 String 格式；
- Properties 实例

```
//设置 Kafka 相关参数

Properties properties = new Properties();//设置 Kafka 的地址和端口

properties.setProperty("bootstrap.servers", "node-1:9092,node-2:9092,node-3:9092");

//读取偏移量策略：如果没有记录偏移量，就从头读，如果记录过偏移量，就接着读
```

```
properties.setProperty("auto.offset.reset", "earliest");

//设置消费者组 ID

properties.setProperty("group.id", "g1");

//没有开启 checkpoint, 让 flink 提交偏移量的消费者定期自动提交偏移量

properties.setProperty("enable.auto.commit", "true");

//创建 FlinkKafkaConsumer 并传入相关参数

FlinkKafkaConsumer<String> kafkaConsumer = new FlinkKafkaConsumer<>(

    "test", //要读取数据的 Topic 名称

    new SimpleStringSchema(), //读取文件的反序列化 Schema

    properties //传入 Kafka 的参数

);

//使用 addSource 添加 kafkaConsumer

DataStreamSource<String> lines = env.addSource(kafkaConsumer);
```

注意：目前这种方式无法保证 Exactly Once，Flink 的 Source 消费完数据后，将偏移量定期的写入到 Kafka 的 `__consumer_offsets` 中，这种方式虽然可以记录偏移量，但是无法保证 Exactly Once；

新版本 API

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

KafkaSource<String> kafkaSource = KafkaSource.<String>builder()

    .setBootstrapServers("doit01:9092")

    .setValueOnlyDeserializer(new SimpleStringSchema())

    .setStartingOffsets(OffsetsInitializer.latest())
```

```
.setTopics("flink-01")

.setGroupId("fk03")

.build();

WatermarkStrategy<String> strategy = WatermarkStrategy

.<String>forBoundedOutOfOrderness(Duration.ZERO)

.withTimestampAssigner(new SerializableTimestampAssigner<String>() {

    @Override

    public long extractTimestamp(String element, long recordTimestamp) {

        String[] arr = element.split(",");

        return Long.parseLong(arr[3]);

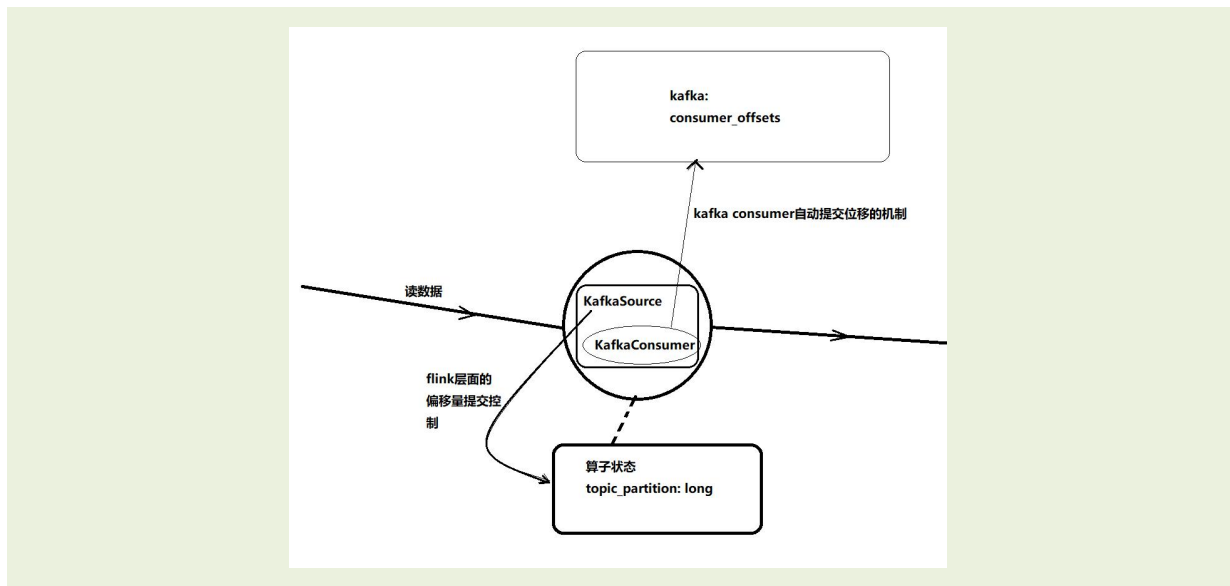
    }

});

// 1,a,100,1646784001000

DataStreamSource<String> stream1 = env.fromSource(kafkaSource,strategy,"").setParallelism(2);
```

新版本 API 中, *flink* 会把 *kafka* 消费者的消费位移记录在算子状态中, 这样就实现了消费位移状态的容错, 从而可以支持端到端的 *exactly-once*;



2.6.5. 自定义 Source

Flink 的 DataStream API 可以让开发者根据实际需要，灵活的自定义 Source，本质上就是定义一个类，实现 SourceFunction 或继承 RichParallelSourceFunction，实现 run 方法和 cancel 方法。

代码示例

```
public class MyParallelSource extends RichParallelSourceFunction<String> {

    private int i = 1; //定义一个 int 类型的变量，从 1 开始

    private boolean flag = true; //定义一个 flag 标志

    //run 方法就是用来读取外部的数据或产生数据的逻辑

    @Override

    public void run(SourceContext<String> ctx) throws Exception {

        //满足 while 循环的条件，就将数据通过 SourceContext 收集起来

        while (i <= 10 && flag) {

            Thread.sleep(1000); //为避免太快，睡眠 1 秒

            ctx.collect("data: " + i++); //将数据通过 SourceContext 收集起来
        }
    }
}
```

```
    }  
  
}  
  
//cancel 方法就是让 Source 停止  
  
@Override  
  
public void cancel() {  
  
    //将 flag 设置成 false, 即停止 Source  
  
    flag = false;  
  
}  
  
}
```

自定义 source

可以实现 `SourceFunction` 或者 `RichSourceFunction` , 这两者都是非并行的 source 算子
也可实现 `ParallelSourceFunction` 或者 `RichParallelSourceFunction` , 这两者都是可并行的 source 算子

-- 带 Rich 的, 都拥有 `open()` , `close()` , `getRuntimeContext()` 方法
-- 带 Parallel 的, 都可多实例并行执行

2.7. 基础 transformation 算子

2.7.1. 映射算子

map 映射 (DataStream → DataStream)

map(new MapFunction)

MapFunction: (x) -> y [1 条变 1 条]

```
public class MapDemo {

    public static void main(String[] args) throws Exception {

        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

        //调用 env 的 fromElements 创建一个非并行的 DataStreamSource

        DataStreamSource<String> words = env.fromElements(

            "hadoop", "spark", "flink", "hbase", "flink", "spark"

        );

        //在 map 方法中传入 MapFunction 实现类实例，重写 map 方法

        DataStream<String> upperWords = words.map(new MapFunction<String, String>() {

            @Override

            public String map(String value) throws Exception {

                //将每一个单词转成大写

                return value.toUpperCase();

            }

        });

        //调用 Sink 将数据打印在控制台

        upperWords.print();

    }

}
```

```
env.execute("MapDemo");  
  
}  
  
}
```

flatMap 扁平化映射 (DataStream → DataStream)

flatMap(new FlatMapFunction)

FlatMapFunction: x-> x1, x2,x3,x4 [1 条变多条, 并展平]

```
DataStream<Tuple2<String, Integer>> wordAndOne = lines.flatMap(  
  
    new FlatMapFunction<String, Tuple2<String, Integer>>() {  
  
        @Override  
  
        public void flatMap(String line, Collector<Tuple2<String, Integer>> out) throws Exception {  
  
            //将一行字符串按空格切分成一个字符串数组  
  
            String[] arr = line.split(" ");  
  
            for (String word : arr) {  
  
                //将单词转成小写放入到 Collector 中  
  
                out.collect(Tuple2.of(word.toLowerCase(), 1));  
  
            }  
  
        }  
  
    }  
  
);
```

如果是调用 flatMap 方法时传入 Lambda 表达式, 需要在调用 flatMap 方法后, 在调用 returns 方法指定返回的数据的类型。不然 Flink 无法自动推断出返回的数据类型, 会出现异常。

```
DataStream<Tuple2<String, Integer>> wAndOne = lines.flatMap(  
  
    (String line, Collector<Tuple2<String, Integer>> out) -> {
```

```
Arrays.asList(line.split("\\W+")).forEach(word -> {  
  
    out.collect(Tuple2.of(word.toLowerCase(), 1));  
  
});  
  
}  
  
).returns(Types.TUPLE(Types.STRING, Types.INT)); //使用 returns 指定返回数据的类型
```

project 投影 (DataStream → DataStream)

该算子只能对 Tuple 类型数据使用，project 方法的功能类似 sql 中的"select 字段"；该方法只有 Java 的 API 有，Scala 的 API 没此方法。

```
//使用 fromElements 生成数据，数据为 Tuple3 类型，分别代表姓名、性别、年龄  
  
DataStreamSource<Tuple3<String, String, Integer>> users = env.fromElements(  
  
    Tuple3.of("佩奇", "女", 5), Tuple3.of("乔治", "男", 3)  
  
);  
  
//投影方法只可以用于类型为 Tuple 的 DataStream，返回的数据只要姓名和年龄  
  
DataStream<Tuple> result = users.project(0, 2);
```

2.7.2. 过滤算子

filter 过滤 (DataStream → DataStream)

filter(new FilterFunction)

FilterFunction : x -> true/false

```
DataStreamSource<Integer> numbers = env.fromElements(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

//过滤掉奇数，保留偶数

DataStream<Integer> even = numbers.filter(new FilterFunction<Integer>() {

    @Override

    public boolean filter(Integer value) throws Exception {

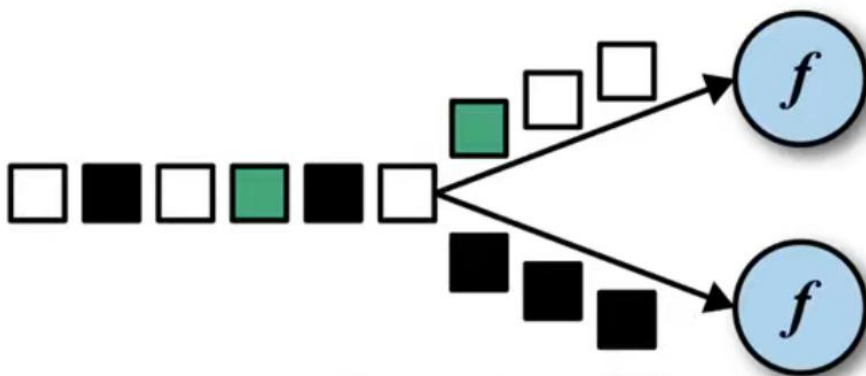
        return value % 2 == 0; //过滤掉返回 false 的数组

    }

});
```

JUST DO IT
多易教育

2.7.3. 分组算子



keyBy 按 key 分组 (DataStream → KeyedStream)

//按照 Tuple2 中的第 0 个位置进行分组，分组后得到 KeyedStream

```
KeyedStream<Tuple2<String, Integer>, Tuple> keyed = wordAndOne.keyBy(0);
```

```
//按照 Bean 中的属性名 word 进行分组
```

```
KeyedStream<CountBean, Tuple> keyed = wordAndOne.keyBy("word");
```

2.7.4. 滚动聚合算子

- 此处所说的滚动聚合算子，是多个聚合算子的统称，有 sum、min、minBy、max、maxBy；
- 这些算子的底层逻辑都是维护一个聚合值，并使用每条流入的数据对聚合值进行滚动更新；
- 这些算子都只能在 KeyedStream 上调用（就是必须 keyby 后调用）；

sum

该算子实现实时滚动相加的功能，即新输入的数据和历史数据进行相加。

```
//按照 Tuple2 中的第 0 个位置进行分组，分组后得到 KeyedStream
```

```
KeyedStream<Tuple2<String, Integer>, Tuple> keyed = wordAndOne.keyBy(0); //将分组后的数据进行 sum 运算
```

```
DataStream<Tuple2<String, Integer>> result = keyed.sum(1);
```

如果是自定义的 POJO 类型数据，可以传入一个要聚合的字段名称。

```
//按照 Bean 中的属性名 word 进行分组
```

```
KeyedStream<CountBean, Tuple> keyed = wordAndOne.keyBy("word");
```

```
//按照 Bean 中的属性名 count 进行 sum 聚合
```

```
DataStream<CountBean> result = keyed.sum("count");
```

min、minBy

这两个算子都是求最小值；min 和 minBy 的区别在于：

- min 的返回值，最小值字段以外，其他字段是第一条输入数据的值；
- minBy 返回值，就是最小值字段所在的那条数据；

底层原理：滚动更新时是更新一个字段，还是更新整条数据的区别；

```
KeyedStream<Tuple3<String, String, Integer>, Tuple> keyedStream = wordAndCount.keyBy(1);
```

```
//将分组后的数据进行调用 min、minBy

DataStream<Tuple3<String, String, Integer>> min1 = keyedStream.min(2);

DataStream<Tuple3<String, String, Integer>> min2 = keyedStream.minBy(2);

DataStream<Tuple3<String, String, Integer>> min3 = keyedStream.minBy(2, false);

//调用 print sink 打印结果

min1.print("min");

min2.print("minBy");

min3.print("minBy last");
```

max、maxBy

这两个算子都是求最大值，用法和 min、minBy 相同。

reduce 归约

它的滚动聚合逻辑没有写死，而是由用户通过 ReduceFunction 来传入。

```
//按照 Tuple2 中的第 0 个位置进行分组，分组后得到 KeyedStream

KeyedStream<Tuple2<String, Integer>, Tuple> keyed = wordAndOne.keyBy(0);

//将分组后的数据进行 reduce

DataStream<Tuple2<String, Integer>> reduced = keyed.reduce(new ReduceFunction<Tuple2<String, Integer>>()
{
    @Override
    public Tuple2<String, Integer> reduce(Tuple2<String, Integer> t1, Tuple2<String, Integer> t2)
    throws Exception {
        t1.f1 += t2.f1; //将元组对应的次数进行累加

        return t1; //返回累加后的元组
    }
})
```

```
    }  
  
};
```

fold 折叠 [已 deprecated, 1.12 版已移除]

该算子与 reduce 方法类似，只不过需要传入两个参数：

- ✓ 初始值
- ✓ 聚合函数

//按照 Tuple2 中的第 0 个位置进行分组，分组后得到 KeyedStream

```
KeyedStream<Tuple2<String, Integer>, Tuple> keyed = wordAndOne.keyBy(0);
```

```
DataStream<Tuple2<String, Integer>> result = keyed.fold(Tuple2.of(null, 0), //指定初始值
```

```
    new FoldFunction<Tuple2<String, Integer>, Tuple2<String, Integer>>() {
```

```
        @Override
```

```
        public Tuple2<String, Integer> fold(Tuple2<String, Integer> accumulator,
```

```
            Tuple2<String, Integer> input) throws Exception {
```

```
            input.f1 += accumulator.f1; //将初始值或中间累加的结果跟输入的数据进行累加
```

```
            return input;
```

```
        }
```

```
    }
```

```
};
```

iterate 迭代 (DataStream → IterativeStream → DataStream)

```
DataStreamSource<String> strs = env.socketTextStream("localhost", 8888);
```

```
DataStreamSource<String> numbers= strs.map(Long::parseLong);
```

```
//对 Nums 进行迭代

IterativeStream<Long> iteration = numbers.iterate();

DataStream<Long> iterationBody = iteration.map(new MapFunction<Long, Long>() {

    @Override

    public Long map(Long value) throws Exception {

        System.out.println("iterate input =>" + value);

        return value-=1;

    }

});

//只要满足 value > 0 的条件，就会形成一个回路，重新的迭代

DataStream<Long> feedback = iterationBody.filter(new FilterFunction<Long>(){

    @Override

    public boolean filter(Long value) throws Exception {

        return value > 0;

    }

});

iteration.closeWith(feedback);

DataStream<Long> output = iterationBody.filter(new FilterFunction<Long>(){

    @Override

    public boolean filter(Long value) throws Exception {

        return value <= 0;

    }

});
```

```
});
```



2.8. 基本 sink 算子

- sink 算子是将计算结果最终输出的算子
- 不同的 sink 算子可以将数据输出到不同的目标，如写入到的文件、输出到指定的网络端口、消息中间件、外部的文件系统或者是打印到控制台。

2.8.1. 打印输出 print

打印是最简单的一个 Sink，通常是用来做实验和测试时使用。

```
DataStream<Tuple2<String, Integer>> result = wordAndOne.keyBy(0).sum(1);

result.print();

//result.print("wc");
```



2.8.2. 文件 Sink

以下 `writeAs...` 方法均已被标记为 *deprecated*

writeAsText 以文本格式输出

该方法是将数据以文本格式实时的写入到指定的目录中，目录中的文件名称是该 Sink 所在 subtask 的 Index + 1。可以额外指定一个参数 `writeMode`，默认是 `WriteMode.NO_OVERWRITE`。如果是 `WriteMode.OVERWRITE`，会将以前的文件覆盖。

```
DataStream<Tuple2<String, Integer>> result = wordAndOne.keyBy(0).sum(1);

result.writeAsText("file:///Users/xing/Desktop/text");

//result.writeAsText("file:///Users/xing/Desktop/text", FileSystem.WriteMode.OVERWRITE);
```

writeAsCsv 以 csv 格式输出

该方法是将数据以 csv 格式写入到指定的目录中，本质上使用的是 `CsvOutputFormat` 格式写入的。该 Sink 并不是将数据实时的写入到文件中，而是有一个 `BufferedOutputStream`，默认缓存的大小为 4096 个字节，只有达到这个大小，才会 flush 到磁盘。另外程序在正常退出，调用 Sink 的 `close` 方法也会 flush 到磁盘。

```
DataStream<Tuple2<String, Integer>> result = wordAndOne.keyBy(0).sum(1);  
  
result.writeAsCsv(path);
```

writeUsingOutputFormat 以指定的格式输出

该方法是将数据以指定的格式写入到指定目录中，该方法要传入一个 `OutputFormat` 接口的实现类。

```
DataStream<Tuple2<String, Integer>> result = wordAndOne.keyBy(0).sum(1);  
  
result.writeUsingOutputFormat(new TextOutputFormat<>(new Path(path)));
```

writeToSocket 输出到网络端口

该方法是将数据输出到指定的 `Socket` 网络地址端口。输出之前，指定的网络端口服务必须已经启动。

```
DataStreamSource<String> lines = env.socketTextStream("localhost", 8888);  
  
lines.writeToSocket("localhost", 9999, new SimpleStringSchema());
```



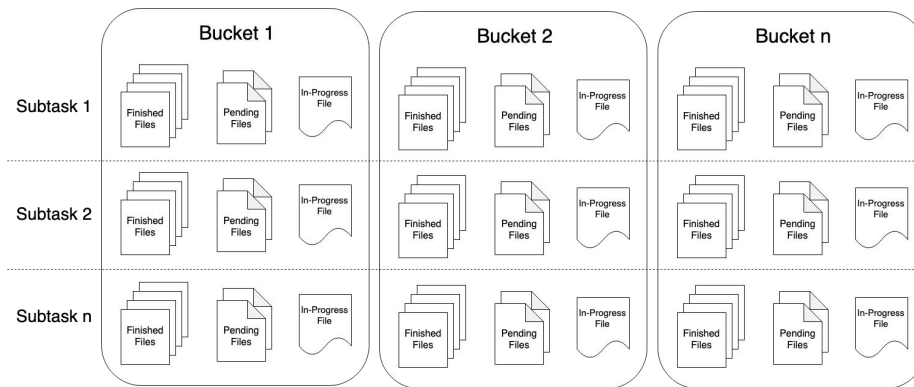
2.9. 扩展 sink 算子

2.9.1. StreamFileSink

该 Sink 不但可以将数据写入到各种文件系统中，而且整合了 `checkpoint` 机制来保证 `Exactly Once` 语义，还可以对文件进行分桶存储，还支持以**列式存储**的格式写入，功能更强大。

`streamFileSink` 中输出的文件，其生命周期会经历 3 中状态：

- in-progress Files
- Pending Files
- Finished Files



需要添加依赖:

```
<dependency>
```

```
    <groupId>org.apache.flink</groupId>
```

```
    <artifactId>flink-parquet_2.12</artifactId>
```

```
    <version>1.14.4</version>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>org.apache.flink</groupId>
```

```
    <artifactId>flink-avro</artifactId>
```

```
    <version>1.14.4</version>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>org.apache.parquet</groupId>
```

```
    <artifactId>parquet-avro</artifactId>
```

```
<version>1.11.1</version>

</dependency>

<dependency>

    <groupId>org.apache.hadoop</groupId>

    <artifactId>hadoop-client</artifactId>

    <version>2.7.7</version>

</dependency>

<dependency>

    <groupId>org.apache.flink</groupId>

    <artifactId>flink-connector-files</artifactId>

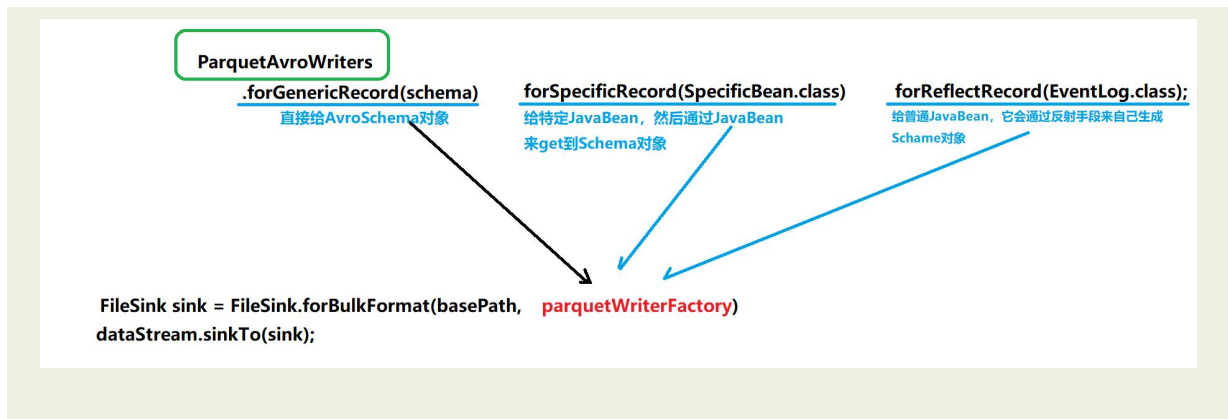
    <version>1.14.4</version>

</dependency>
```

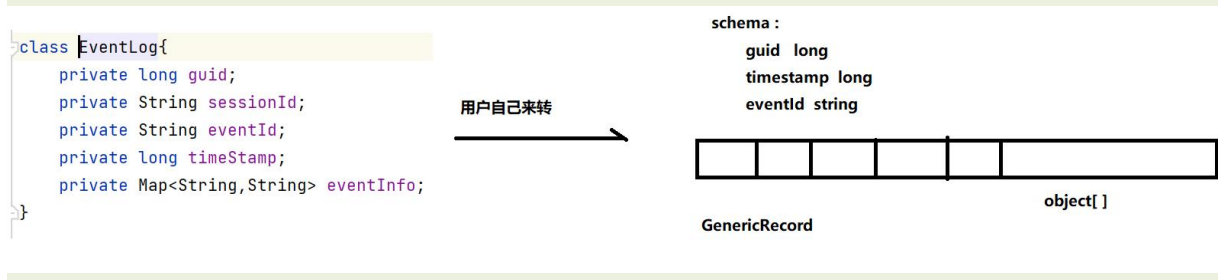
代码模板要点

通过 `DefaultRollingPolicy` 这个工具类，指定文件滚动生成的策略。这里设置的文件滚动生成策略有两个，一个是距离上一次生成文件时间超过 30 秒，另一个是文件大小达到 100 mb。这两个条件只要满足其中一个即可滚动生成文件。然后 `StreamingFileSink.forRowFormat` 方法将文件输出目录、文件写入的编码传入，再调用 `withRollingPolicy` 关联上面的文件滚动生成策略，接着调用 `build` 方法构建好 `StreamingFileSink`，最后将其作为参数传入到 `addSink` 方法中。

列式存储文件输出代码结构剖析



GenericRecord 与 Schema 的关系



2.9.1.1. Row 格式文件输出代码示例

```
DataStream<String> dataStream = ...//构建文件滚动生成的策略

DefaultRollingPolicy<String, String> rollingPolicy = DefaultRollingPolicy.create()

    .withRolloverInterval(30 * 1000L) //30 秒滚动生成一个文件

    .withMaxPartSize(1024L * 1024L * 100L) //当文件达到 100m 滚动生成一个文件

    .build();

//创建 StreamingFileSink, 数据以行格式写入

StreamingFileSink<String> sink = StreamingFileSink.forRowFormat(

    new Path(outputPath), //指的文件存储目录

    new SimpleStringEncoder<String>("UTF-8")) //指的文件编码

    .withRollingPolicy(rollingPolicy) //传入文件滚动生成策略
```

```
.build();

//调用 DataStream 的 addSink 添加该 Sink

dataStream.addSink(sink);
```

2.9.1.2. Bulk 列式存储文件输出代码示例 1

手动构建 Avro 的 Schema 对象，得到 ParquetWriterFactory 的方式

```
/**

 *

 * @Author: deep as the sea

 * @Site: www.51doit.com

 * @QQ: 657270652

 * @Date: 2022/4/26

 * @Desc: 要把处理好的数据流，输出到文件系统（hdfs）

 * 使用的 sink 算子，是扩展包中的 StreamFileSink

 */

public class _09_StreamFileSinkOperator_Demo1 {

    public static void main(String[] args) throws Exception {

        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

        // 开启 checkpoint

        env.enableCheckpointing(5000, CheckpointingMode.EXACTLY_ONCE);
```

```
env.getCheckpointConfig().setCheckpointStorage("file:///d:/ckpt");

// 构造好一个数据流

DataStreamSource<EventLog> streamSource = env.addSource(new MySourceFunction());

// 将上面的数据流输出到文件系统（假装成一个经过了各种复杂计算后的结果数据流）

/**

 * 方式一：

 * 核心逻辑：

 *   - 构造一个 schema

 *   - 利用 schema 构造一个 parquetWriterFactory

 *   - 利用 parquetWriterFactory 构造一个 FileSink 算子

 *   - 将原始数据转成 GenericRecord 流，输出到 FileSink 算子

 */

// 1. 先定义 GenericRecord 的数据模式

Schema schema = SchemaBuilder.builder()

    .record("DataRecord")

    .namespace("cn.doitedu.flink.avro.schema")

    .doc("用户行为事件数据模式")

    .fields()
```

```
.requiredInt("gid")

.requiredLong("ts")

.requiredString("eventId")

.requiredString("sessionId")

.name("eventInfo")

.type()

.map()

.values()

.type("string")

.noDefault()

.endRecord();
```

// 2. 通过定义好的 schema 模式，来得到一个 parquetWriter

```
ParquetWriterFactory<GenericRecord> writerFactory =
ParquetAvroWriters.forGenericRecord(schema);
```

// 3. 利用生成好的 parquetWriter，来构造一个 支持列式输出 parquet 文件的 sink 算子

```
FileSink<GenericRecord> sink1 = FileSink.forBulkFormat(new Path("d:/datasink/"), writerFactory)

.withBucketAssigner(new DateTimeBucketAssigner<GenericRecord>("yyyy-MM-dd--HH"))

.withRollingPolicy(OnCheckpointRollingPolicy.build())

.withOutputFileConfig(OutputFileConfig.builder().withPartPrefix("doit_edu").withPartSuffix(".parquet").build())
```

```
.build();

// 4. 将自定义 javabean 的流, 转成 上述 sink 算子中 parquetWriter 所需要的 GenericRecord 流

SingleOutputStreamOperator<GenericRecord> recordStream = streamSource

    .map((MapFunction<EventLog, GenericRecord>) eventLog -> {

        // 构造一个 Record 对象

        GenericData.Record record = new GenericData.Record(schema);

        // 将数据填入 record

        record.put("gid", (int) eventLog.getGuid());

        record.put("eventId", eventLog.getEventId());

        record.put("ts", eventLog.getTimeStamp());

        record.put("sessionId", eventLog.getSessionId());

        record.put("eventInfo", eventLog.getEventInfo());

        return record;

    }).returns(new GenericRecordAvroTypeInfo(schema)); // 由于 avro 的相关类、对象需要用 avro
的序列化器, 所以需要显式指定 AvroTypeInfo 来提供 AvroSerializer

// 5. 输出数据

recordStream.sinkTo(sink1);
```

```
env.execute();

}

}
```

2.9.1.3. Bulk 列式存储文件输出代码示例 2

编写 avsc 配置文件，并利用插件生成“特定 JavaBean”，得到 ParquetWriterFactory 的方式；

```
{ "namespace": "cn.doitedu.flink.avro.schema",

  "type": "record",

  "name": "AvroEventLog",

  "fields": [

    { "name": "guid", "type": "long" },

    { "name": "sessionId", "type": "string" },

    { "name": "eventId", "type": "string" },

    { "name": "timeStamp", "type": "long" },

    { "name": "eventInfo", "type": { "type": "map", "values": "string" } }

  ]

}
```

编译后，生成如下所谓“特定 Bean 类”

```
public class AvroEventLogBean extends SpecificRecordBase implements SpecificRecord
```

完整代码

```
/**
 *
 * @Author: deep as the sea
 * @Site: www.51doit.com
 * @QQ: 657270652
 * @Date: 2022/4/26
 * @Desc: 要把处理好的数据流，输出到文件系统（hdfs）
 * 使用的 sink 算子，是扩展包中的 StreamFileSink
 */

public class _09_StreamFileSinkOperator_Demo2 {

    public static void main(String[] args) throws Exception {

        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

        // 开启 checkpoint

        env.enableCheckpointing(5000, CheckpointingMode.EXACTLY_ONCE);

        env.getCheckpointConfig().setCheckpointStorage("file:///d:/ckpt");

        // 构造好一个数据流

        DataStreamSource<EventLog> streamSource = env.addSource(new MySourceFunction());
```

```
// 将上面的数据流输出到文件系统（假装成一个经过了各种复杂计算后的结果数据流）

/**
 * 方式二：
 *
 * 核心逻辑：
 *
 *   - 编写一个 avsc 文本文件（json），来描述数据模式
 *
 *   - 添加 maven 代码生成器插件，来针对上述的 avsc 生成 avro 特定格式的 JavaBean 类
 *
 *   - 利用代码生成器生成的 JavaBean，来构造一个 parquetWriterFactory
 *
 *   - 利用 parquetWriterFactory 构造一个 FileSink 算子
 *
 *   - 将原始数据流 转成 特定格式 JavaBean 流，输出到 FileSink 算子
 */

// 1. 先定义 avsc 文件放在 resources 文件夹中，并用 maven 的插件，来编译一下，生成特定格式的 JavaBean :
AvroEventLog

// 这种根据 avsc 生成的 JavaBean 类，自身就已经带有了 Schema 对象

// AvroEventLog avroEventLog = new AvroEventLog();

// Schema schema = avroEventLog.getSchema();

// 2. 通过自动生成 AvroEventLog 类，来得到一个 parquetWriter

ParquetWriterFactory<AvroEventLog> parquetWriterFactory =
ParquetAvroWriters.forSpecificRecord(AvroEventLog.class);
```

// 3. 利用生成好的 `parquetWriter`，来构造一个 支持列式输出 `parquet` 文件的 `sink` 算子

```
FileSink<AvroEventLog> bulkSink = FileSink.forBulkFormat(new Path("d:/datasink2/"),
parquetWriterFactory)

    .withBucketAssigner(new DateTimeBucketAssigner<AvroEventLog>("yyyy-MM-dd--HH"))

    .withRollingPolicy(OnCheckpointRollingPolicy.build())

    .withOutputFileConfig(OutputFileConfig.builder().withPartPrefix("doit_edu").withPartSuffix(".parquet").build())

    .build();
```

// 4. 将自定义 `javabean` 的 `EventLog` 流，转成 上述 `sink` 算子中 `parquetWriter` 所需要的 `AvroEventLog` 流

```
SingleOutputStreamOperator<AvroEventLog> avroEventLogStream = streamSource.map(new
MapFunction<EventLog, AvroEventLog>() {

    @Override

    public AvroEventLog map(EventLog eventLog) throws Exception {

        HashMap<CharSequence, CharSequence> eventInfo1 = new HashMap<>();

        // 进行 hashmap<charsequence, charsequence>类型的数据转移

        Map<String, String> eventInfo2 = eventLog.getEventInfo();

        Set<Map.Entry<String, String>> entries = eventInfo2.entrySet();

        for (Map.Entry<String, String> entry : entries) {

            eventInfo1.put(entry.getKey(), entry.getValue());
```

```
    }

    return new AvroEventLog(eventLog.getGuid(), eventLog.getSessionId(),
eventLog.getEventId(), eventLog.getTimeStamp(), eventInfo1);

    }

});

// 5. 输出数据

avroEventLogStream.sinkTo(bulkSink);

env.execute();

}

}
```

2.9.1.4. Bulk 列式存储文件输出代码示例 3

直接利用普通 JavaBean，利用工具自身的反射机制，得到 ParquetWriterFactory 的方式；

```
/**

 * @Author: deep as the sea

 * @Site: www.51doit.com
```

```
* @QQ: 657270652

* @Date: 2022/4/26

* @Desc: 要把处理好的数据流，输出到文件系统（hdfs）

* 使用的 sink 算子，是扩展包中的 StreamFileSink

**/

public class _09_StreamFileSinkOperator_Demo3 {

    public static void main(String[] args) throws Exception {

        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

        // 开启 checkpoint

        env.enableCheckpointing(5000, CheckpointingMode.EXACTLY_ONCE);

        env.getCheckpointConfig().setCheckpointStorage("file:///d:/ckpt");

        // 构造好一个数据流

        DataStreamSource<EventLog> streamSource = env.addSource(new MySourceFunction());

        // 将上面的数据流输出到文件系统（假装成一个经过了各种复杂计算后的结果数据流）

        /**

         * 方式三：

         * 核心逻辑：
```

```
*   - 利用自己的 JavaBean 类，来构造一个 parquetWriterFactory

*   - 利用 parquetWriterFactory 构造一个 FileSink 算子

*   - 将原始数据流，输出到 FileSink 算子

*/

// 2. 通过自己的 JavaBean 类，来得到一个 parquetWriter

ParquetWriterFactory<EventLog> parquetWriterFactory =
ParquetAvroWriters.forReflectRecord(EventLog.class);

// 3. 利用生成好的 parquetWriter，来构造一个 支持列式输出 parquet 文件的 sink 算子

FileSink<EventLog> bulkSink = FileSink.forBulkFormat(new Path("d:/datasink3/"),
parquetWriterFactory)

    .withBucketAssigner(new DateTimeBucketAssigner<EventLog>("yyyy-MM-dd--HH"))

    .withRollingPolicy(OnCheckpointRollingPolicy.build())

    .withOutputFileConfig(OutputFileConfig.builder().withPartPrefix("doit_edu").withPartSuffix(".parquet").build())

    .build();

// 5. 输出数据

streamSource.sinkTo(bulkSink);

env.execute();

}
```

```
}
```

2.9.2. KafkaSink

Flink 可以和 Kafka 多个版本整合，比如 0.11.x、1.x、2.x 等；
从 Flink1.9 开始，使用的是 kafka 2.2 的客户端。

核心类

- KafkaStringSerializationSchema -- 反序列化
- FlinkKafkaProducer -- 生产者（即 sink）

老版本 API

//自定义 String 类型数据 Kafka 的序列化 Schema

```
public class KafkaStringSerializationSchema implements KafkaSerializationSchema<String> {

    private String topic;

    private String charset;

    //构造方法传入要写入的 topic 和字符集，默认使用 UTF-8

    public KafkaStringSerializationSchema(String topic) {

        this(topic, "UTF-8");

    }

    public KafkaStringSerializationSchema(String topic, String charset) {

        this.topic = topic;

        this.charset = charset;

    }

    //调用该方法将数据进行序列化
```

```
@Override

public ProducerRecord<byte[], byte[]> serialize(

    String element, @Nullable Long timestamp) {

    //将数据转成 bytes 数组

    byte[] bytes = element.getBytes(Charset.forName(charset));

    //返回 ProducerRecord

    return new ProducerRecord<>(topic, bytes);

}

}
```

```
DataStream<String> dataStream = ...

//写入 Kafka 的 topic

String topic = "test";

//设置 Kafka 相关参数

Properties properties = new Properties();

properties.setProperty("bootstrap.servers", "node-1:9092,node-2:9092,node-3:9092");

//创建 FlinkKafkaProducer

FlinkKafkaProducer<String> kafkaProducer = new FlinkKafkaProducer<String>(

    topic, //指定 topic

    new KafkaStringSerializationSchema(topic), //指定写入 Kafka 的序列化 Schema

    properties, //指定 Kafka 的相关参数

    FlinkKafkaProducer.Semantic.EXACTLY_ONCE //指定写入 Kafka 为 EXACTLY_ONCE 语义
```

```
);  
  
//添加 KafkaSink  
  
dataStream.addSink(kafkaProducer);
```

新版本 API

```
// 读数据，写入 kafka  
  
DataStreamSource<String> source = env.socketTextStream("localhost", 9099);  
  
KafkaSink<String> kafkaSink = KafkaSink.<String>builder()  
  
    .setBootstrapServers("doit01:9092,doit02:9092,doit03:9092")  
  
    .setRecordSerializer(  
  
        KafkaRecordSerializationSchema.builder()  
  
            .setTopic("topic-name")  
  
            .setValueSerializationSchema(new SimpleStringSchema())  
  
            .build()  
  
        )  
  
    .setDeliverGuarantee(DeliveryGuarantee.AT_LEAST_ONCE)  
  
    .setTransactionalIdPrefix("doitedu-")  
  
    .build();  
  
source.sinkTo(kafkaSink);
```

KafkaSink 是能结合 Flink 的 Checkpoint 机制，来支持端到端精确一次语义的；
(底层，当然是利用了 kafka producer 的事务机制)

2.9.3. JdbcSink

不保证 Exactly-Once 的 JdbcSink

```
// 构造一个 jdbc 的 sink

// 该 sink 的底层并没有去开启 mysql 的事务，所以并不能真正保证 端到端的 精确一次

SinkFunction<Student> jdbcSink = JdbcSink.sink(

    "insert into flink_stu values (?, ?, ?, ?) on duplicate key update name=?,gender=?,score=?",

    new JdbcStatementBuilder<Student>() {

        @Override

        public void accept(PreparedStatement preparedStatement, Student student) throws SQLException

        {

            preparedStatement.setInt(1, student.getId());

            preparedStatement.setString(2, student.getName());

            preparedStatement.setString(3, student.getGender());

            preparedStatement.setFloat(4, (float) student.getScore());

            preparedStatement.setString(5, student.getName());

            preparedStatement.setString(6, student.getGender());

            preparedStatement.setFloat(7, (float) student.getScore());

        }

    },

    JdbcExecutionOptions.builder()
```

```
.withMaxRetries(3)

.withBatchSize(1)

.build(),

new JdbcConnectionOptions.JdbcConnectionOptionsBuilder()

.withUrl("jdbc:mysql://doit01:3306/abc")

.withUsername("root")

.withPassword("ABC123.abc123")

.build()

);
```

保证 Exactly-Once 的 JdbcSink

```
SinkFunction<Student> exactlyOnceSink = JdbcSink.exactlyOnceSink(

    "insert into flink_stu values (?, ?, ?, ?) on duplicate key update name=?,gender=?,score=? ",

    /*"insert into flink_stu values (?, ?, ?, ?) ",*/

    (PreparedStatement preparedStatement, Student student) -> {

        preparedStatement.setInt(1, student.getId());

        preparedStatement.setString(2, student.getName());

        preparedStatement.setString(3, student.getGender());

        preparedStatement.setFloat(4, (float) student.getScore());

        preparedStatement.setString(5, student.getName());

        preparedStatement.setString(6, student.getGender());
```

```
        preparedStatement.setFloat(7, (float) student.getScore());

    },

    JdbcExecutionOptions.builder()

        .withMaxRetries(3)

        .withBatchSize(1)

        .build(),

    JdbcExactlyOnceOptions.builder()

        // mysql 不支持同一个连接上存在并行的多个事务，必须把该参数设置为 true

        .withTransactionPerConnection(true)

        .build(),

    new SerializableSupplier<XADataSource>() {

        @Override

        public XADataSource get() {

            // XADataSource 就是 jdbc 连接，不过它是支持分布式事务的连接

            // 而且它的构造方法，不同的数据库构造方法不同

            MysqlXADataSource xaDataSource = new MysqlXADataSource();

            xaDataSource.setUrl("jdbc:mysql://doit01:3306/abc");

            xaDataSource.setUser("root");

            xaDataSource.setPassword("ABC123.abc123");

            return xaDataSource;

        }

    }

}
```

```
);
```

将 JdbcSink 添加到流

```
// 把构造好的 sink 添加到流中
```

```
studentStream.addSink(exactlyOnceSink);
```



2.9.4. RedisSink

Redis 是一个基于内存、性能极高的 NoSQL 数据库，数据还可以持久化到磁盘，读写速度快，适合存储 key-value 类型的数据。Redis 不仅仅支持简单的 key-value 类型的数据，同时还提供 list, set, zset, hash 等数据结构的存储。Flink 实时计算出的结果，需要快速的输出存储起来，要求写入的存储系统的速度要快，这个才不会造成数据积压。Redis 就是一个非常不错的选择。

首先在 maven 项目中的 pom.xml 中添加 Redis Sink 的依赖。

```
<dependency>

    <groupId>org.apache.bahir</groupId>

    <artifactId>flink-connector-redis_2.12</artifactId>

    <version>1.1-SNAPSHOT</version>

</dependency>
```

这个依赖，在 maven 官方依赖仓库中是没有的，需要下载 bahir 的源码到本地，并进行编译和安装到 maven 本地仓库，然后才能按照下面的方式引入；

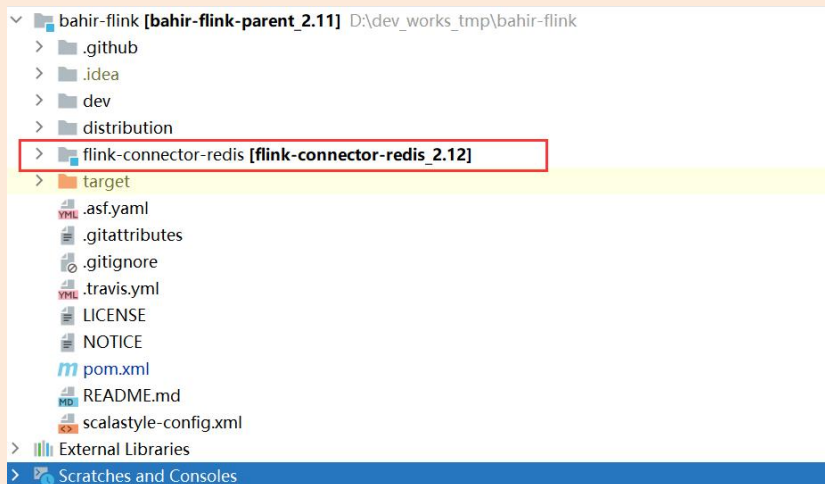
编译步骤：

1. Clone 源码：<https://github.com/apache/bahir-flink.git>
2. 修改源码中父工程的 pom 文件：

```
<properties>
<project.build.outputTimestamp>10</project.build.outputTimestamp>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
<java.version>1.8</java.version>
<maven.compiler.source>${java.version}</maven.compiler.source>
<maven.compiler.target>${java.version}</maven.compiler.target>

<!-- Flink version -->
<flink.version>1.14.4</flink.version>
<scala.binary.version>2.12</scala.binary.version>
<scala.version>2.12.12</scala.version>
```

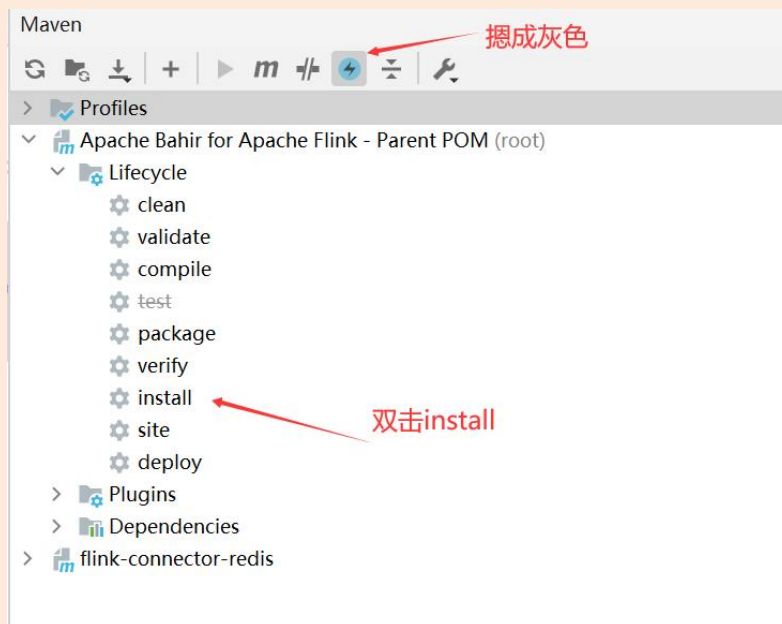
3. 移除掉其他的 connector 子模块



4. 同时，从父 pom 中，也删除对 module 的引用

```
<modules>
  <module>flink-connector-redis</module>
</modules>
```

5. 然后，按如下方式编译安装



接下来就是定义一个类（或者静态内部类）实现 RedisMapper 即可，需要指定一个泛型，即写入到 Redis 中的数据的数据类型，并实现三个方法：

- getCommandDescription 方法，返回 RedisCommandDescription 实例，在该构造方法中可以指定写入到 Redis 的方法类型为 HSET，和 Redis 的 additionalKey 即 value 为 HASH 类型外面 key 的值：

- getKeyFromData 是指定 value 为 HASH 类型对应 key 的值；
- geVauleFromData 是指定 value 为 HASH 类型对应 value 的值。

```
public static class RedisWordCountMapper implements RedisMapper<Tuple2<String, Integer>> {  
  
    @Override  
  
    public RedisCommandDescription getCommandDescription() {  
  
        //写入 Redis 的方法，value 使用 HASH 类型，并指定外面 key 的值得名称  
  
        return new RedisCommandDescription(RedisCommand.HSET, "WORD_COUNT");  
  
    }  
  
    @Override  
  
    public String getKeyFromData(Tuple2<String, Integer> data) {  
  
        return data.f0; //指定写入 Redis 的 value 里面 key 的值  
  
    }  
  
    @Override  
  
    public String getValueFromData(Tuple2<String, Integer> data) {  
  
        return data.f1.toString(); //指定写入 value 里面 value 的值  
  
    }  
  
}
```

在使用之前，先 new FlinkJedisPoolConfig，设置 Redis 的 ip 地址或主机名、端口号、密码等。然后 new RedisSink 将准备好的 conf 和 RedisWordCountMapper 实例传入到其构造方法中，最后调用 DataStream 的 addSink 方法，将 new 好的 RedisSink 作为参数传入。

```
DataStream<Tuple2<String, Integer>> result = wordAndOne.keyBy(0).sum(1); //设置 Redis 的参数，如地址、端口号等
```

```
FlinkJedisPoolConfig conf = new  
FlinkJedisPoolConfig.Builder().setHost("localhost").setPassword("123456").build();  
  
//将数据写入 Redis  
  
result.addSink(new RedisSink<>(conf, new RedisWordCountMapper()));
```



3. flink 多流操作 api

3.1. split 分流操作 [已 deprecated]

split 拆分 (DataStream → SplitStream)

该方法是将一个 DataStream 中的数据流打上不同的标签，逻辑的拆分成多个不同类型的流，返回一个新的 SplitStream，本质上还是一个数据流，只不过是流中的数据打上了不同的标签。

返回的 SplitStream 已经标记为过时，1.12 版本已删除了 split 和 select 方法，可以使用侧流输出代替。

```
DataStreamSource<Integer> numbers = env.fromElements(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
```

```
//将数据打上标签，拆分成奇数和偶数
```

```
SplitStream<Integer> splited = numbers.split(new OutputSelector<Integer>() {
```

```
    @Override
```

```
    public Iterable<String> select(Integer value) {
```

```
        List<String> out = new ArrayList<String>();
```

```
        if (value % 2 == 0) {
```

```
            out.add("偶数"); //将该条数据打上 even 的标签
```

```
        } else {
```

```
            out.add("奇数"); //将该条数据打上 odd 的标签
```

```
        }
```

```
        return out; //返回带有标签的集合
```

```
    }
```

```
});
```

```
//挑选出偶数的数据流
```

```
DataStream<Integer> even = splited.select("偶数");

//挑选出奇数的数据流

DataStream<Integer> odd = splited.select("奇数");

//挑选出奇数、偶数全部的数据流

DataStream<Integer> all = splited.select("even","odd");
```

3.2. 分流操作[使用侧流输出]

```
SingleOutputStreamOperator<Student> mainStream = students.process(new ProcessFunction<Student, Student>()
{
    @Override

    public void processElement(Student student, ProcessFunction<Student, Student>.Context ctx,
Collector<Student> collector) throws Exception {

        if (student.getGender().equals("m")) {

            // 输出到侧流

            ctx.output(maleOutputTag, student);

        } else if (student.getGender().equals("f")) {

            // 输出到侧流

            ctx.output(femaleOutputTag, student.toString());

        } else {

            // 在主流中输出

            collector.collect(student);

        }
    }
});
```

```
}

});

SingleOutputStreamOperator<Student> side1 = mainStream.getSideOutput(maleOutputTag);

SingleOutputStreamOperator<String> side2 = mainStream.getSideOutput(femaleOutputTag);
```

3.3. sideOutput 侧流输出

以下 function 函数，支持将特定数据输出到侧流中：

- ProcessFunction
- KeyedProcessFunction
- CoProcessFunction
- KeyedCoProcessFunction
- ProcessWindowFunction
- ProcessAllWindowFunction



```
DataStream<Integer> input = ...;

final OutputTag<String> outputTag = new OutputTag<String>("side-output"){};

SingleOutputStreamOperator<Integer> mainDataStream = input

    .process(new ProcessFunction<Integer, Integer>() {

        @Override

        public void processElement(

            Integer value,

            Context ctx,
```

```
Collector<Integer> out) throws Exception {  
  
    // emit data to regular output  
  
    out.collect(value);  
  
  
    // emit data to side output  
  
    ctx.output(outputTag, "sideout-" + String.valueOf(value));  
  
}  
  
});  
  
// 通过"主流"获取侧流  
  
DataStream<String> sideOutputStream = mainDataStream.getSideOutput(outputTag);
```



3.4. connect 连接操作

connect 连接 (DataStream, DataStream → ConnectedStreams)

connect 翻译成中文意为连接，可以将两个数据类型一样也可以类型不一样 DataStream 连接成一个新的 ConnectedStreams。需要注意的是，connect 方法与 union 方法不同，虽然调用 connect 方法将两个流连接成一个新的 ConnectedStreams，但是里面的两个流依然是相互独立的，这个方法最大的好处是可以让两个流共享 State 状态，状态相关的内容在后面章节讲解

```
//使用 fromElements 创建两个 DataStream

DataStreamSource<String> word = env.fromElements("a","b","c","d");

DataStreamSource<Integer> num = env.fromElements(1,3,5,7,9);

//将两个 DataStream 连接到一起

ConnectedStreams<String, Integer> connected = word.connect(num);
```



coMap (ConnectedStreams → DataStream)

对 ConnectedStreams 调用 map 方法时需要传入 CoMapFunction 函数；

该接口需要指定 3 个泛型：

- ① 第一个输入 DataStream 的数据类型
- ② 第二个输入 DataStream 的数据类型
- ③ 返回结果的数据类型。
- ④ 该接口需要重写两个方法：
 - ① map1 方法，是对第 1 个流进行 map 的处理逻辑。
 - ② map2 方法，是对 2 个流进行 map 的处理逻辑

这两个方法必须是相同的返回值类型。

```
//将两个 DataStream 连接到一起
```

```
ConnectedStreams<String, Integer> wordAndNum = word.connect(num);

//对 ConnectedStreams 中两个流分别调用个不同逻辑的 map 方法

DataStream<String> result = wordAndNum.map(new CoMapFunction<String, Integer, String>() {

    @Override

    public String map1(String value) throws Exception {

        return value.toUpperCase(); //第一个 map 方法是将第一个流的字符变大写

    }

    @Override

    public String map2(Integer value) throws Exception {

        return String.valueOf(value * 10); //第二个 map 方将是第二个流的数字乘以 10 并转成 String

    }

});
```

coFlatMap (ConnectedStreams → DataStream)

对 ConnectedStreams 调用 flatMap 方法。调用 flatMap 方法，传入的 Function 是 CoFlatMapFunction；这个接口要重写两个方法：

- ① flatMap1 方法，是对第 1 个流进行 flatMap 的处理逻辑；
- ② flatMap2 方法，是对 2 个流进行 flatMap 的处理逻辑；

这两个方法都必须返回是相同的类型。

```
//使用 fromElements 创建两个 DataStream

DataStreamSource<String> word = env.fromElements("a b c", "d e f");

DataStreamSource<String> num = env.fromElements("1,2,3", "4,5,6");

//将两个 DataStream 连接到一起
```

```
ConnectedStreams<String, String> connected = word.connect(num);

//对 ConnectedStreams 中两个流分别调用个不同逻辑的 flatMap 方法

DataStream<String> result = connected.flatMap(new CoFlatMapFunction<String, String, String>() {

    @Override

    public void flatMap1(String value, Collector<String> out) throws Exception {

        String[] words = value.split(" ");

        for (String w : words) {

            out.collect(w);

        }

    }

    @Override

    public void flatMap2(String value, Collector<String> out) throws Exception {

        String[] nums = value.split(",");

        for (String n : nums) {

            out.collect(n);

        }

    }

});
```

3.5. union 合并操作

union 合并 (DataStream * → DataStream)

该方法可以将两个或者多个数据类型一致的 `DataStream` 合并成一个 `DataStream`。`DataStream<T>.union(DataStream<T>... streams)`可以看出 `DataStream` 的 `union` 方法的参数为可变参数，即可以合并两个或多个数据类型一致的 `DataStream`。

下面的例子是使用 `fromElements` 生成两个 `DataStream`，一个是基数的，一个是偶数的，然后将两个 `DataStream` 合并成一个 `DataStream`。

```
//使用 fromElements 创建两个 DataStream

DataStreamSource<Integer> odd = env.fromElements(1,3,5,7,9);

DataStreamSource<Integer> even = env.fromElements(2,4,6,8,10);

//将两个 DataStream 合并到一起

DataStream<Integer> result = odd.union(even);
```

3.6. coGroup 协同分组



`coGroup` 本质上是上述 `join` 算子的底层算子；功能类似；

代码结构如下：

JUST DO IT
只言片语

```
DataStreamSource<String> stream1 = env.fromElements("1,aa,m,18", "2,bb,m,28", "3,cc,f,38");

DataStreamSource<String> stream2 = env.fromElements("1:aa:m:18", "2:bb:m:28", "3:cc:f:38");

DataStream<String> res = stream1.coGroup(stream2).where(new KeySelector<String, String>() {

    @Override

    public String getKey(String value) throws Exception {

        return value;

    }

}).equalTo(new KeySelector<String, String>() {

    @Override

    public String getKey(String value) throws Exception {
```

```
        return value;

    }

    }).window(TumblingEventTimeWindows.of(Time.seconds(5)))

        .apply(new CoGroupFunction<String, String, String>() {

            @Override

            public void coGroup(Iterable<String> first, Iterable<String> second, Collector<String> out)
            throws Exception {

                }

            });
```

3.7.join 关联操作



用于关联两个流（类似于 sql 中 join），需要指定 join 的条件；需要在窗口中进行关联后的逻辑计算；

join 算子的代码结构：

```
stream.join(otherStream)

    .where(<KeySelector>)

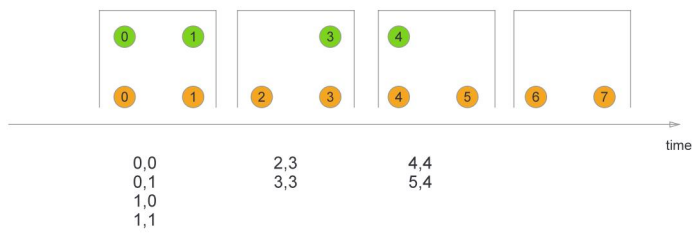
    .equalTo(<KeySelector>)

    .window(<WindowAssigner>)

    .apply(<JoinFunction>)
```

不同窗口 join 的结果示意：

- tumbling window



```
orangeStream.join(greenStream)

    .where(<KeySelector>)

    .equalTo(<KeySelector>)

    .window(SlidingEventTimeWindows.of(Time.milliseconds(2) /* size */, Time.milliseconds(1) /* slide */))

    .apply (new JoinFunction<Integer, Integer, String> (){

        @Override

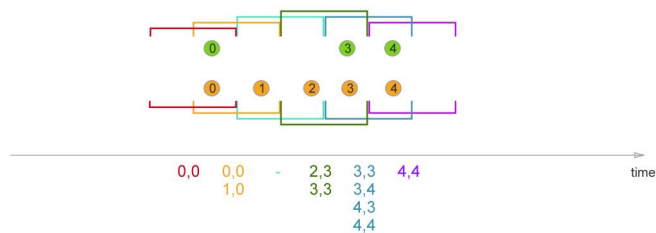
        public String join(Integer first, Integer second) {

            return first + "," + second;

        }

    });
```

● sliding window



```
orangeStream.join(greenStream)
```

```
    .where(<KeySelector>)
```

```
    .equalTo(<KeySelector>)
```

```

.window(SlidingEventTimeWindows.of(Time.milliseconds(2) /* size */, Time.milliseconds(1) /* slide */))

.apply (new JoinFunction<Integer, Integer, String> (){

    @Override

    public String join(Integer first, Integer second) {

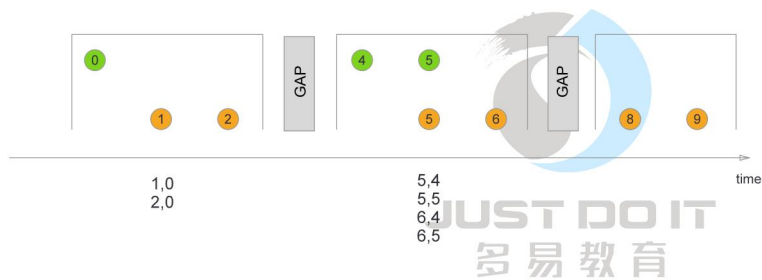
        return first + "," + second;

    }

});

```

● session window



```

orangeStream.join(greenStream)

.where(<KeySelector>)

.equalTo(<KeySelector>)

.window(EventTimeSessionWindows.withGap(Time.milliseconds(1)))

.apply (new JoinFunction<Integer, Integer, String> (){

    @Override

    public String join(Integer first, Integer second) {

        return first + "," + second;

    }

});

```

```
});
```

完整代码示例

```
// 数据流 1

SingleOutputStreamOperator<Student> s1;

// 数据流 2

SingleOutputStreamOperator<StuInfo> s2;

// join 两个流，此时并没有具体的计算逻辑

JoinedStreams<Student, StuInfo> joined = s1.join(s2);

// 对 join 流进行计算处理

DataStream<String> stream = joined

    // where 流 1 的某字段 equalTo 流 2 的某字段

    .where(s -> s.getId()).equalTo(s -> s.getId())

    // join 实质上只能在窗口中进行

    .window(TumblingProcessingTimeWindows.of(Time.seconds(20)))

    // 对窗口中满足关联条件的数据进行计算

    .apply(new JoinFunction<Student, StuInfo, String>() {

        // 这边传入的两个流的两条数据，是能够满足关联条件的

        @Override

        public String join(Student first, StuInfo second) throws Exception {

            // first: 左流数据 ; second: 右流数据
```

```
// 计算逻辑

// 返回结果

return

}

});

// 对 join 流进行计算处理

joined.where(s -> s.getId()).equalTo(s -> s.getId())

.window(TumblingProcessingTimeWindows.of(Time.seconds(20)))

.apply(new FlatJoinFunction<Student, StuInfo, String>() {

    @Override

    public void join(Student first, StuInfo second, Collector<String> out) throws Exception {

        out.collect();

    }

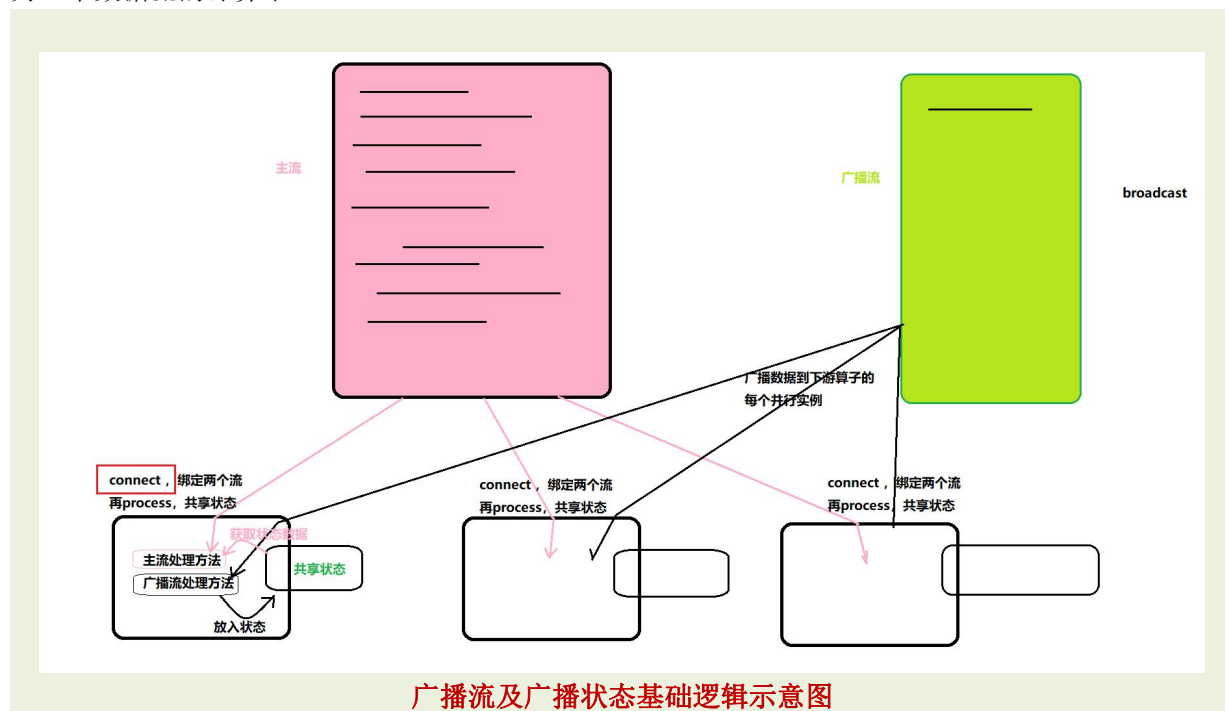
});
```

3.8. broadcast 广播

Broadcast State 是 Flink 1.5 引入的新特性。

在开发过程中，如果遇到需要下发/广播配置、规则等低吞吐事件流到下游所有 task 时，就可以使用

Broadcast State 特性。下游的 task 接收这些配置、规则并保存为 BroadcastState, 将这些配置应用到另一个数据流的计算中。



API 介绍 ， 核心要点

- 将需要广播出去的流，调用 broadcast 方法进行广播转换，得到广播流 BroadCastStream
- 然后在主流上调用 connect 算子，来连接广播流（以实现广播状态的共享处理）
- 在连接流上调用 process 算子，就会在同一个 ProcessFunction 中提供两个方法分别对两个流进行处理，并在这个 ProcessFunction 内实现“广播状态”的共享

```
/**
 * @Author: deep as the sea
 *
 * @Site: www.51doit.com
 *
 * @QQ: 657270652
 *
 * @Date: 2022/4/26
 *
 * @Desc: 广播流及广播状态的使用示例
 */

public class _16_BroadCast_Demo {
```

```
public static void main(String[] args) throws Exception {

    Configuration configuration = new Configuration();

    configuration.setInteger("rest.port", 8822);

    StreamExecutionEnvironment env =
StreamExecutionEnvironment.createLocalEnvironmentWithWebUI(configuration);

    env.setParallelism(1);

    // id,eventId

    DataStreamSource<String> stream1 = env.socketTextStream("localhost", 9998);

    SingleOutputStreamOperator<Tuple2<String, String>> s1 = stream1.map(s -> {

        String[] arr = s.split(",");

        return Tuple2.of(arr[0], arr[1]);

    }).returns(new TypeHint<Tuple2<String, String>>() {

    });

    // id,age,city

    DataStreamSource<String> stream2 = env.socketTextStream("localhost", 9999);
```

```
SingleOutputStreamOperator<Tuple3<String, String, String>> s2 = stream2.map(s -> {

    String[] arr = s.split(",");

    return Tuple3.of(arr[0], arr[1], arr[2]);

}).returns(new TypeHint<Tuple3<String, String, String>>() {

});

/**

 * 案例背景：

 *   流 1：  用户行为事件流（持续不断，同一个人也会反复出现，出现次数不定

 *   流 2：  用户维度信息（年龄，城市），同一个人的数据只会来一次，来的时间也不定（作为广播流）

 *

 *   需要加工流 1，把用户的维度信息填充好，利用广播流来实现

 */

// 将字典数据所在流： s2 ， 转成 广播流

MapStateDescriptor<String, Tuple2<String, String>> userInfoStateDesc = new
MapStateDescriptor<>("userInfoStateDesc", TypeInformation.of(String.class), TypeInformation.of(new
TypeHint<Tuple2<String, String>>() {}));

BroadcastStream<Tuple3<String, String, String>> s2BroadcastStream =
s2.broadcast(userInfoStateDesc);

// 哪个流处理中需要用到广播状态数据，就要 去 连接 connect 这个广播流
```

```
BroadcastConnectedStream<Tuple2<String, String>, Tuple3<String, String, String>> connected =
s1.connect(s2BroadcastStream);

/**

* 对 连接了广播流之后的 ”连接流“ 进行处理

* 核心思想:

* 在 processBroadcastElement 方法中, 把获取到的广播流中的数据, 插入到 “广播状态”中

* 在 processElement 方法中, 对取到的主流数据进行处理 (从广播状态中获取要拼接的数据, 拼接后输出)

*/

SingleOutputStreamOperator<String> resultStream = connected.process(new
BroadcastProcessFunction<Tuple2<String, String>, Tuple3<String, String, String>, String>() {

    /*BroadcastState<String, Tuple2<String, String>> broadcastState;*/

    /**

    * 本方法, 是用来处理 主流中的数据 (每来一条, 调用一次)

    * @param element 左流 (主流) 中的一条数据

    * @param ctx 上下文

    * @param out 输出器

    * @throws Exception

    */

    @Override
```

```
public void processElement(Tuple2<String, String> element,
BroadcastProcessFunction<Tuple2<String, String>, Tuple3<String, String, String>, String>.ReadOnlyContext
ctx, Collector<String> out) throws Exception {

    // 通过 ReadOnlyContext ctx 取到的广播状态对象，是一个“只读”的对象；

    ReadOnlyBroadcastState<String, Tuple2<String, String>> broadcastState =
ctx.getBroadcastState(userInfoStateDesc);

    if (broadcastState != null) {

        Tuple2<String, String> userInfo = broadcastState.get(element.f0);

        out.collect(element.f0 + "," + element.f1 + "," + (userInfo == null ? null : userInfo.f0)
+ "," + (userInfo == null ? null : userInfo.f1));

    } else {

        out.collect(element.f0 + "," + element.f1 + "," + null + "," + null);

    }

}

/**
 *
 * @param element 广播流中的一条数据
 * @param ctx 上下文
 * @param out 输出器
 * @throws Exception
```

```
        */

        @Override

        public void processBroadcastElement(Tuple3<String, String, String> element,
BroadcastProcessFunction<Tuple2<String, String>, Tuple3<String, String, String>, String>.Context ctx,
Collector<String> out) throws Exception {

            // 从上下文中，获取广播状态对象（可读可写的状态对象）

            BroadcastState<String, Tuple2<String, String>> broadcastState =
ctx.getBroadcastState(userInfoStateDesc);

            // 然后将获得的这条 广播流数据， 拆分后，装入广播状态

            broadcastState.put(element.f0, Tuple2.of(element.f1, element.f2));

        }

    });

    resultStream.print();

    env.execute();

}

}
```

4. Flink 编程 process function

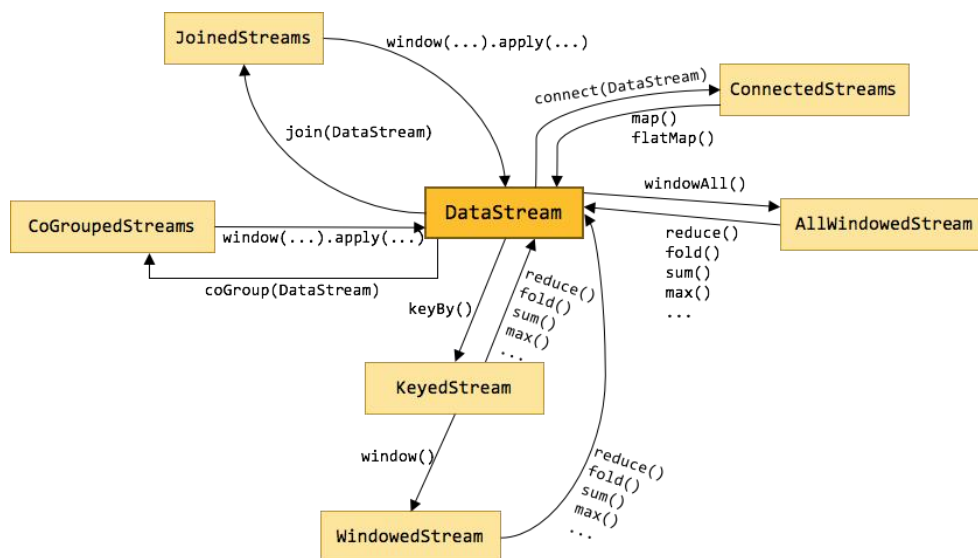
4.1. process function 概述

process function 相对于前文所述的 map、flatMap、filter 算子来说，最大的区别是其让开发人员对数据的处理逻辑拥有更大的自由度；同时，ProcessFunction 继承了 RichFunction，因而具备了 getRuntimeContext()，open()，close()等方法；

在不同类型的 datastream 上，（比如 keyed stream、windowedStream、ConnectedStream 等），应用 process function 时，flink 提供了大量不同类型的 process function，让其针对不同的 datastream 拥有更具针对性的功能；

- ProcessFunction （普通 DataStream 上调 process 时）
- KeyedProcessFunction （KeyedStream 上调 process 时）
- ProcessWindowFunction （WindowedStream 上调 process 时）
- ProcessAllWindowFunction （AllWindowedStream 上调 process 时）
- CoProcessFunction （ConnectedStreams 上调 process 时）
- ProcessJoinFunction （JoinedStreams 上调 process 时）
- BroadcastProcessFunction （BroadcastConnectedStreams 上调 process 时）
- KeyedBroadcastProcessFunction （KeyedBroadcastConnectedStreams 上调 process 时）

各种算子运算后所生成的 datastream 类型，及各种 datastream 类型之间的互相转换关系



4.2. Process function 代码示例

```
/**  
  
 * @Author: deep as the sea  
  
 * @Site: www.51doit.com  
  
 * @QQ: 657270652  
  
 * @Date: 2022/4/26  
  
 * @Desc: process 算子及 ProcessFunction 示例  
  
 *  
 * 在不同类型的 数据流上，调用 process 算子时，所需要传入的 ProcessFunction 也会有不同  
  
 **/  
  
public class _17_ProcessFunctions_Demo {  
  
    public static void main(String[] args) throws Exception {  
  
        Configuration configuration = new Configuration();  
  
        configuration.setInteger("rest.port", 8822);  
  
        StreamExecutionEnvironment env =  
        StreamExecutionEnvironment.createLocalEnvironmentWithWebUI(configuration);  
  
        env.setParallelism(1);  
  
        // id,eventId
```

```
DataStreamSource<String> stream1 = env.socketTextStream("localhost", 9998);

/**
 * 在普通的 datastream 上调用 process 算子，传入的是 "ProcessFunction"
 */

SingleOutputStreamOperator<Tuple2<String, String>> s1 = stream1.process(new
ProcessFunction<String, Tuple2<String, String>>() {

    // 可以使用 生命周期 open 方法

    @Override

    public void open(Configuration parameters) throws Exception {

        // 可以调用 getRuntimeContext 方法拿到各种运行时上下文信息

        RuntimeContext runtimeContext = getRuntimeContext();

        runtimeContext.getTaskName();

        super.open(parameters);

    }

    @Override

    public void processElement(String value, ProcessFunction<String, Tuple2<String,
String>>.Context ctx, Collector<Tuple2<String, String>> out) throws Exception {

        // 可以做测流输出

        ctx.output(new OutputTag<String>("s1", Types.STRING), value);
```

```
// 可以做主流输出

String[] arr = value.split(",");

out.collect(Tuple2.of(arr[0], arr[1]));

}

// 可以使用 生命周期 close 方法

@Override

public void close() throws Exception {

    super.close();

}

});

/**

 * 在 keyedStream 上调用 process 算子，传入的是 "KeyedProcessFunction"

 * KeyedProcessFunction 中的 ， 泛型 1： 流中的 key 的类型； 泛型 2： 流中的数据的数据的类型 ； 泛型 3： 处

理后的输出结果的类型

 */

// 对 s1 流进行 keyby 分组

KeyedStream<Tuple2<String, String>, String> keyedStream = s1.keyBy(tp2 -> tp2.f0);

// 然后在 keyby 后的数据流上调用 process 算子

SingleOutputStreamOperator<Tuple2<Integer, String>> s2 = keyedStream.process(new

KeyedProcessFunction<String, Tuple2<String, String>, Tuple2<Integer, String>>() {
```

```
@Override

    public void processElement(Tuple2<String, String> value, KeyedProcessFunction<String,
Tuple2<String, String>, Tuple2<Integer, String>>.Context ctx, Collector<Tuple2<Integer, String>> out)
throws Exception {

        // 把 id 变整数, 把 eventId 变大写

        out.collect(Tuple2.of(Integer.parseInt(value.f0), value.f1.toUpperCase()));

    }

});

s2.print();

env.execute();

}

}
```

多易教育

5. 并行度相关概念和 API

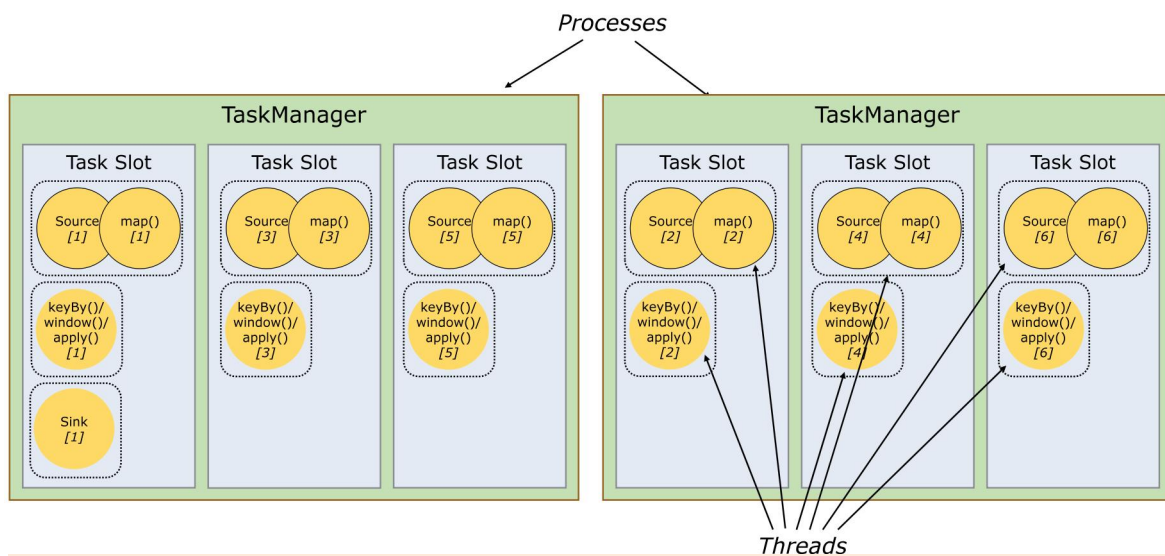
5.1. 基础概念

- 用户通过算子 api 所开发的代码, 会被 flink 任务提交客户端解析成 jobGraph
- 然后, jobGraph 提交到集群 JobManager, 转化成 ExecutionGraph (并行化后的执行图)
- 然后, ExecutionGraph 中的各个 task 会以多并行实例 (subTask) 部署到 taskmanager 上执行;
- subTask 运行的位置是 taskmanager 所提供的槽位 (task slot), 槽位简单理解就是线程;

重要提示

- 一个算子的逻辑, 可以封装在一个独立的 task 中 (可以有多个运行时实例: subTask);
- 也可把多个算子的逻辑 chain 在一起后封装在一个独立的 task 中 (可以有多个运行时实例);

```
subTask) ;
```



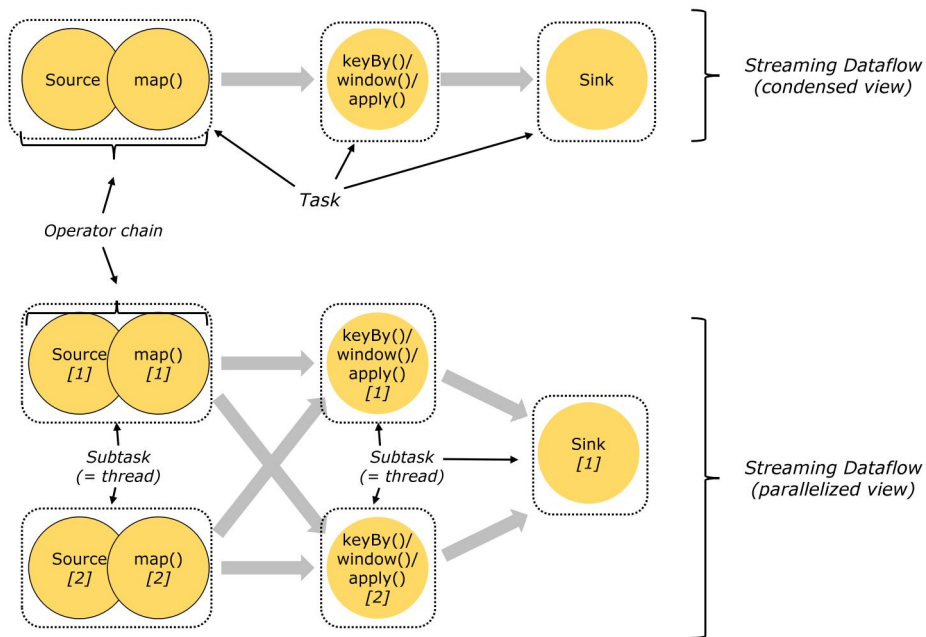
同一个 task 的不同运行实例，必须放在不同的 task slot 上运行；
同一个 task slot，可以运行多个不同 task 的各个并行实例；

5.2.task 与算子链（operator chain）

上下游算子，能否 chain 在一起，放在一个 Task 中，取决于如下 3 个条件：

- 上下游算子实例间是 oneToOne 数据传输（forward）；
- 上下游算子并行度相同；
- 上下游算子属于相同的 slotSharingGroup（槽位共享组）；

3 个条件都满足，才能合并为一个 task；否则不能合并成一个 task；



当然，即使满足上述 3 个条件，也不一定就非要把上下游算子绑定成算子链；

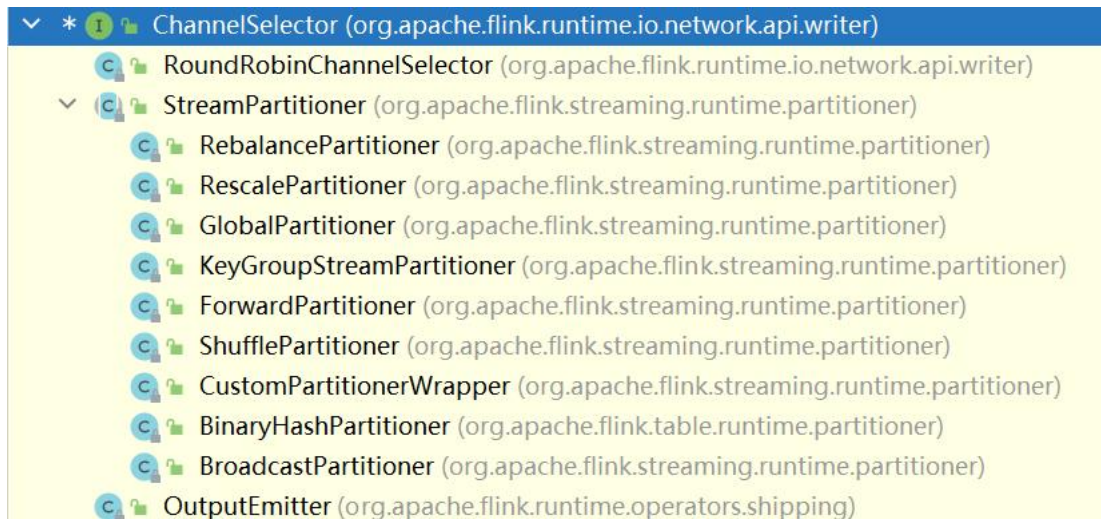
flink 提供了相关的 api，来让用户可以根据自己的需求，进行灵活的算子链合并或拆分；

- setParallelism 设置算子的并行度
- slotSharingGroup 设置算子的槽位共享组
- disableChaining 对算子禁用前后链合并
- startNewChain 对算子开启新链（即禁用算子前链合并）

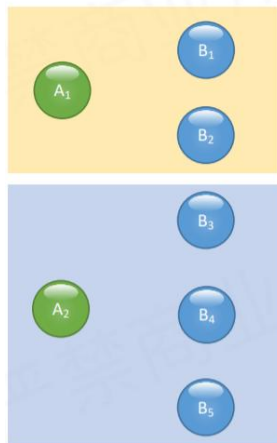
5.3. 分区 partition 算子

分区算子：用于指定上游 task 的各并行 subtask 与下游 task 的 subtask 之间如何传输数据：

Flink 中，对于上下游 subTask 之间的数据传输控制，由 ChannelSelector 策略来控制，而且 Flink 内针对各种场景，开发了众多 ChannelSelector 的具体实现



设置数据传输策略时，不需要显式指定 partitioner，而是调用封装好的算子即可：



类型	描述
<code>dataStream.global();</code>	全部发往第1个task
<code>dataStream.broadcast();</code>	广播
<code>dataStream.forward();</code>	上下游并发度一样时一对一发送
<code>dataStream.shuffle();</code>	随机均匀分配
<code>dataStream.rebalance();</code>	Round-Robin（轮流分配）
<code>dataStream.recale();</code>	Local Round-Robin（本地轮流分配）
<code>dataStream.partitionCustom()</code>	自定义单播

`dataStream.keyBy(KeySelector)` 根据 key 的 hashCode 来进行 hash 分发

默认情况下，flink 会优先使用 REBALANCE 分发策略

6. Flink 时间语义

6.1. 三种时间概念

在实时流式计算中，“时间”是一个能影响计算结果的非常重要因素！

试想场景：每隔 1 分钟计算一次最近 10 分钟的活跃用户量：

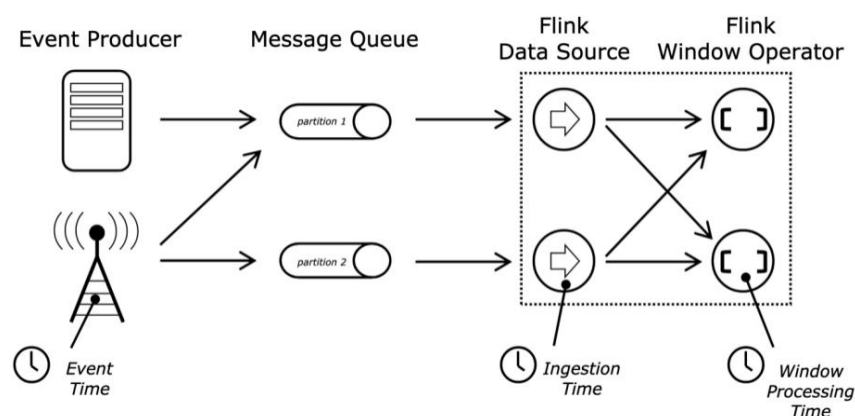
①假设此刻的时间是 13:10，要计算的活跃用户量时间段为：[13:00,13:10)；

②有一条行为日志中记录的用户的行为时间是 12:59，但到达 flink 计算程序时已是 13:02；

那么，这个用户是否要纳入本次计算的结果中呢？看如何定义：

①如果时段 [13:00, 13:10) 定义的是用户行为的发生时间（数据中的业务时间），则不应纳入；

②如果时段 [13:00, 13:10) 定义的是计算时的时间，则应该纳入；



flink 内部为了直观地统一计算时所用的时间标准，特制定了两种时间语义：

- processing time 处理时间
- event time 事件时间
- Ingestion time 注入时间

时间语义主要影响“窗口计算”；

6.2. 两种时间语义

时间语义，是 flink 中用于时间推进和时间判断的机制；

时间推进和时间判断，以什么为标准，就产出了两种不同的时间语义：

- 以 processing time 为依据，则叫做**处理时间语义**
- 以 event time 为依据，则叫做**事件时间语义**

时间语义的设计意义

```
process(EventLog eventlog){
    Long eventTime = eventLog.getTimestamp();
    Long processTime = System.currentTimeMillis()
    // 用户完全可以自己根据需求中的时间定义来进行相应的计算
}
```

Flink 为什么还要搞出一个 “事件时间语义”：时间按数据中的业务时间戳来推进！

主要是，实时流式计算中，有大量跟时间相关的统计需求，比如：时间窗口计算，定时器等，而这些需求，如果都让用户像上面的代码那样自己去进行判断、处理，那么它觉得自己的 api 不够强大！

所以，flink 想在 api 的层面，将两类时间定义的计算需求进行 api 层面的统一，它才搞出这么一种“事件时间语义”，有了这种语义，那么，处理时间 和 事件时间，都可以看成 “时间”

用户在不同时间定义下，要进行一个定时动作时，就不需要再像上面的代码那种去进行各种判断，而是一个**统一的动作**：到 xxx 时间，给我做个什么事！

```
process(EventLog eventlog, Timestamp timestamp){
    // 不管需求是需要用哪种时间来计算，用户代码只需要看到一个 timestamp 了
}
```

代码中的 timestamp 到底是事件时间，还是处理时间，取决于环境中设置的“时间语义”

处理时间（processing time）语义

Processing Time 是指数据被 Operator 处理时所在机器的系统时间。

处理时间遵循客观世界中时间的特性：单调递增，恒定速度，永不停滞，永不回退；

事件时间（event time）语义

Event Time 是指在数据本身的业务时间（如用户行为日志中的用户行为时间戳）；

Event Time 语义中，时间的推进完全由流入 flink 系统的数据来驱动：

数据中的业务时间推进到哪，flink 就认为自己的时间推进到了哪；

它可能停滞，也可能速度不恒定，**但也一定是单调递增不可回退！**

6.3. 时间语义的设置

老版 api 中指定时间语义

1.12 以前, flink 默认以 processing time 作为默认的时间语义; 可以在 env 上设置所想要的时间语义;

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

//设置 EventTime 作为时间标准

env.setStreamTimeCharacteristic(TimeCharacteristic.EventTime);

//设置 IngestionTime 作为时间标准

//env.setStreamTimeCharacteristic(TimeCharacteristic.IngestionTime);

//设置 ProcessingTime 作为时间标准

//env.setStreamTimeCharacteristic(TimeCharacteristic.ProcessingTime);
```

新版 api 中指定时间语义

1.12 及以后, flink 以 event time 作为默认的时间语义, 并 deprecated 了上述设置 api;

在需要指定时间语义的相关操作 (如时间窗口) 时, **可以通过显式的 api 来使用特定的时间语义;**

```
keyedStream.window(SlidingEventTimeWindows.of(Time.seconds(5), Time.seconds(1)));

keyedStream.window(SlidingProcessingTimeWindows.of(Time.seconds(5), Time.seconds(1)));

keyedStream.window(TumblingEventTimeWindows.of(Time.seconds(5)));

keyedStream.window(TumblingProcessingTimeWindows.of(Time.seconds(5)));
```

新版 api 中禁用事件时间语义

如果需要禁用 event time 机制, 则可以通过设置 watermark 生成频率间隔来实现:

```
// 如果设置为 0, 则禁用了 watermark 的生成; 从而失去了 event time 语义

ExecutionConfig.setAutoWatermarkInterval(0L);
```

提示: 如果需要使用已过期的 ingestion time, 可以通过设置恰当的 watermark 来实现;

7. 事件时间语义中的 watermark

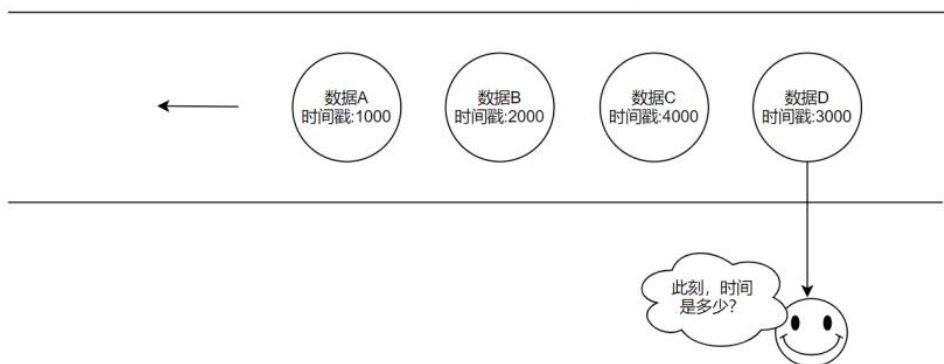
7.1. 事件时间推进的困难

由于在事件时间语义的世界观中，时间是由流入系统的数据（事件）而推进的；

而事件时间，并不能像处理时间那样，由宇宙客观规律以恒定速度，不可停滞地推进；

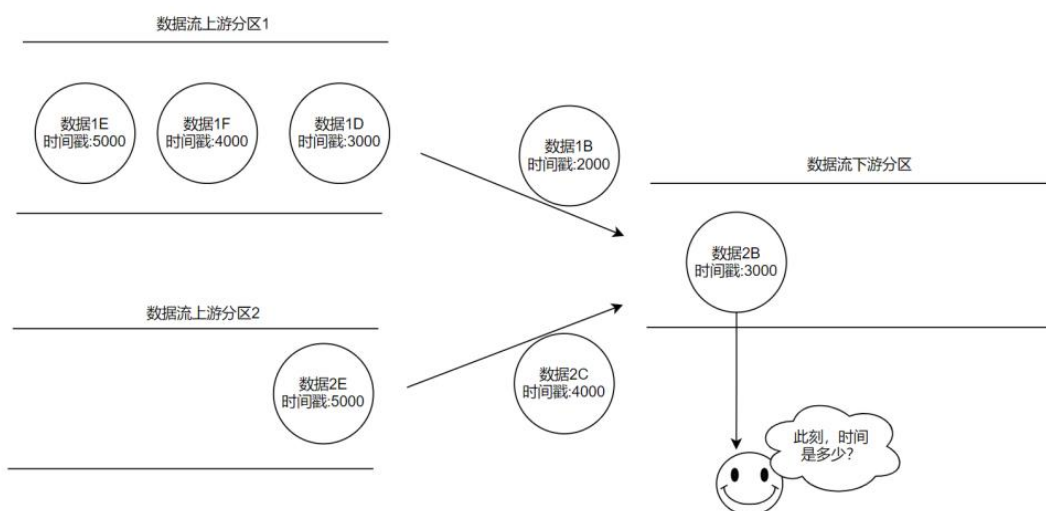
从而，在事件时间语义的世界观中，时间的推进，并不是一件显而易见的事情；

● 场景 1:



数据时间存在乱序的可能性，但时光不能倒流啊！

● 场景 2:



下游分区接收上游多个分区的数据，数据时间错落有致，那以谁为准？！

7.2. watermark 来推进时间

- 所谓 watermark，就是在事件时间语义世界观中，用于单调递增向前推进时间的一种标记；
- 它的核心机制是在数据流中周期性地插入一种时间戳单调递增的特殊数据元素（watermark），来不可逆转地在整个数据流中进行时间的推进；

Watermark 是从某一个算子实例（源头）开始，根据数据中的事件时间，来周期性地产生，并插入到数据流中，持续不断地往下游传递，以推进整个计算链条上各个算子实例的时间！

// watermark 的生成周期（默认值即为 200ms）

```
env.getConfig().setAutoWatermarkInterval(200);
```

watermark 源码片段

```
public final class Watermark implements Serializable {

    private final long timestamp;

    public Watermark(long timestamp) {

        this.timestamp = timestamp;

    }

    public long getTimestamp() {

        return timestamp;

    }

}
```

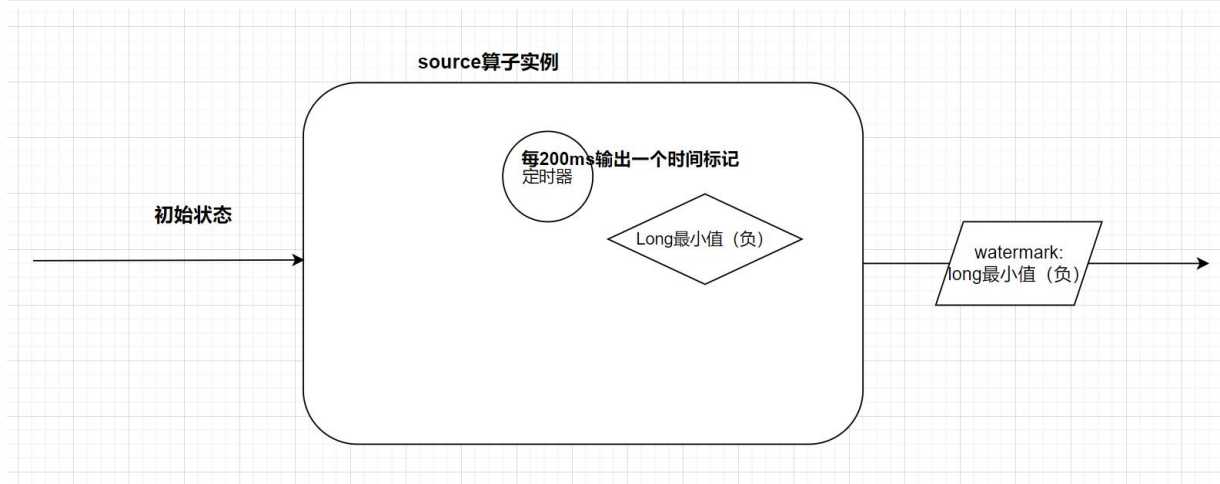
```
public String getFormattedTimestamp() {  
  
    return TS_FORMATTER.get().format(new Date(timestamp));  
  
}  
  
}
```

- watermark，本质上也是 flink 中各算子间流转的一种标记数据，只不过与用户的数据不同，它是 flink 内部自动产生并插入到数据流的；
- 它本身所携带的信息很简单，就是一个时间戳！

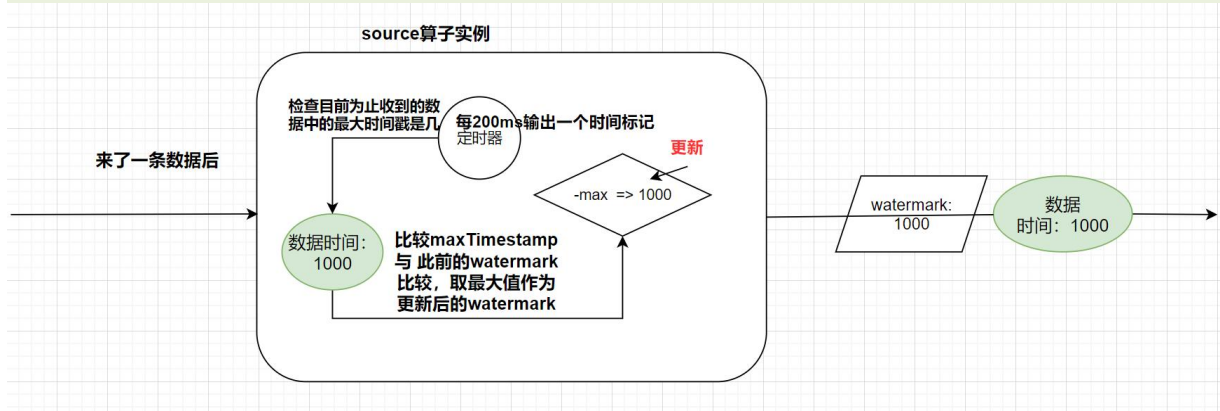


watermark 产生源头示意图

● 初始状态



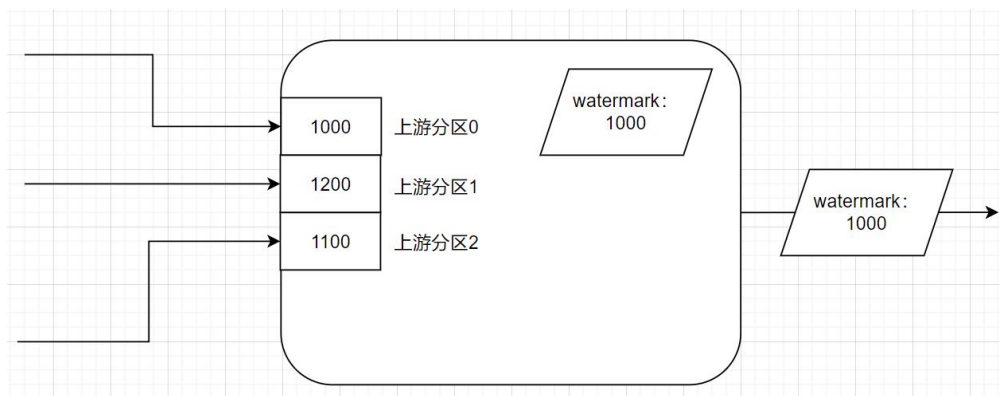
● 收到一条新数据后



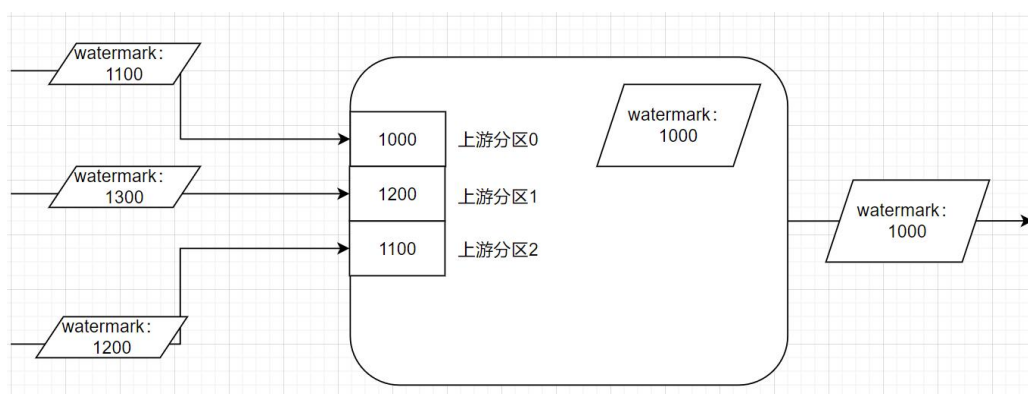
简单说，就是在 watermark 产生的源头算子实例中，实例程序会用一个定时器，去周期性地检查截止此刻所收到过的数据的事件时间最大值，如果超过了之前的最大值，则将这个最大值更新为最新的 watermark，并向下游传递；

watermark 往下游推进的示意图

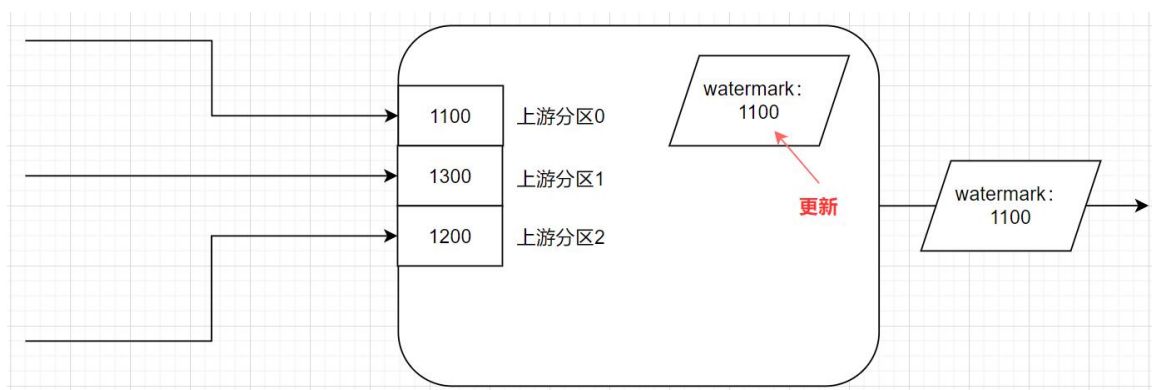
● 初始状态



● 新的上游 watermark 即将到达



● 上游的新 watermark 最终产生的效果

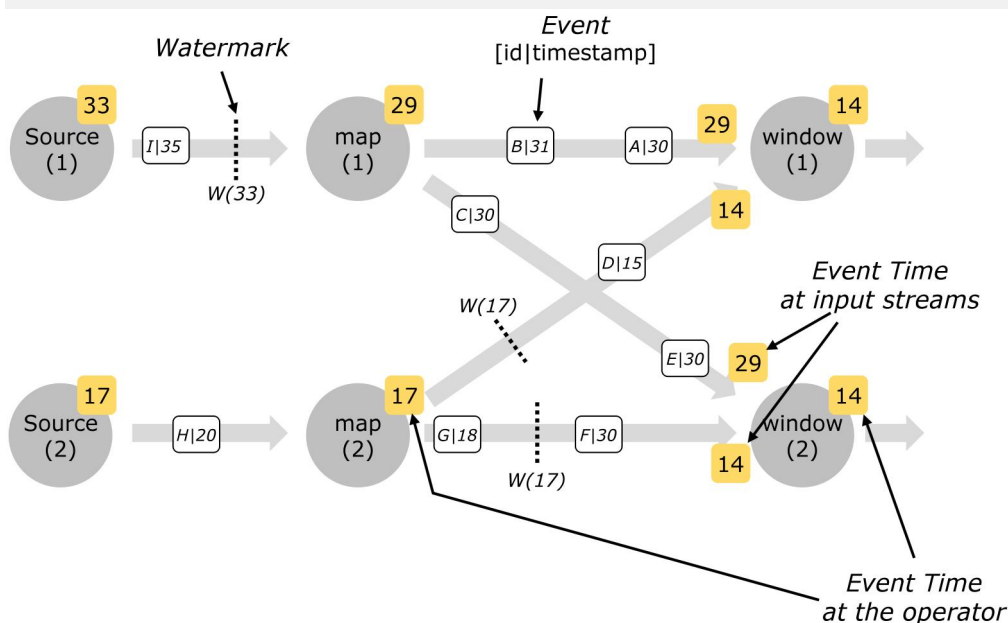


一个下游算子实例，如果消费着多个上游算子实例：

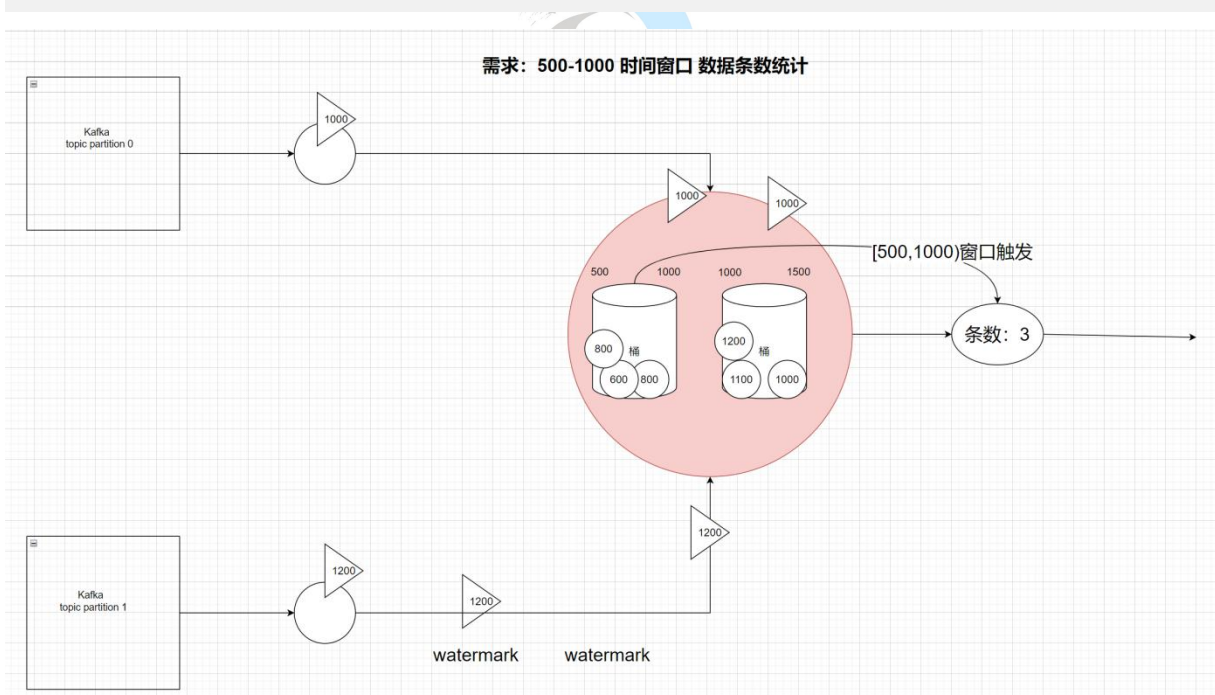
则选择 “Min(游各实例的最新 watermark)” 作为自己当前的 watermark；

并将自己最新的 watermark 往下游广播；

watermark 从源头到下游推进完整示意图



watermark 推进事件时间与窗口计算的结合示例



从这里可以看出，如果某一个上游实例 watermark 一直停滞，则会导致下游实例的 watermark 也一直停滞，从而迟滞窗口计算的触发，并造成大量数据的积压；

对此，flink 提供了一个机制：设置 watermark 的 idle 超时：如果某个分区超过 idle 时长没有收到数据，则会自主往前推进时间；从而不会导致下游的 watermark 一直停滞不前！

7.3. 内置 watermark 生成策略

watermark 生成时机策略

此前有两种生成时机策略：

- `AssignerWithPeriodicWatermarks` 周期性生成 watermark
- `AssignerWithPunctuatedWatermarks` 已过期 按指定标记性事件生成 watermark

在 flink1.12 后，watermark 默认是按固定频率周期性地产生；

watermark 的生成值算法策略

- 紧跟最大事件时间的 watermark 生成策略（完全不容忍乱序）

```
WatermarkStrategy.forMonotonousTimestamps();
```

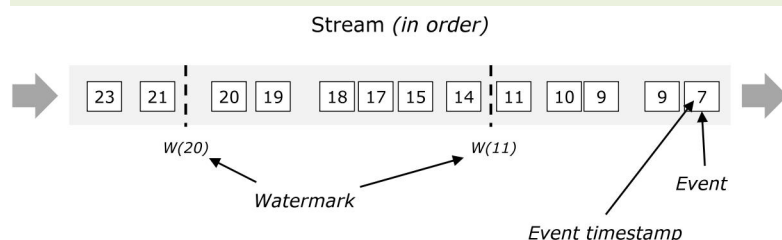
- 允许乱序的 watermark 生成策略（最大事件时间-容错时间）

```
WatermarkStrategy.forBoundedOutOfOrderness(Duration.ofSeconds(10)); // 根据实际数据的最大乱序情况来设置
```

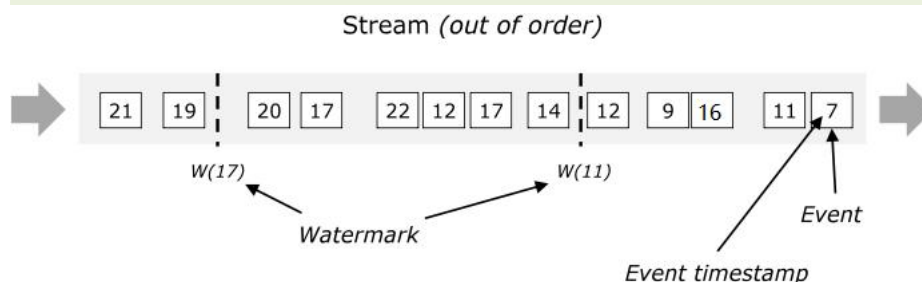
- 自定义 watermark 生成策略

```
WatermarkStrategy.forGenerator(new WatermarkGenerator(){ ... } );
```

- Monotonous 策略的时间推进示意图



- BoundedOutOfOrderness 策略的时间推进示意图 watermark = MaxEventTime - 5)



7.4. 设置 watermark 策略的代码模板

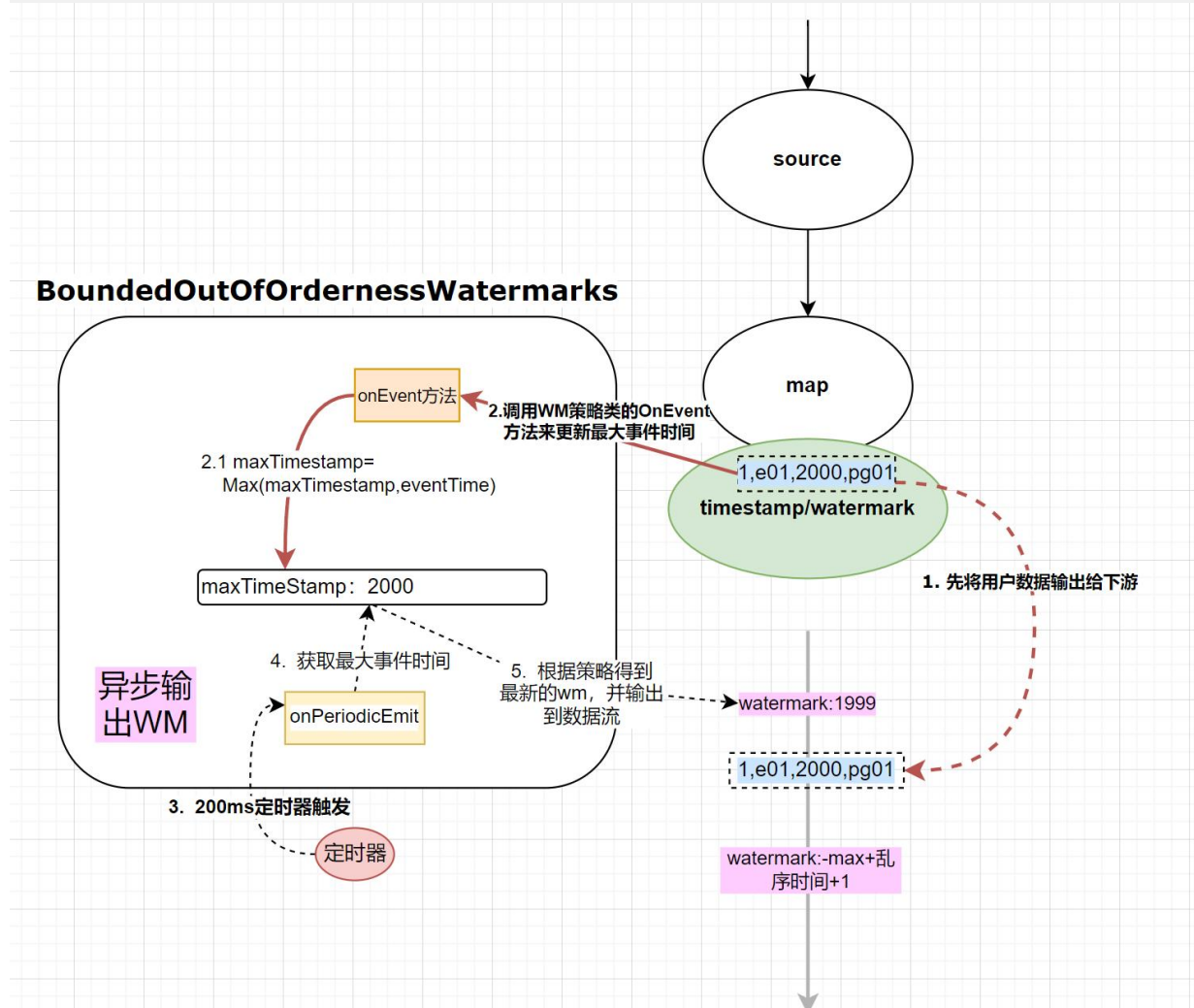
```
SingleOutputStreamOperator<Score> stream1 = source.map(line -> {  
  
    String[] arr = line.split(",");  
  
    return new Score(...);  
  
}).assignTimestampsAndWatermarks(  
  
    WatermarkStrategy.<Score>forBoundedOutOfOrderness(Duration.ZERO)  
  
        .withTimestampAssigner((element, recordTimestamp) -> element.getTimeStamp())  
  
        // 防止上游某些分区的水位线不推进导致下游的窗口一直不触发（这个分区很久都没数据）  
  
        .withIdleness(Duration.ofMillis(2000))  
  
);
```



7.5. watermark 源码分析

7.5.1. WaterMark 生成算子工作机制

背景代码: `source.map(s->bean).assignWatermarkAndTimestamps().process().print();`



7.5.2. WaterMark 生成器

WatermarkGenerator 接口

@Public

```
public interface WatermarkGenerator<T> {
```

```
/**

 * Called for every event, allows the watermark generator to examine

 * and remember the event timestamps, or to emit a watermark based on

 * the event itself.

 */

void onEvent(T event, long eventTimestamp, WatermarkOutput output);


/**

 * Called periodically, and might emit a new watermark, or not.

 *

 * <p>The interval in which this method is called and Watermarks

 * are generated depends on {@link ExecutionConfig#getAutoWatermarkInterval()}.

 */

void onPeriodicEmit(WatermarkOutput output);

}
```

BoundedOutOfOrdernessWatermarks 乱序水印实现

```
public class BoundedOutOfOrdernessWatermarks<T> implements WatermarkGenerator<T> {

    /** The maximum timestamp encountered so far. */

    private long maxTimestamp;
```

```
/** The maximum out-of-orderness that this watermark generator assumes. */

private final long outOfOrdernessMillis;

/**
 * Creates a new watermark generator with the given out-of-orderness bound.
 *
 * @param maxOutOfOrderness The bound for the out-of-orderness of the event timestamps.
 */

public BoundedOutOfOrdernessWatermarks(Duration maxOutOfOrderness) {

    checkNotNull(maxOutOfOrderness, "maxOutOfOrderness");

    checkArgument(!maxOutOfOrderness.isNegative(), "maxOutOfOrderness cannot be negative");

    this.outOfOrdernessMillis = maxOutOfOrderness.toMillis();

    // start so that our lowest watermark would be Long.MIN_VALUE.

    this.maxTimestamp = Long.MIN_VALUE + outOfOrdernessMillis + 1;
}

// -----

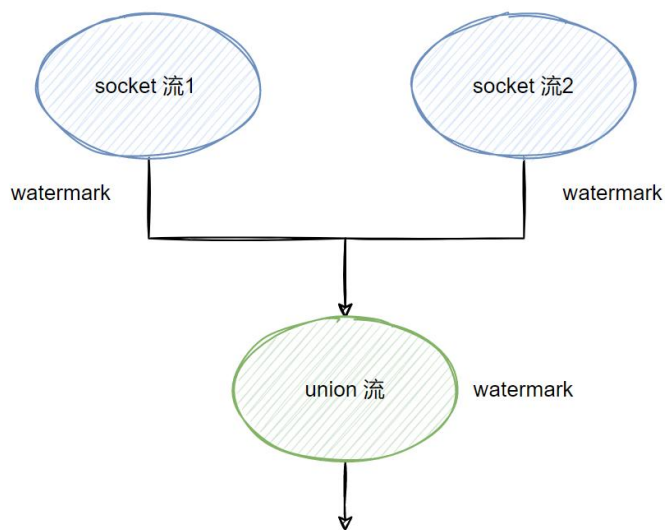
@Override
```

```
public void onEvent(T event, long eventTimestamp, WatermarkOutput output) {  
  
    // 视当前的事件时间戳，更新（或不更新）maxTimestamp  
  
    maxTimestamp = Math.max(maxTimestamp, eventTimestamp);  
  
}  
  
@Override  
  
public void onPeriodicEmit(WatermarkOutput output) {  
  
    // 以当前 eventTimestamp-乱序延迟数-1，作为生成的 watermark 值  
  
    output.emitWatermark(new Watermark(maxTimestamp - outOfOrdernessMillis - 1));  
  
}  
  
}
```

7.5.3. watermark 的测试观察

测试思路

构造两条单并行度流，合并成一个单并行度流，来 watermark，及重点观察“接收多个上游分区”的算子的 watermark 推进规律；



测试代码

```
/**  
  
 * @author 深海 "deep as the sea"  
  
 * @date 2022/4/10  
  
 * @contact qq:657270652 wx:doitedu2018  
  
 * @site 多易教育 www.51doit.cn  
  
 * @desc watermark 推进测试观察  
  
 **/  
  
public class WatermarkTest {  
  
    public static void main(String[] args) throws Exception {  
  
        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();  
  
        env.setParallelism(1);  
  
        // 构造一个无乱序延迟的 watermark 生成策略
```

```
WatermarkStrategy<String> stringWatermarkStrategy =
WatermarkStrategy.<String>forMonotonousTimestamps().withTimestampAssigner(new
SerializableTimestampAssigner<String>() {

    @Override

    public long extractTimestamp(String element, long recordTimestamp) {

        return Long.parseLong(element.split(",")[1]);

    }

});

// 构造单并行度流 1

DataStreamSource<String> source1 = env.socketTextStream("localhost", 9988);

SingleOutputStreamOperator<String> s1 =
source1.assignTimestampsAndWatermarks(stringWatermarkStrategy);

// 构造单并行度流 2

DataStreamSource<String> source2 = env.socketTextStream("localhost", 9999);

SingleOutputStreamOperator<String> s2 =
source2.assignTimestampsAndWatermarks(stringWatermarkStrategy);

// 两条单并行度流，合并到一条单并行度流

DataStream<String> s = s1.union(s2);

// 打印 watermark 信息，观察 watermark 推进情况
```

```
s.process(new ProcessFunction<String, String>() {  
  
    @Override  
  
    public void processElement(String value, Context ctx, Collector<String> out) throws Exception  
    {  
  
        // 获取当前的 watermark（注意：此处所谓当前 watermark 是指处理当前数据前的 watermark）  
  
        long currentWatermark = ctx.timerService().currentWatermark();  
  
        Long timestamp = ctx.timestamp();  
  
        System.out.println("s"+ " : " + timestamp + " : " + currentWatermark + " => "+value);  
  
        out.collect(value);  
  
    }  
  
}).print();  
  
env.execute();  
  
}
```

测试结果

在 9988 端口输入数据: a,1000

在 9999 端口输入数据: b,1000 b,4000 b,5000

输出结果:

s: 1000 : -9223372036854775808 => a,1000

s: 1000 : -9223372036854775808 => b,1000

s: 4000 : 999 => b,4000

s: 5000 : 999 => b,5000

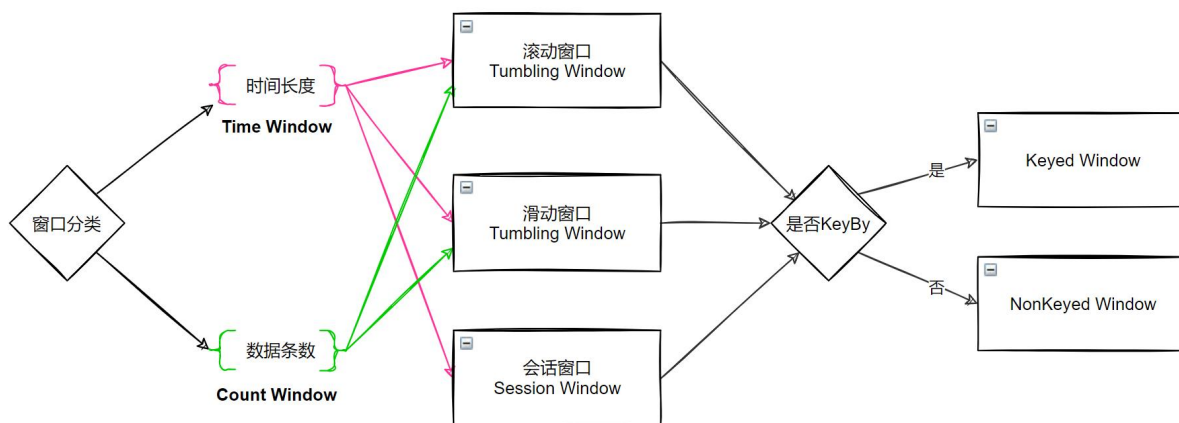


8. Flink 窗口计算 API

8.1. 窗口（window）概念

窗口，就是把无界的数据流，依据一定规则划分成一段一段的有界数据流来计算；

既然划分成有界数据段，通常都是为了"聚合"；

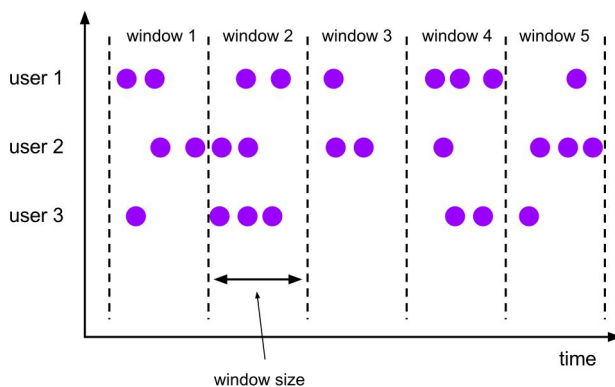


Keyedwindow 重要特性：任何一个窗口，都绑定在自己所属的 key 上；不同 key 的数据肯定不会划分到相同窗口中去！

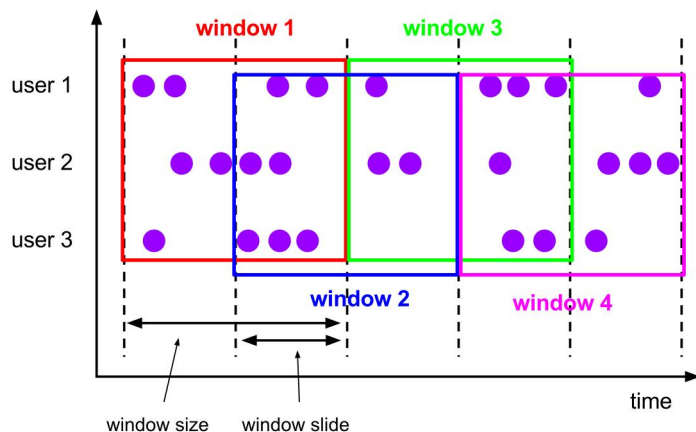
JUST DO IT
多易教育

8.2. 窗口细分类型

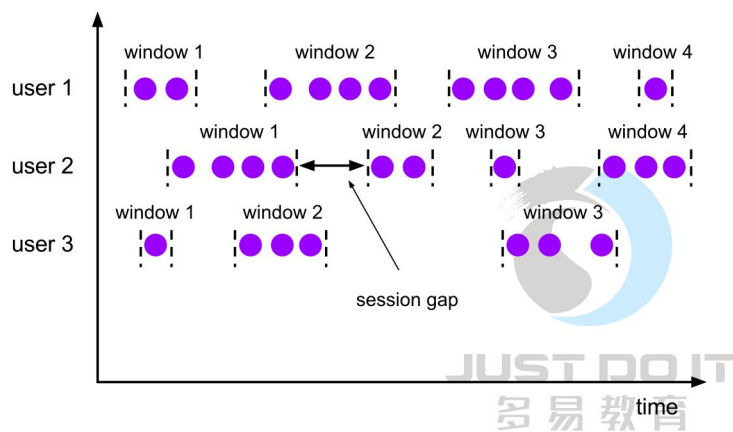
滚动窗口



滑动窗口



会话窗口



没有固定的窗口长度，也没有固定的滑动步长，而是根据数据流中前后两个事件的时间间隔是否超出阈值（session gap）来划分；

8.3. 窗口计算 API 模板

KeyedWindows

stream

```
.keyBy(...)          <- keyed versus non-keyed windows

.window(...)         <- required: "assigner"

[.trigger(...)]      <- optional: "trigger" (else default trigger)

[.evictor(...)]      <- optional: "evictor" (else no evictor)
```

```
[.allowedLateness(...)]    <- optional: "lateness" (else zero)

[.sideOutputLateData(...)] <- optional: "output tag" (else no side output for late data)

.reduce/aggregate/apply()  <- required: "function"

[.getSideOutput(...)]      <- optional: "output tag"
```

NonKeyedWindows

stream

```
.windowAll(...)          <- required: "assigner"

[.trigger(...)]           <- optional: "trigger" (else default trigger)

[.evictor(...)]           <- optional: "evictor" (else no evictor)

[.allowedLateness(...)]   <- optional: "lateness" (else zero)

[.sideOutputLateData(...)] <- optional: "output tag" (else no side output for late data)

.reduce/aggregate/apply() <- required: "function"

[.getSideOutput(...)]     <- optional: "output tag"
```

8.4. 窗口指派 API

```
/**

 * NonKeyed 窗口，全局窗口

 */

// 处理时间语义，滚动窗口

source.windowAll(TumblingProcessingTimeWindows.of(Time.seconds(5)));

// 处理时间语义，滑动窗口
```

```
source.windowAll(SlidingProcessingTimeWindows.of(Time.seconds(5),Time.seconds(1)));

// 事件时间语义，滚动窗口

source.windowAll(TumblingEventTimeWindows.of(Time.seconds(5)));

// 事件时间语义，滑动窗口

source.windowAll(SlidingEventTimeWindows.of(Time.seconds(5),Time.seconds(1)));

// 计数滚动窗口

source.countWindowAll(100);

// 计数滑动窗口

source.countWindowAll(100,20);


/**

 * Keyed 窗口

 */

KeyedStream<String, String> keyedStream = source.keyBy(s -> s);

// 处理时间语义，滚动窗口

keyedStream.window(TumblingProcessingTimeWindows.of(Time.seconds(5)));

// 处理时间语义，滑动窗口

keyedStream.window(SlidingProcessingTimeWindows.of(Time.seconds(5),Time.seconds(1)));

// 事件时间语义，滚动窗口

keyedStream.window(TumblingEventTimeWindows.of(Time.seconds(5)));

// 事件时间语义，滑动窗口
```

```
keyedStream.window(SlidingEventTimeWindows.of(Time.seconds(5),Time.seconds(1)));

// 计数滚动窗口

keyedStream.countWindow(1000);

// 计数滑动窗口

keyedStream.countWindow(1000,100);
```

8.5. 窗口聚合算子

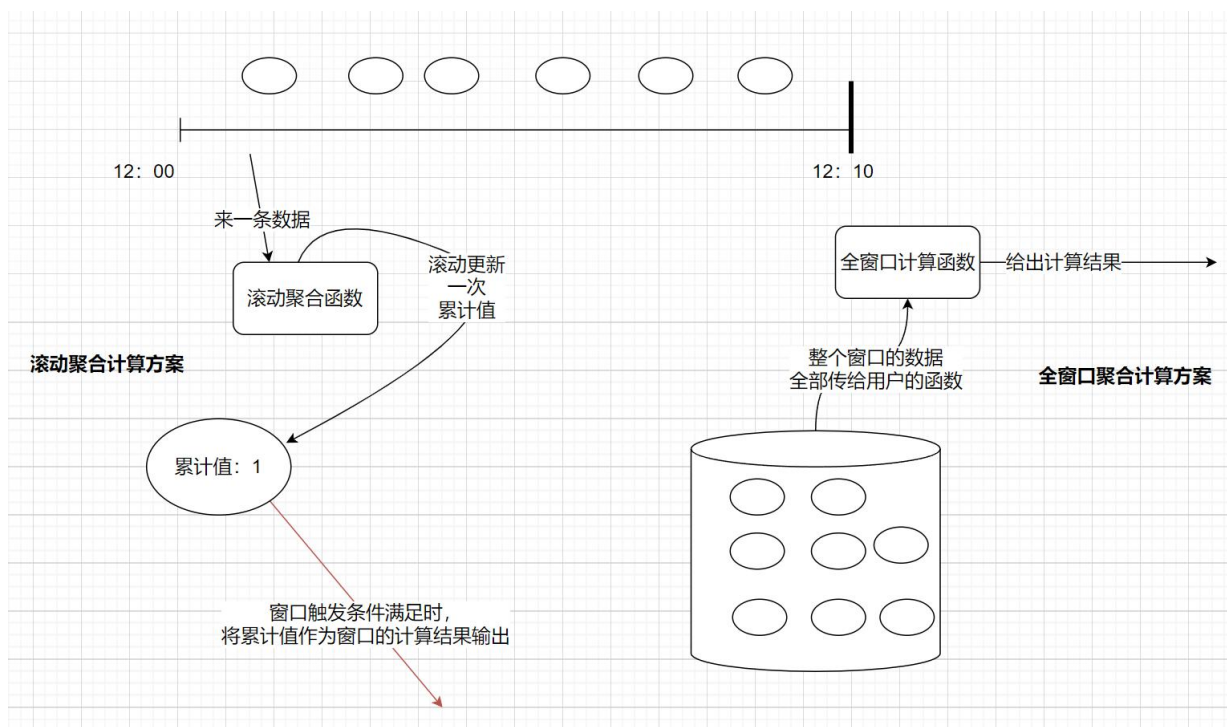
8.5.1. 两类窗口聚合算子的区别

窗口聚合算子，整体上分为两类

- 增量聚合算子，如 min、max、minBy、maxBy、sum、reduce、aggregate
- 全量聚合算子，如 apply、process

两类聚合算子的底层区别

- 增量聚合：一次取一条数据，用聚合函数对中间累加器更新；窗口触发时，取累加器输出结果；
- 全量聚合：数据“攒”在状态容器中，窗口触发时，把整个窗口的数据交给聚合函数；



8.5.2. 各种聚合算子代码示例

简单滚动聚合算子

```
keyedStream.countWindow(5, 2)
```

```
/*max("score") */ // 得到的结果中, 除了 score 是符合逻辑的结果外, 其他字段是窗口中第一条的值
```

```
/*min("score")*/
```

```
/*maxBy("score") */ // 得到的结果是: 最大 score 所在的那一行数据
```

```
/*minBy("score")*/ // 得到的结果是: 最小 score 所在的那一行数据
```

```
/*sum("score")*/ // 得到的结果中, 除了 score 是符合逻辑(score之和)的结果外, 其他字段是窗口中第一条
```

reduce 聚合算子 (增量聚合)

```
source.windowAll(TumblingProcessingTimeWindows.of(Time.seconds(10)))
```

```
.reduce(new ReduceFunction<Long>() {
```

```
@Override

    public Long reduce(Long value1, Long value2) throws Exception {

        return null;

    }

});
```

Aggregate 聚合算子示例 1

```
/**
 * 滚动聚合 api 使用示例
 * 需求 一： 每隔 10s，统计最近 30s 的数据中，每个用户的行为事件条数
 * 使用 aggregate 算子来实现
 */

SingleOutputStreamOperator<Integer> resultStream = watermarkedBeanStream

    .keyBy(EventBean2::getGuid)

    // 参数 1： 窗口长度； 参数 2： 滑动步长

    .window(SlidingEventTimeWindows.of(Time.seconds(30), Time.seconds(10)))

    // reduce：滚动聚合算子，它有个限制，聚合结果的数据类型与数据源中的数据类型，是一致

    /*.reduce(new ReduceFunction<EventBean>() {

        @Override

        public EventBean reduce(EventBean value1, EventBean value2) throws Exception {

            return null;

        }

    })*/

    .aggregate(new AggregateFunction<EventBean2, Integer, Integer>() {

        /**

         * 初始化累加器

         * @return
```

```
*/

@Override

public Integer createAccumulator() {

    return 0;

}

/**

 * 滚动聚合的逻辑（拿到一条数据，如何去更新累加器）

 * @param value The value to add

 * @param accumulator The accumulator to add the value to

 * @return

 */

@Override

public Integer add(EventBean2 value, Integer accumulator) {

    return accumulator + 1;

}

/**

 * 从累加器中，计算出最终要输出的窗口结算结果

 * @param accumulator The accumulator of the aggregation

 * @return

 */

@Override

public Integer getResult(Integer accumulator) {

    return accumulator;

}

/**

 * 批计算模式下，可能需要将多个上游的局部聚合累加器，放在下游进行全局聚合
```

```
* 因为需要对两个累加器进行合并
* 这里就是合并的逻辑
* 流计算模式下，不用实现！
* @param a An accumulator to merge
* @param b Another accumulator to merge
* @return
*/
@Override
public Integer merge(Integer a, Integer b) {
    return a + b;
}

});

/*resultStream.print();*/
```

Aggregate 聚合算子示例 2

```
/**
 * 需求 二： 每隔 10s，统计最近 30s 的数据中，每个用户的平均每次行为时长
 * 要求用 aggregate 算子来做聚合
 * 滚动聚合 api 使用示例
 */
SingleOutputStreamOperator<Double> resultStream2 = watermarkedBeanStream
    .keyBy(EventBean2::getGuid)
    .window(SlidingEventTimeWindows.of(Time.seconds(30), Time.milliseconds(10)))
    // 泛型 1: 输入的数据的类型 ; 泛型 2: 累加器的数据类型 ; 泛型 3: 最终结果的类型
    .aggregate(new AggregateFunction<EventBean2, Tuple2<Integer, Integer>, Double>() {
        @Override
        public Tuple2<Integer, Integer> createAccumulator() {
            return Tuple2.of(0, 0);
        }
    })
```

```
@Override

public Tuple2<Integer, Integer> add(EventBean2 eventBean, Tuple2<Integer, Integer>
accumulator) {

    // accumulator.setField(accumulator.f0+1,0);

    // accumulator.setField(accumulator.f1+eventBean.getActTimeLong(),1);

    // return accumulator;

    return Tuple2.of(accumulator.f0 + 1, accumulator.f1 + eventBean.getActTimeLong());
}

@Override

public Double getResult(Tuple2<Integer, Integer> accumulator) {

    return accumulator.f1 / (double) accumulator.f0;
}

/**
 * 在批计算模式中，shuffle 的上游可以做局部聚合，然后会把局部聚合结果交给下游去做全局聚合
 * 因此，就需要提供 两个局部聚合结果进行合并的逻辑
 *
 * 在流式计算中，不存在：上游局部聚合后再交给下游全局聚合的机制！
 * 所以，在流式计算模式下，不用实现下面的方法
 * @param a An accumulator to merge
 * @param b Another accumulator to merge
 * @return
 */

@Override

public Tuple2<Integer, Integer> merge(Tuple2<Integer, Integer> a, Tuple2<Integer,
Integer> b) {
```

```
        return Tuple2.of(a.f0 + b.f0, a.f1 + b.f1);
    }

    });

    /*resultStream2.print();*/
```

apply 聚合算子（全量聚合）

```
/**
 * 全窗口计算 api 使用示例
 *
 * 需求 三： 每隔 10s，统计最近 30s 的数据中，每个用户的行为事件中，行为时长最长的前 2 条记录
 *
 * 要求用 apply 或者 process 算子来实现
 *
 */

// 1. 用 apply 算子来实现需求

SingleOutputStreamOperator<EventBean2> resultStream3 =
watermarkedBeanStream.keyBy(EventBean2::getGuid)

    .window(SlidingEventTimeWindows.of(Time.seconds(30), Time.seconds(10)))

    // 泛型 1: 输入数据类型; 泛型 2: 输出结果类型; 泛型 3: key 的类型, 泛型 4: 窗口类型
    .apply(new WindowFunction<EventBean2, EventBean2, Long, TimeWindow>() {

        /**
         *
         * @param key 本次传给咱们的窗口是属于哪个 key 的
         * @param window 本次传给咱们的窗口的各种元信息（比如本窗口的起始时间，结束时间）
         * @param input 本次传给咱们的窗口中所有数据的迭代器
         * @param out 结果数据输出器
         * @throws Exception
         */
    })
```

```
@Override

    public void apply(Long key, TimeWindow window, Iterable<EventBean2> input,
Collector<EventBean2> out) throws Exception {

        // low bi 写法: 从迭代器中迭代出数据, 放入一个 arraylist, 然后排序, 输出前 2 条

        ArrayList<EventBean2> tmpList = new ArrayList<>();

        // 迭代数据, 存入 list

        for (EventBean2 eventBean2 : input) {

            tmpList.add(eventBean2);

        }

        // 排序

        Collections.sort(tmpList, new Comparator<EventBean2>() {

            @Override

            public int compare(EventBean2 o1, EventBean2 o2) {

                return o2.getActTimelong() - o1.getActTimelong();

            }

        });

        // 输出前 2 条

        for (int i = 0; i < Math.min(tmpList.size(), 2); i++) {

            out.collect(tmpList.get(i));

        }

    }

});

/*resultStream3.print();*/
```

process 聚合算子

```
/**
 * 全窗口计算 api 使用示例
 *
 * 需求 三： 每隔 10s，统计最近 30s 的数据中，每个用户的行为事件中，行为时长最长的前 2 条记录
 *
 * 要求用 apply 或者 process 算子来实现
 *
 */

// 2. 用 process 算子来实现需求

SingleOutputStreamOperator<String> resultStream4 =
watermarkedBeanStream.keyBy(EventBean2::getGuid)

    .window(SlidingEventTimeWindows.of(Time.seconds(30), Time.seconds(10)))

    .process(new ProcessWindowFunction<EventBean2, String, Long, TimeWindow>() {

        @Override

        public void process(Long aLong, ProcessWindowFunction<EventBean2, String, Long,
TimeWindow>.Context context, Iterable<EventBean2> input, Collector<String> out) throws Exception {

            // 本次窗口的元信息

            TimeWindow window = context.window();

            long maxTimestamp = window.maxTimestamp();// 本窗口允许的最大时间戳 [1000,2000)，其中 1999 就是允许的最大时间戳； 2000 就是窗口的 end

            long windowStart = window.getStart();

            long windowEnd = window.getEnd();

            // low bi 写法： 从迭代器中迭代出数据，放入一个 arraylist，然后排序，输出前 2 条

            ArrayList<EventBean2> tmpList = new ArrayList<>();

            // 迭代数据，存入 list

            for (EventBean2 eventBean2 : input) {
```

```
        tmpList.add(eventBean2);

    }

    // 排序

    Collections.sort(tmpList, new Comparator<EventBean2>() {

        @Override

        public int compare(EventBean2 o1, EventBean2 o2) {

            return o2.getActTimeLong() - o1.getActTimeLong();

        }

    });

    // 输出前 2 条

    for (int i = 0; i < Math.min(tmpList.size(), 2); i++) {

        EventBean2 bean = tmpList.get(i);

        out.collect( "窗口 start:"+windowStart + "," + "窗口 end:"+ windowEnd + ","

+ bean.getGuid() + "," + bean.getEventId() + "," + bean.getTimeStamp() + "," +bean.getPageId() + ","

+bean.getActTimeLong());

    }

}

});

resultStream4.print();
```

8.6. 数据延迟处理

延迟处理的方案

- 小延迟（乱序），用 watermark 容错（减慢时间的推进，让本已迟到的数据被认为没有迟到）
- 中延迟（乱序），用 allowedLateness（允许一定限度内的迟到，并对迟到数据重新触发窗口计算）
- 大延迟（乱序），用 sideOutputLateData（将超出 allowedLateness 的迟到数据输出到一个侧流中）

代码示例

```
SingleOutputStreamOperator<String> stream4 =

    stream3.windowAll(SlidingEventTimeWindows.of(Time.seconds(5), Time.seconds(5)))

    .allowedLateness(Time.milliseconds(2000))    // 允许迟到 2 秒；默认是 0

    .sideOutputLateData(lateTag)    // 超过迟到最大允许时间的数据，收集到侧输出流

    .apply(new Ex3_WindowFunc1());

// 获取侧流，做一些自己的补救逻辑

stream4.getSideOutput(lateTag).print();
```

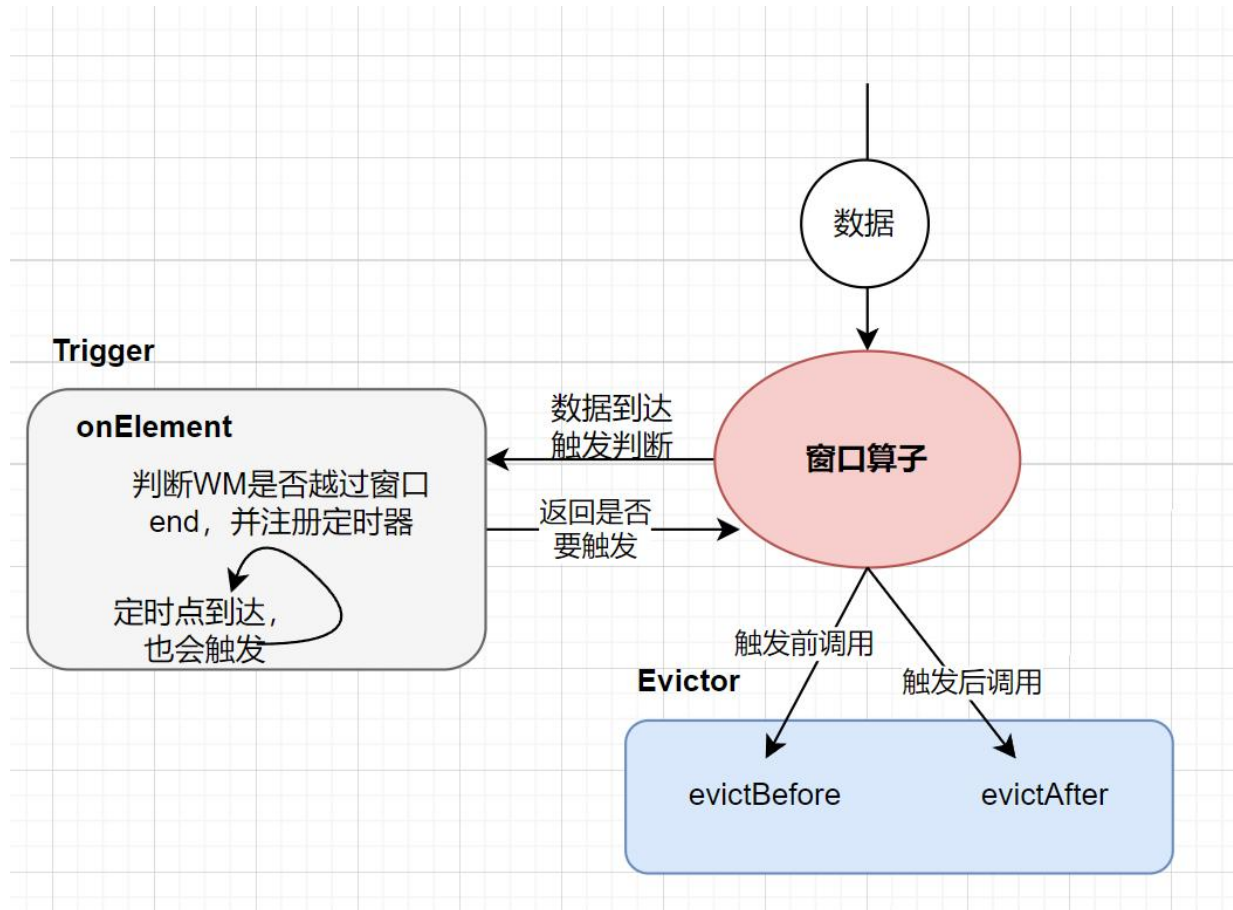
注意正确理解延迟时间！

如：allowedLateness (2s)，表示：

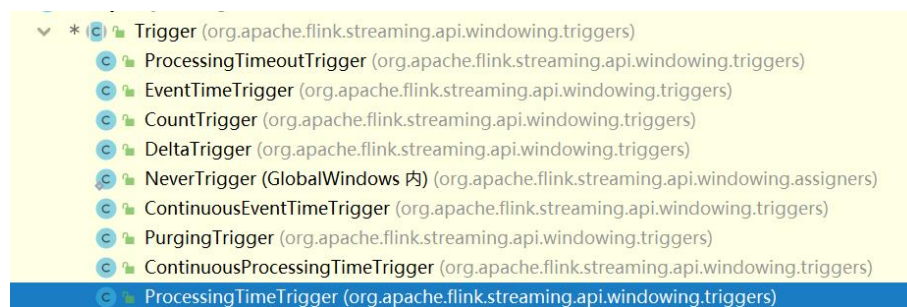
如果 watermark（此刻的事件时间）推进到了 A 窗口结束点后 2s，如果还来 A 窗口的数据，就算迟到，就不会再去触发 A 窗口的计算，而是输出到迟到侧流了

8.7. 窗口的触发机制

窗口计算的触发，是由 Trigger 类来决定；当 Trigger 返回需要触发时；窗口算子还要调用 Evictor 来进行元素移除操作；



- Flink 中为各类内置的 WindowsAssigner 都设计了对应的默认 Trigger;



一般情况下不需要自己去重写 Trigger；除非有特别的需求；

- Evictor 是窗口触发前，或者触发后，对窗口中的数据移除的机制；
 - 1) 当触发条件成立时，先调用 Evictor 的 `evictBefore()` 方法 来进行元素移除；
 - 2) 然后，计算；
 - 3) 计算完后，还会再调用 Evictor 的 `evictAfter()` 方法，来进行元素移除；

Trigger 和 Evictor 的设置代码

```
SingleOutputStreamOperator<String> sumResult = beanStream.keyBy(tp -> tp.f0.getGuid())

.window(TumblingEventTimeWindows.of(Time.seconds(10))) // 事件时间滚动窗口，窗口长度为 10

// 设置自定义的 Trigger

.trigger(MyEventTimeTrigger.create())

// 设置自定义的 Evictor，它会在窗口触发计算前，对窗口中的 e0x 标记事件进行移除

.evictor(MyTimeEvictor.of(Time.seconds(10)))

.apply(new WindowFunction<Tuple2<EventBean2, Integer>, String, Long, TimeWindow>() {

    @Override

    public void apply(Long aLong, TimeWindow window, Iterable<Tuple2<EventBean2, Integer>> input,

Collector<String> out) throws Exception {

        // 计算逻辑

    }

});
```

Trigger 代码示例

背景：不光要按照滚动时间窗口的时间条件触发，还要根据数据中的 eventId=e0x 来触发

```
class MyEventTimeTrigger extends Trigger<Tuple2<EventBean2,Integer>, TimeWindow>{

    private MyEventTimeTrigger() {}

    /**

     * 来一条数据时，需要检查 watermark 是否已经越过窗口结束点需要触发

     */

    @Override

    public TriggerResult onElement(

        Tuple2<EventBean2,Integer> element, long timestamp, TimeWindow window, TriggerContext ctx)

        throws Exception {
```

```
// 如果窗口结束点 <= 当前的 watermark

if (window.maxTimestamp() <= ctx.getCurrentWatermark()) {

    return TriggerResult.FIRE;

} else {

    // 注册定时器，定时器的触发时间为：窗口的结束点时间

    ctx.registerEventTimeTimer(window.maxTimestamp());

    // 判断，当前数据的用户行为事件 id 是否等于 e0x，如是，则触发

    if("e0x".equals(element.f0.getEventId())) return TriggerResult.FIRE;

    return TriggerResult.CONTINUE;

}

}

/**
 * 当事件时间定时器的触发时间（窗口的结束点时间）到达了，检查是否满足触发条件
 * 下面的方法，是定时器在调用
 */
@Override

public TriggerResult onEventTime(long time, TimeWindow window, TriggerContext ctx) {

    return time == window.maxTimestamp() ? TriggerResult.FIRE : TriggerResult.CONTINUE;

}

/**
 * 当处理时间定时器的触发时间（窗口的结束点时间）到达了，检查是否满足触发条件
 */
@Override

public TriggerResult onProcessingTime(long time, TimeWindow window, TriggerContext ctx)

    throws Exception {
```

```
        return TriggerResult.CONTINUE;
    }

    @Override
    public void clear(TimeWindow window, TriggerContext ctx) throws Exception {
        ctx.deleteEventTimeTimer(window.maxTimestamp());
    }

    @Override
    public boolean canMerge() {
        return true;
    }

    @Override
    public void onMerge(TimeWindow window, OnMergeContext ctx) {
        long windowMaxTimestamp = window.maxTimestamp();
        if (windowMaxTimestamp > ctx.getCurrentWatermark()) {
            ctx.registerEventTimeTimer(windowMaxTimestamp);
        }
    }

    public static MyEventTimeTrigger create() {
        return new MyEventTimeTrigger();
    }
}
```

Evictor 代码示例

背景：不光要按照滚动时间窗口的时间条件移除数据，还要将 eventId=e0x 的数据移除

```
class MyTimeEvictor implements Evictor<Object, TimeWindow> {
```

```
private static final long serialVersionUID = 1L;

private final long windowSize;

private final boolean doEvictAfter;

public MyTimeEvictor(long windowSize) {

    this.windowSize = windowSize;

    this.doEvictAfter = false;

}

public MyTimeEvictor(long windowSize, boolean doEvictAfter) {

    this.windowSize = windowSize;

    this.doEvictAfter = doEvictAfter;

}

/**
 * 窗口触发前，调用
 */
@Override
public void evictBefore(
    Iterable<TimestampedValue<Object>> elements, int size, TimeWindow window, EvictorContext ctx) {

    if (!doEvictAfter) {

        evict(elements, size, ctx);

    }

}

/**
 * 窗口触发后，调用
 */
```

```
@Override

public void evictAfter(
    Iterable<TimestampedValue<Object>> elements, int size, TimeWindow window, EvictorContext ctx)
{
    if (doEvictAfter) {
        evict(elements, size, ctx);
    }
}

/**
 * 元素移除的核心逻辑
 */

private void evict(Iterable<TimestampedValue<Object>> elements, int size, EvictorContext ctx) {
    if (!hasTimestamp(elements)) {
        return;
    }

    long currentTime = getMaxTimestamp(elements);
    long evictCutoff = currentTime - windowSize;

    for (Iterator<TimestampedValue<Object>> iterator = elements.iterator();
        iterator.hasNext(); ) {
        TimestampedValue<Object> record = iterator.next();

        Tuple2<EventBean2, Integer> tuple = (Tuple2<EventBean2, Integer>) record.getValue();

        // 加了一个条件： 数据的 eventId=e0x， 也移除

        if (record.getTimestamp() <= evictCutoff || tuple.f0.getEventId().equals("e0x")) {
            iterator.remove();
        }
    }
}
```

```
    }

    }

}

private boolean hasTimestamp(Iterable<TimestampedValue<Object>> elements) {

    Iterator<TimestampedValue<Object>> it = elements.iterator();

    if (it.hasNext()) {

        return it.next().hasTimestamp();

    }

    return false;

}

/**
 * 用于计算移除的时间截止点
 */

private long getMaxTimestamp(Iterable<TimestampedValue<Object>> elements) {

    long currentTime = Long.MIN_VALUE;

    for (Iterator<TimestampedValue<Object>> iterator = elements.iterator();

        iterator.hasNext(); ) {

        TimestampedValue<Object> record = iterator.next();

        currentTime = Math.max(currentTime, record.getTimestamp());

    }

    return currentTime;

}

public static MyTimeEvictor of(Time windowSize) {

    return new MyTimeEvictor(windowSize.toMilliseconds());

}
```

}



9. Flink 状态（state）及使用

9.1. 状态基本概念

什么是状态（raw 状态）

在稍复杂的流式计算逻辑中，基本都需要记录和利用一些历史累积信息；

例如，对整数流进行全局累计求和的逻辑：

- ① 我们在算子中定义一个变量来记录累计到当前的总和；
- ② 在一条新数据到达时，就将新数据累加到总和变量中；
- ③ 然后输出结果；

由上可知：状态就是用户在程序逻辑中用于记录信息的变量

（当然，依据不同的需求，状态数据可多可少，可简单可复杂）！

如上所述，state 只不过是用户编程时自定义的“变量”，跟 flink 又有何关系呢？

因为状态（状态中记录的数据），需要容错！！程序一旦在运行中突然失败，则用户自定义的状态所记录的数据会丢失，因而无法实现失败后重启的接续！

flink 托管状态

flink 提供了内置的状态数据管理机制（简称状态机制）；

flink 会进行状态数据管理：包括故障发生后的状态一致性维护、以及状态数据的高效存储和访问，

用户借由 flink 所提供的的状态管理机制来托管自己的状态数据，则不用担心状态数据在程序失败及恢复时所引入的一系列问题；从而使得开发人员可以专注于应用程序的逻辑开发；

9.2. 算子状态与键控状态

9.2.1. 算子状态与键控状态的区别

算子状态

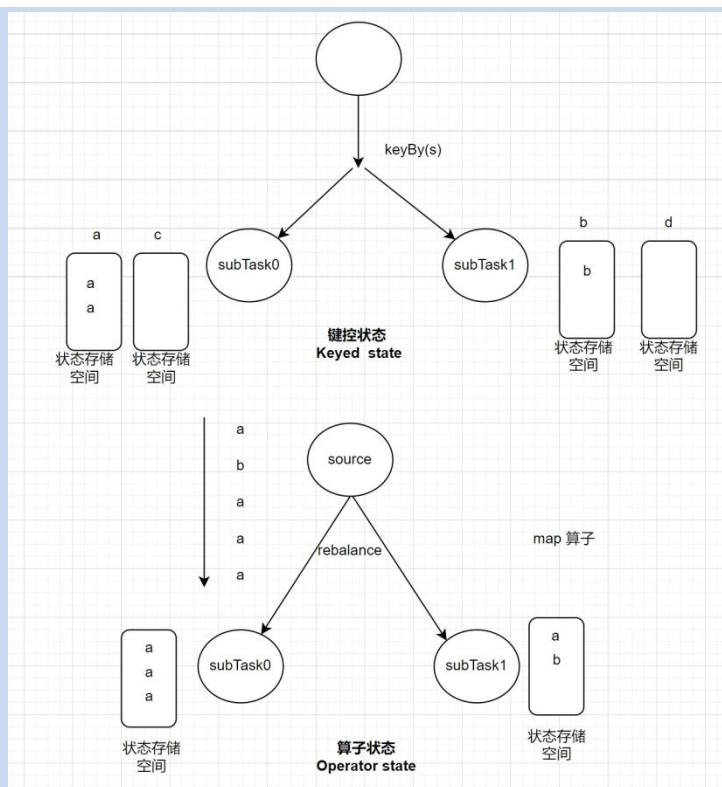
- 算子状态，是每个 subtask 自己持有一份独立的状态数据（但如果在失败恢复后，算子并行度发生变化，则状态将在新的 subtask 之间均匀分配）；
- 算子函数实现 CheckpointedFunction 后，即可使用算子状态；
- 算子状态，通常用于 source 算子中；其他场景下建议使用 KeyedState（键控状态）；

算子状态，在逻辑上，由算子 task 下所有 subtask 共享；

如何理解：正常运行时，subtask 自己读写自己的状态数据；而一旦 job 重启且带状态算子发生了并行度的变化，则之前的状态数据将在新的一批 subtask 间均匀分配

键控状态

- 键控状态，只能应用于 KeyedStream 的算子中（keyby 后的处理算子中）；
- 算子为每一个 key 绑定一份独立的状态数据；



算子状态和键控状态的区别

9.2.2. 算子状态 api 示例

要使用算子状态（operator state），需要让算子函数实现 CheckpointedFunction 接口；

代码示例

```
/**
 * @author hunter.d
 * @qq 657270652
 * @wx haitao-duan
 * @date 2022/4/10
 **/

public class OperatorStateTest {

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        conf.setString("execution.savepoint.path",
"D:\\ckpt\\27270525e8f166834f2bbf7c617ad6d3\\chk-11");

        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(conf);

        env.enableCheckpointing(2000);

        env.getCheckpointConfig().setCheckpointStorage("file:///d:/ckpt");

        DataStreamSource<String> source = env.socketTextStream("localhost", 9999);

        //以下两个 map 算子，其中一个是带状态的

        // 如果修改代码逻辑（如调整两个 map 算子顺序），且没有设置 uid，则从 savepoints 恢复时将失败
```

```
source

    .map(new StatefulMapFunc()).uid("stateful-mapfunc-001").setParallelism(2)

    .map(new NoStateMapFunc()).setParallelism(1)

    .print().setParallelism(1);

env.execute();

}

public static class NoStateMapFunc implements MapFunction<String, String> {

    @Override

    public String map(String value) throws Exception {

        return value.toUpperCase();

    }

}

// 带状态的 map 函数（将接收的字符串记在状态中，以不断拼接新数据返回）

public static class StatefulMapFunc extends RichMapFunction<String, String> implements
CheckpointedFunction {

    ListState<String> lstState;

    /**

     * 正常的 map 映射逻辑方法

     */
}
```

```
@Override

public String map(String value) throws Exception {

    lstState.add(value);

    StringBuilder sb = new StringBuilder();

    for (String s : lstState.get()) {

        sb.append(s).append(",");

    }

    return sb.toString();

}

/**

 * checkpoint 触发时会调用的方法

 */

@Override

public void snapshotState(FunctionSnapshotContext context) throws Exception {

}

/**

 * 初始化算子任务时会调用的方法，以加载、初始化状态数据

 */

@Override

public void initializeState(FunctionInitializationContext context) throws Exception {
```

```
lstState = context.getOperatorStateStore().getListState(new
ListStateDescriptor<String>("lst", String.class));

int indexOfThisSubtask = getRuntimeContext().getIndexOfWorkThisSubtask();

Iterable<String> iter = lstState.get();

Iterator<String> it = iter.iterator();

// 用于观察 task 失败恢复后的状态恢复情况

System.out.println("-----" + indexOfThisSubtask + " - 初始化时，打印状态: -----");

while (it.hasNext()) {

    System.out.println(indexOfWorkThisSubtask + ":" + it.next());

}

System.out.println("-----" + indexOfThisSubtask + " - 初始化时，打印状态: -----");

}

}

}
```

9.2.3. 键控状态 api 示例

要使用键控状态（Keyed State），需要在实现 RichFunction 的函数中；

代码示例

```
public class _15_ChannelEventsCntMapFunc extends RichMapFunction<EventLog, String> {

    ValueState<Integer> valueState;

    @Override

    public void open(Configuration parameters) throws Exception {

        StateTtlConfig ttlConfig =
        StateTtlConfig.newBuilder(Time.milliseconds(2000)).cleanupFullSnapshot().neverReturnExpired().useProcessingTime().updateTtlOnReadAndWrite().build();

        ValueStateDescriptor<Integer> desc = new ValueStateDescriptor<>("cnt", Integer.class);

        desc.enableTimeToLive(ttlConfig);

        // 获取单值状态管理器

        valueState = getRuntimeContext().getState(desc);

    }

    @Override

    public String map(EventLog eventLog) throws Exception {

        // 来一条数据，就对状态更新

        valueState.update((valueState.value() == null ? 0 : valueState.value()) + 1);

        return eventLog.getChannel() + " : " + valueState.value();

    }

}
```

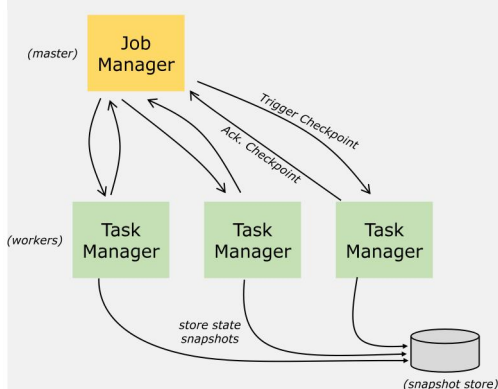


9.3. 状态后端

9.3.1. 基础概念

状态后端的基本概念

- 所谓状态后端，就是状态数据的存储管理实现，包含状态数据的本地读写、快照远端存储功能；
- 状态后端是可插拔替换的，它对上层屏蔽了底层的差异，因为在更换状态后端时，用户的代码不需要做任何更改；



JUST DO IT

可用的状态后端类型（flink-1.13 版以后）

- HashMapStateBackend
- EmbeddedRocksDBStateBackend

老版本（flink-1.12 版及以前）~~Fsstatebackend~~ ~~MemoryStatebackend~~ ~~RocksdbStateBackend~~

新版本中，**Fsstatebackend** 和 **MemoryStatebackend** 整合成了 **HashMapStateBackend**

而且 **HashMapStateBackend** 和 **EmBeddedRocksDBStateBackend** 所生成的快照文件也统一了格式，因而在 job 重新部署或者版本升级时，可以任意替换 statebackend

如需使用 rocksdb-backend，需要引入依赖

```
<dependency>

    <groupId>org.apache.flink</groupId>

    <artifactId>flink-statebackend-rocksdb_2.12</artifactId>

    <version>${flink.version}</version>
```

```
</dependency>
```

状态后端的配置代码

- 可以在代码中设置

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

env.setStateBackend(...);
```

- 可以在配置文件 `flink-conf.yaml` 中配置

```
# The backend that will be used to store operator state checkpoints

state.backend: hashmap

# Directory for storing checkpoints

state.checkpoints.dir: hdfs://doitedu:8020/flink/checkpoints
```

9.3.2. HashMapStateBackend 状态后端

HashMapStateBackend:

- 状态数据是以 java 对象形式存储在 heap 内存中;
- 内存空间不够时, 也会溢出一部分数据到本地磁盘文件;
- 可以支撑大规模的状态数据; (只不过在状态数据规模超出内存空间时, 读写效率就会明显降低)

对于 KeyedState 来说:

HashMapStateBackend 在内存中是使用 **CopyOnWriteStateMap** 结构来存储用户的状态数据; 注意, 此数据结构类, 名为 **Map**, 实非 **Map**, 它其实是一个单向链表的数据结构

对于 OperatorState 来说:

```
public class DefaultOperatorStateBackend implements OperatorStateBackend {

    private static final Logger LOG = LoggerFactory.getLogger(DefaultOperatorStateBackend.class);

    /** The default namespace for state in cases where no state name is provided */
    public static final String DEFAULT_OPERATOR_STATE_NAME = "_default_";

    /** Map for all registered operator states. Maps state name -> state */
    private final Map<String, PartitionableListState<?>> registeredOperatorStates;
```

可以清楚看出，它底层直接用一个 **Map 集合** 来存储用户的状态数据：状态名称 --> 状态 List

9.3.3. EmBeddedRocksDbStateBackend 状态后端

- 状态数据是交给 rocksdb 来管理；
- Rocksdb 中的数据是以序列化的 kv 字节进行存储；
- Rocksdb 中的数据，有内存缓存的部分，也有磁盘文件的部分；
- Rocksdb 的磁盘文件数据读写速度相对还是比较快的，所以在支持超大规模状态数据时，数据的读写效率不会有太大的降低

注意：上述 2 中状态后端，在生成 checkpoint 快照文件时，生成的文件格式是完全一致的；
所以，用户的 flink 程序在更改状态后端后，重启时依然可以加载和恢复此前的快照文件数据；

9.4. 状态数据结构介绍

算子状态提供的数据结构

- ✓ ListState
- ✓ UnionListState

UnionListState 和普通 ListState 的区别：

- 1) UnionListState 的快照存储数据，在系统重启后，list 数据的重分配模式为：广播模式；在每个 subtask 上都拥有一份完整的数据；
- 2) ListState 的快照存储数据，系统重启后，list 数据的重分配模式为：round-robin 轮询平均分配；

键控状态提供的数据结构

- ✓ ValueState
- ✓ ListState
- ✓ MapState
- ✓ ReducingState
- ✓ AggregateState

9.5. 状态数据使用 api 示例

ValueState 单值状态

```
// 定义 valueState 描述器

ValueStateDescriptor<Tuple2<String, Integer>> valueStateDesc =

    new ValueStateDescriptor<>("valueState", TypeInformation.of(new TypeHint<Tuple2<String,
Integer>>() {}));

// 获取 ValueState

ValueState<Tuple2<String, Integer>> valueState = getRuntimeContext().getState(valueStateDesc);

// 从 valueState 中取数据

Tuple2<String, Integer> value = valueState.value();

// 为 valueState 更新数据

valueState.update(Tuple2.of("newvalue", 100));
```

ListState List 状态

```
// 定义 listState 描述器

ListStateDescriptor<String> listStateDescriptor = new ListStateDescriptor<>("listState", String.class);

// 获取 listState

ListState<String> listState = getRuntimeContext().getListState(listStateDescriptor);

// 转成可迭代的

Iterable<String> iterable = listState.get();
```

```
// 一次添加多个元素到 state 数据中

listState.addAll(Arrays.asList("a","b"));

// 添加一个元素到 state 数据中

listState.add("ele");

// 直接更换掉 state 的所有数据为新数据

listState.update(Arrays.asList("1","2","3"));
```

MapState Map 状态

```
// 定义 mapState 描述器

MapStateDescriptor<String, Integer> mapStateDescriptor = new MapStateDescriptor<>("mapState",
String.class, Integer.class);

// 获取 mapState

MapState<String, Integer> mapState = getRuntimeContext().getMapState(mapStateDescriptor);

// 插入 key-value

mapState.put("a",3);

// 根据 key 取 value

Integer v = mapState.get("a");

// 取所有 key

Iterable<String> keys = mapState.keys();

// 取所有 value
```

```
Iterable<Integer> values = mapState.values();

// 判断是否包含 key

boolean contains = mapState.contains("a");

// 判空

boolean empty = mapState.isEmpty();

// 删除指定 key

mapState.remove("a");

// 取迭代器

Iterator<Map.Entry<String, Integer>> iterator = mapState.iterator();

// 取所有 entry

Iterable<Map.Entry<String, Integer>> entries = mapState.entries();
```

JUST DO IT
多易教育

Reducing 聚合状态

用户传入一个增量聚合函数后，状态实现自动增量聚合（输入数据与聚合结果类型必须一致）

```
// 获取一个 reduce 聚合状态

reduceState = runtimeContext.getReducingState(new ReducingStateDescriptor<Integer>("reduceState", new
ReduceFunction<Integer>() {

    @Override

    public Integer reduce(Integer value1, Integer value2) throws Exception {

        return value1 + value2;

    }

}, Integer.class));
```

```
/**  
 * reduce 状态使用  
 */  
  
reduceState.add(10); // 往聚合状态中添加数据 ,此刻状态中的数据是 10  
  
reduceState.add(20); // 往聚合状态中添加数据 ,此刻状态中的数据是 30  
  
Integer stateValue = reduceState.get(); // 从聚合状态中获取数据值
```

Reducing 聚合状态

用户传入一个增量聚合函数后，状态实现自动增量聚合（输入数据与聚合结果类型可以不一致）

```
// 获取一个 aggregate 聚合状态  
  
// 比如，我们要插入整数，返回平均值  
  
aggState = runtimeContext.getAggregatingState(new AggregatingStateDescriptor<Integer, Tuple2<Integer, Integer>, Double>("aggState", new AggregateFunction<Integer, Tuple2<Integer, Integer>, Double>() {  
  
    @Override  
  
    public Tuple2<Integer, Integer> createAccumulator() {  
  
        return Tuple2.of(0, 0);  
  
    }  
  
    @Override  
  
    public Tuple2<Integer, Integer> add(Integer value, Tuple2<Integer, Integer> accumulator) {  
  
        return Tuple2.of(accumulator.f0 + 1, accumulator.f1 + value);  
  
    }  
  
    @Override  
  
    public Double getResult(Tuple2<Integer, Integer> accumulator) {  
  
        return accumulator.f1 / (double) accumulator.f0;  
  
    }  
  
    @Override  
  
    public Tuple2<Integer, Integer> merge(Tuple2<Integer, Integer> a, Tuple2<Integer, Integer> b) {
```

```
        return Tuple2.of(a.f0 + b.f0, a.f1 + a.f1);  
    }  
}, TypeInformation.of(new TypeHint<Tuple2<Integer, Integer>>() {  
})));
```

```
/**  
 * aggregate 状态使用  
 */  
aggState.add(10);  
aggState.add(20);  
  
Double avgDouble = aggState.get(); // 获取状态值 : 15.0
```



9.6. 广播状态 (broadcast state)

代码示例

```
// 主数据流

SingleOutputStreamOperator<Student> s1;

// 待广播出去的流

SingleOutputStreamOperator<StuInfo> s2;

// 定义广播状态的状态描述对象

MapStateDescriptor<Integer, StuInfo> stateDescriptor = new MapStateDescriptor<>("info", Integer.class,
StuInfo.class);

// 将 s2 流广播出去

BroadcastStream<StuInfo> stuInfoBroadcastStream = s2.broadcast(stateDescriptor);

// 用主数据流 connect 连接 广播数据流，并处理

s1.connect(stuInfoBroadcastStream)

.process(new BroadcastProcessFunction<Student, StuInfo, String>() {

    @Override

    public void processElement(Student student, ReadOnlyContext readOnlyContext, Collector<String>
collector) throws Exception {

        // 对 "主流" 中的元素进行处理

        readOnlyContext.getBroadcastState(); // 只读状态
```

```
}

@Override

    public void processBroadcastElement(StuInfo stuInfo, Context context, Collector<String> collector)
    throws Exception {

        // 对 "广播流" 中的元素进行处理

        context.getBroadcastState(); // 可读可写

    }

});
```



9.7. 状态的 TTL 管理 (time to live)

9.7.1. TTL 基本概念

- flink 可以对状态数据进行存活时长管理，即"新陈代谢"；
- 淘汰的机制主要是基于存活时间；
- 存活时长的计时器可以在数据被读、写时重置；
- Ttl 存活管理粒度是到元素级的（如 liststate 中的每个元素，mapstate 中的每个 entry）

代码示例

```
// 构造一个 ttl 的参数配置对象

StateTtlConfig ttlConfig = StateTtlConfig.newBuilder(Time.seconds(60))

    .useProcessingTime() // 默认是用 eventTime 语义，如果需要用 processingTime 语义，则需要显式指定

    //.updateTtlOnReadAndWrite() // ttl 重置刷新策略：数据只要被读取或者被写更新，则将其的 ttl 计时重置

    //.updateTtlOnCreateAndWrite() // ttl 重置刷新策略：数据被创建及被写入更新，就将其的 ttl 计时重置

    .setStateVisibility(StateTtlConfig.StateVisibility.NeverReturnExpired) // 状态数据的可见性：如果状态中存在还未清理掉的但是已经超出 ttl 的数据，是否让用户程序可见

    .build();

// 构造一个 list 状态的描述器

ListStateDescriptor<UserEvent> stateDescriptor =

    new ListStateDescriptor<>("events", UserEvent.class);

// 为描述器设置启用 ttl 功能

stateDescriptor.enableTimeToLive(ttlConfig);
```

```
// 获取一个 list 状态实例
```

```
listState = getRuntimeContext().getListState(stateDescriptor);
```

9.7.2. TTL 相关参数和机制解析

Ttl 的相关配置参数及其内含的机制，全部封装在 StateTtlConfig 类中

```
private final UpdateType updateType;

private final StateVisibility stateVisibility;

private final TtlTimeCharacteristic ttlTimeCharacteristic;

private final Time ttl;

private final CleanupStrategies cleanupStrategies;

private StateTtlConfig(
    UpdateType updateType,
    StateVisibility stateVisibility,
    TtlTimeCharacteristic ttlTimeCharacteristic,
    Time ttl,
    CleanupStrategies cleanupStrategies) {
    this.updateType = checkNotNull(updateType);
    this.stateVisibility = checkNotNull(stateVisibility);
    this.ttlTimeCharacteristic = checkNotNull(ttlTimeCharacteristic);
    this.ttl = checkNotNull(ttl);
    this.cleanupStrategies = cleanupStrategies;

    checkArgument(ttl.toMilliseconds() > 0, "TTL is expected to be positive.");
}
```

Ttlconfig 各参数详解

- updateType (Ttl 计时重置策略, 可以取值为)

```
public enum UpdateType {  
  
    /** TTL is disabled. State does not expire. */  
  
    Disabled,  
  
    /** 计时从写入或更新时开始(重置) */  
  
    OnCreateAndWrite,  
  
    /** 与上一致, 不过读取也会导致计时重置 */  
  
    OnReadAndWrite  
  
}
```

- stateVisibility (过期数据可见策略, 可以取值为)

```
public enum StateVisibility {  
  
    /** 可以返回已过期尚未清理的数据 */  
  
    ReturnExpiredIfNotCleanedUp,  
  
    /** 不返回已过期的数据 */  
  
    NeverReturnExpired  
  
}
```

- ttlTimeCharacteristic (Ttl 计时的时间语义, 目前只支持 processing time 语义)

```
public enum TtlTimeCharacteristic {  
  
    /**  
  
    ProcessingTime  
  
    */  
  
}
```

- Ttl (数据存活时长, 一个简单的长整数参数)

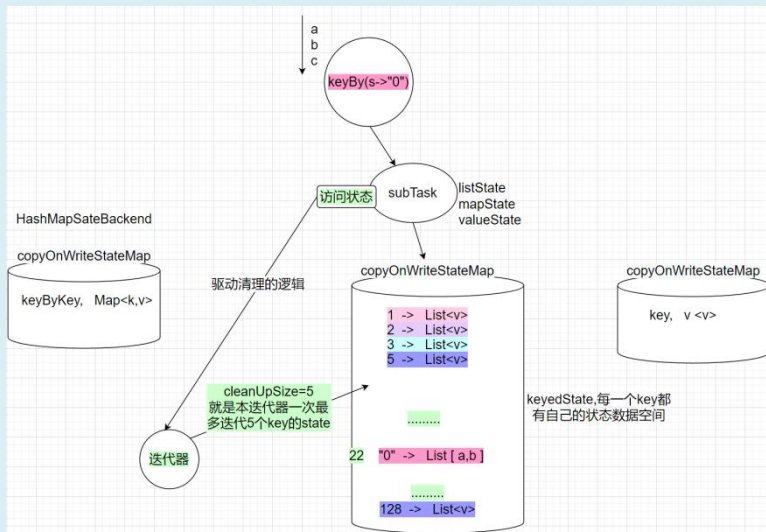
● cleanupStrategies（过期数据清理策略，目前支持的策略有）

1) cleanupIncrementally：增量清除

每当访问状态时，都会驱动一次过期检查（算子注册了很多 key 的 state，一次检查只针对其中一部分：由参数 cleanupSize 决定）

算子持有一个包含所有 key 的迭代器，每次检查后，迭代器都会向前 advance 指定的 key 数量；

本策略，针对“本地状态空间”，且只用于 HashMapStateBackend



2) cleanupFullSnapshot

在进行全量快照（checkpoint）时，清理掉过期数据；

注意：只是在生成的 checkpoint 数据中不包含过期数据；在本地状态空间中，并没有做清理；

本策略，针对“快照”生效

3) cleanupInRocksdbCompactFilter

只针对 rocksdbStateBackend 有效；

它是利用 rocksdb 的 compact 功能，在 rocksdb 进行 compact 时，清除掉过期数据；

本策略，针对“本地状态空间”，且只用于 EmbeddedRocksDbStateBackend

9.7.3. Ttl 参数配置完整 api 示例

```
StateTtlConfig ttlConfig = StateTtlConfig.newBuilder(Time.milliseconds(1000))

// 配置数据的存活时长为 4s（覆盖 builder 构造传入的 1s）

.setTtl(Time.milliseconds(4000))

// 当插入和更新的时候，导致该条数据的 ttl 计时重置
```

```
.updateTtlOnCreateAndWrite()

// 读、写，都导致该条数据的 ttl 计时重置

.updateTtlOnReadAndWrite()

// 不允许返回已过期但尚未被清理的数据

.setStateVisibility(StateTtlConfig.StateVisibility.NeverReturnExpired)

// 允许返回已过期但尚未被清理的数据

.setStateVisibility(StateTtlConfig.StateVisibility.ReturnExpiredIfNotCleanedUp)

// ttl 计时的时间语义: 设置为处理时间

.setTtlTimeCharacteristic(StateTtlConfig.TtlTimeCharacteristic.ProcessingTime)

// ttl 计时的时间语义: 设置为处理时间

.useProcessingTime()

// 增量清理（每当一条状态数据被访问，则会驱动过期检查及清除）

.cleanupIncrementally(1000, true)

// 全量快照清理策略（在 checkpoint 的时候，保存到快照文件中的只包含未过期的状态数据，但是它并不会清理算子本地的状态数据）

.cleanupFullSnapshot()

// 在 rockdb 的 compact 机制中添加过期数据过滤器，以在 compact 过程中清理掉过期状态数据

.cleanupInRocksdbCompactFilter(1000)

// 禁用默认后台清理策略

.disableCleanupInBackground()
```

10. Flink 容错机制概述

10.1. 为什么需要容错机制

Flink 是一个 stateful（带状态）的数据处理系统；系统在处理数据的过程中，各算子所记录的状态会随着数据的处理而不断变化；

一旦系统崩溃，需要重启后能够恢复出崩溃前的状态才能进行数据的接续处理；因此，必须要一种机制能对系统内的各种状态进行持久化容错；

10.2. Flink 的精确一次概述

10.3. EOS 基本概念

Exactly-Once 语义：指端到端的一致性，从数据读取、引擎计算、写入外部存储的整个过程中，即使机器或软件出现故障，都确保数据仅处理一次，不会重复、也不会丢失。

对于 flink 程序来说，端到端 EOS 语义则包含 source、state、sink 三个环节的紧密配合



10.4. Flink 的 eos 要点

要实现端到端的 EOS 保证，核心点在于：

一条（或者一批）数据，从 注入系统、中间处理、到输出结果的整个流程中，要么每个环节都处理

成功，要么失败回滚（回到从未处理过的状态）！

Flink 在目前的各类分布式计算引擎中，对 EOS 的支持是最完善的；

在合理的数据源选择，合理的算子选择，合理的目标存储系统选择，合适的参数配置下，可以实现严格意义上的端到端 EOS

source 端的保证

Flink 的很多 source 算子都能为 EOS 提供保障，如 kafka Source：

- ✓ 能够记录偏移量
- ✓ 能够重放数据
- ✓ 将偏移量记录在 state 中，与下游的其他算子的 state 一起，经由 checkpoint 机制实现了“状态数据的”快照统一

算子状态的 EOS 语义保证

基于分布式快照算法：（Chandy-Lamport），flink 实现了整个数据流中各算子的状态数据快照统一；

既：一次 checkpoint 后所持久化的各算子的状态数据，确保是经过相同数据的影响；

这样一来，就能确保：

- ✓ 一条（或一批）数据要么是经过了完整正确处理；
- ✓ 如果这条（批）数据在中间任何过程失败，则重启恢复后，所有算子的 state 数据都能回到这条数据从未处理过时的状态

多易教育

sink 端的保证

从前文所述的 source 端和内部 state 的容错机制来看，一批数据如果在 sink 端写出过程中失败（可能已经有一部分数据进入目标存储系统），则重启后重放这批数据时有可能造成目标存储系统中出现数据重复，从而破坏 EOS；

对此，flink 中也设计了相应机制来确保 EOS

- ✓ 采用幂等写入方式
- ✓ 采用两阶段提交（2PC，two phase）事务写入方式
- ✓ 采用预写日志 2PC 提交方式

11. Flink checkpoint 机制

11.1. 分布式 Checkpoint 的难题

由于 flink 是一个分布式的系统，数据在流经系统中各个算子时，是有先后顺序的；换个角度来说就是：整个系统对一条数据的处理过程，并不是一个原子性的过程；

这样一来，对系统中各算子的状态进行持久化（快照），就成了一件棘手的事情；

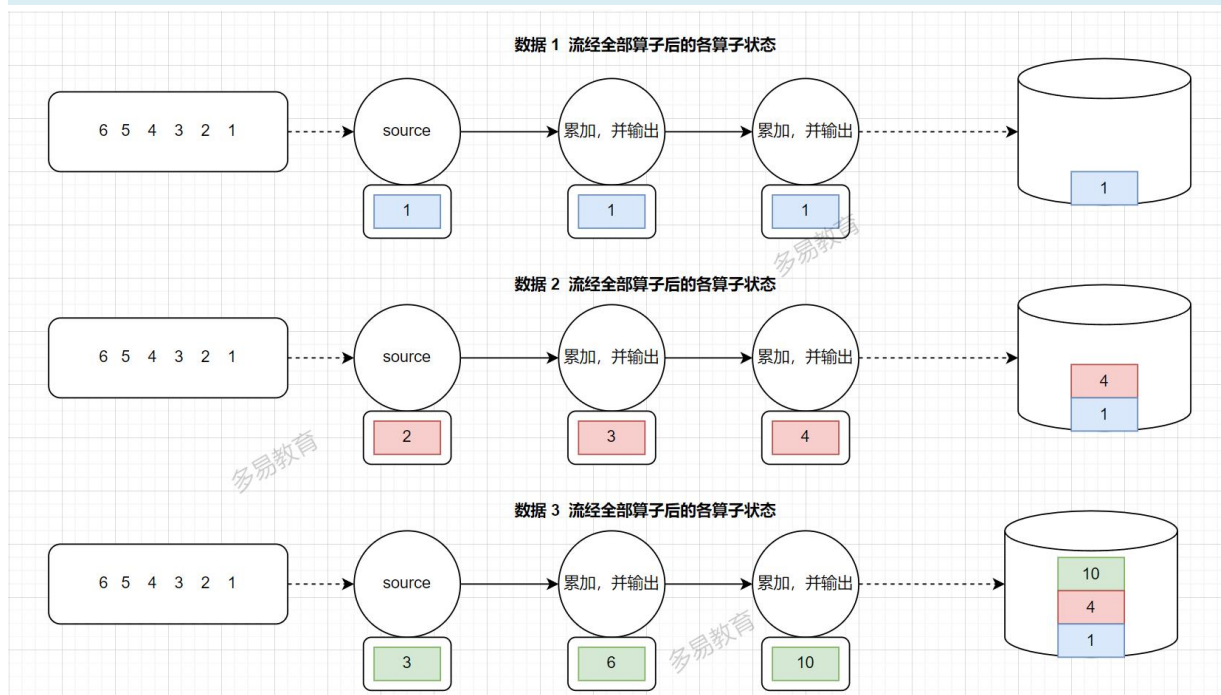
来看如下数据处理场景：

算子 1： 从 kafka 中读取数据，并在状态中记录消费位移

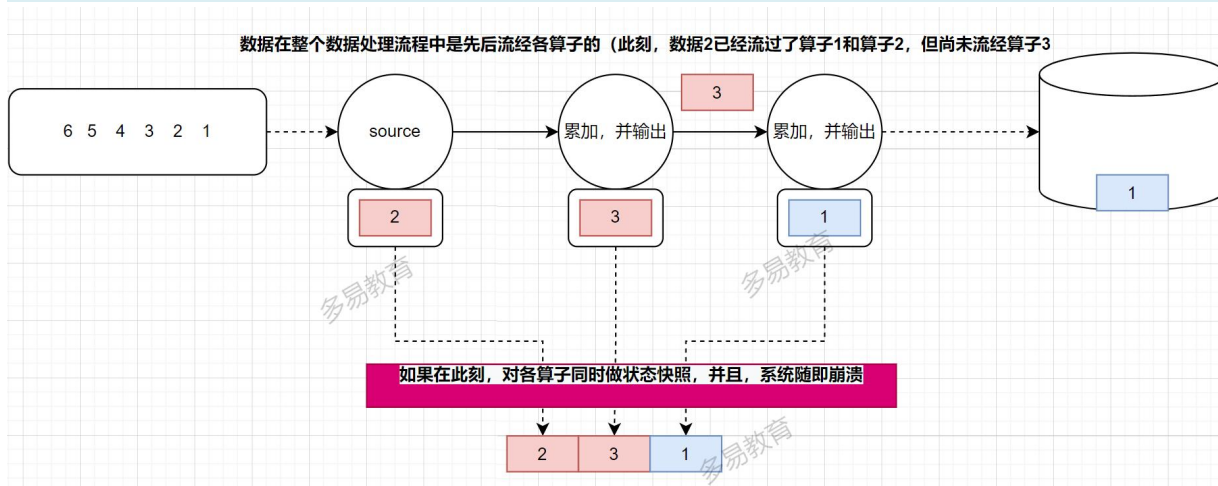
算子 2： 对流入的整数进行累加，并输出累加结果

算子 3： 对流入是整数进行累加，并输出累加结果

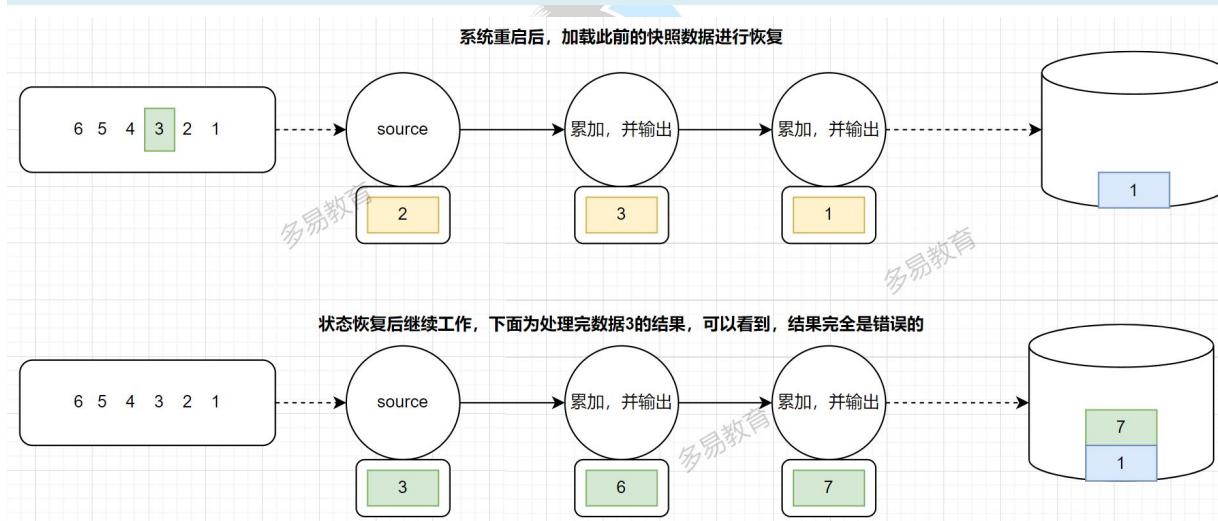
先注意观察正常情况下，整个系统的各算子状态变化及最终输出结果



出于容错考虑，需要在某个时机对整个系统各个算子的状态数据进行快照持久化，如下



系统重启后加载快照数据，恢复各算子崩溃前的状态，但是会发现，处理结果相对正常时完全错误



从最终结果来看，整个计算

- 丢失了数据 2 的结果
- 数据 3 则因为内部状态的紊乱而产生了错误的结果

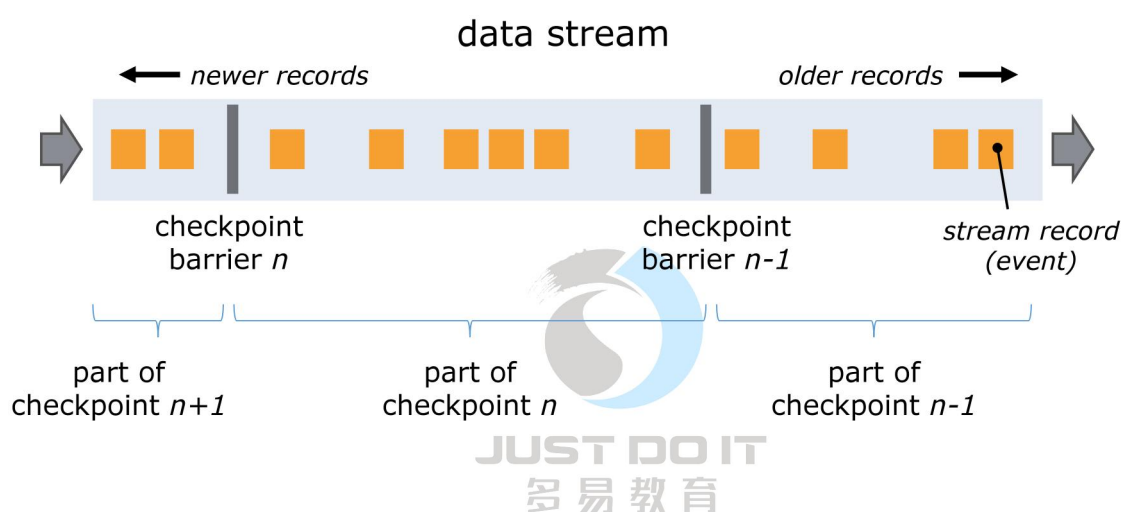
11.2. Checkpoint 的核心要点

checkpoint 是 flink 内部对状态数据的快照机制；

flink 的 checkpoint 机制是源于 Chandy-Lamport 算法（分布式快照算法）；

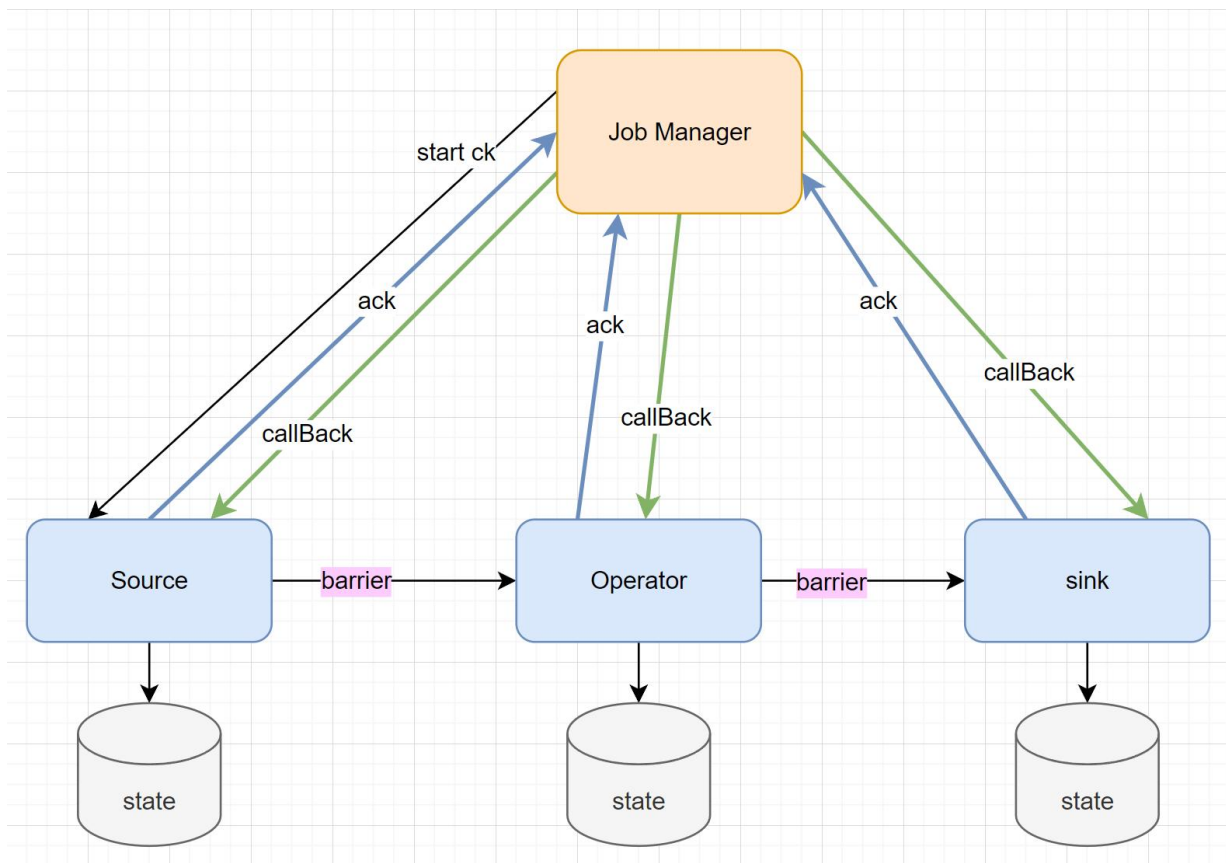
底层逻辑：通过插入序号单调递增的 barrier，把无界数据流划分成逻辑上的数据批（段），并通过段落标记（barrier）来为这段数据的处理，加持"事务（transaction）"特性：

- 每一段数据流要么被完整成功处理；
- 要么回滚一切不完整的影响（状态变化）；



11.3. Checkpoint 的整体流程

- 1) JobMaster 即 CheckpointCoordinator 会定期向每个 source task 发送命令 start checkpoint (trigger checkpoint)；
- 2) 当 source task 收到 trigger checkpoint 指令后，产生 barrier 并通过广播的方式发送到下游。source task 及所有其他 task, 收到 barrier-n, 会执行本地 checkpoint-n, 当 checkpoint-n 完成后，向 JobMaster 发送 ack；
- 3) 当流图的所有节点都完成 checkpoint-n, JobMaster 会收到所有节点的 ack, 那么就表示完成 checkpoint-n, 随即就会向所有 task 广播一条 checkpoint-n 全部完成的通知消息；



说明：checkpoint 机制的调用流程实质是 2PC。JobMaster 是协调者，所有 operator task 是执行者。start checkpoint 是 pre-commit 的开始信号，而每个 operator task 的 checkpoint 是 pre-commit 过程，ack 是执行者 operator task 反馈给协调者 JobMaster，最后 callback 是 commit。

1) Barrier 会在数据流源头被注入并行数据流中

Barrier-n 所在的位置就是恢复时数据重新处理的起始位置。例如，在 Kafka 中，这个位置就是最后一个记录在分区内的消费位移（offset），作业恢复时，会根据这个位置从这个偏移量向 kafka 请求数据，这个偏移量就是 State 中保存的内容之一。

2) Barrier 接着向下游传递

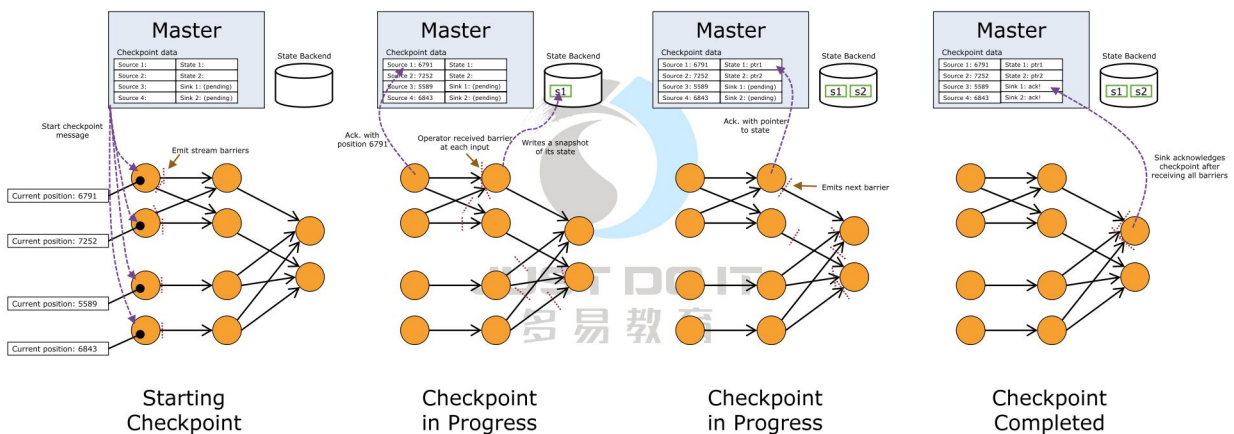
当一个非数据源算子从所有的输入流中收到 Barrier-n 时，该算子就会对自己的 State 保存快照，并向自己的下游广播发送 Barrier-n；

3) 一旦 Sink 算子接收到 Barrier，有两种情况：

(1) 如果是引擎内严格一次处理保证，当 Sink 算子已经收到了所有上游的 Barrier-n 时，Sink 算子对自己的 State 进行快照，然后通知检查点协调器(CheckpointCoordinator)。当所有的算子都向检查点协调器汇报成功之后，检查点协调器向所有的算子确认本次快照完成。

(2) 如果是端到端严格一次处理保证，当 Sink 算子已经收到了所有上游的 Barrier-n 时，Sink 算子对自己的 State 进行快照，并预提交事务（两阶段提交的第一阶段），再通知检查点协调器(CheckpointCoordinator)，检查点协调器向所有的算子确认本次快照完成，Sink 算子提交事务（两阶段提交的第二阶段），本次事务完成；

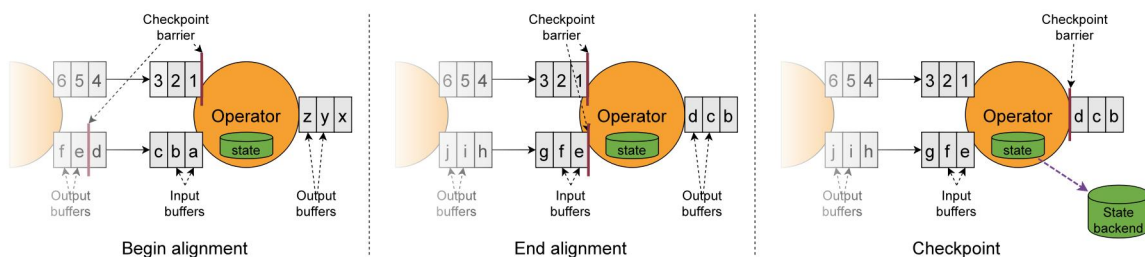
snapshotting operator 算子



- For each parallel stream data source, the offset/position in the stream when the snapshot was started
- For each operator, a pointer to the state that was stored as part of the snapshot

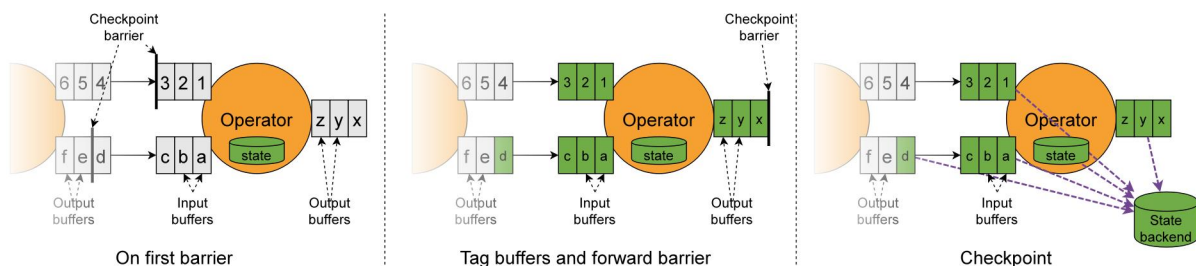
11.4. 对齐与非对齐 checkpoint

对齐的 checkpoint



- 算子收到数字流 Barrier,字母流对应 barrier 尚未到达 ;
- 算子收到数字流 Barrier,会继续从数字流中接收数据,但这些流只能被搁置,记录不能被处理,而是放入缓存中,等待字母流 Barrier 到达。在字母流到达前, 1, 2, 3 数据已经被缓存。
- 字母流到达,算子开始对齐 State 进行异步快照,并将 Barrier 向下游广播,并不等待快照完毕。
- 算子做异步快照,首先处理缓存中积压数据,然后再从输入通道中获取数据。

非对齐的 checkpoint



barrier 不对齐: 就是指当还有其他流的 barrier 还没到达时, 为了不影响性能, 也不用理会, 直接处理 barrier 之后的数据。等到所有流的 barrier 的都到达后, 就可以对该 Operator 做 CheckPoint 了;

如果不对齐, 那么在 chk-100 快照之前, 已经处理了一些 chk-100 对应的 offset 之后的数据, 当程序从 chk-100 恢复任务时, chk-100 对应的 offset 之后的数据还会被处理一次, 所以就出现了重复消费。

11.5. checkpoint 相关参数和 API

```
env.enableCheckpointing(3000); // 传入的参数是 checkpoint 的间隔时间
```

```
// 指定 checkpoint 数据的存储位置
```

```
env.getCheckpointConfig().setCheckpointStorage(new  
Path("hdfs://doit01:8020/flink-jobs-checkpoints/"));
```

```
// 容许 checkpoint 失败的最大次数
```

```
env.getCheckpointConfig().setTolerableCheckpointFailureNumber(10);

// checkpoint 的算法模式（是否需要对齐）

env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.EXACTLY_ONCE);

// job 取消时是否保留 checkpoint 数据

env.getCheckpointConfig().setExternalizedCheckpointCleanup(CheckpointConfig.ExternalizedCheckpointCleanup.RETAIN_ON_CANCELLATION);

// 设置 checkpoint 对齐的超时时间

env.getCheckpointConfig().setAlignedCheckpointTimeout(Duration.ofMillis(2000));

// 两次 checkpoint 的最小间隔时间，为了防止两次 checkpoint 的间隔时间太短

env.getCheckpointConfig().setCheckpointInterval(2000);

// 最大并行的 checkpoint 数

env.getCheckpointConfig().setMaxConcurrentCheckpoints(3);

// 要用状态，最好还要指定状态后端(默认是 HashMapStateBackend)

env.setStateBackend(new EmbeddedRocksDBStateBackend());
```

12. sink 端容错机制

12.1. 幂等写入方式

Sink 端主要的问题是，作业失败重启时，数据重放可能造成最终目标存储中被写入了重复数据；

如果目标存储系统支持幂等写入，且数据中有合适的 key（主键），则 flink 的 sink 端完全可以利用目标系统的幂等写入特点，来实现数据的最终一致（精确一次）；

只是，幂等写入的方式，能实现最终一致，但依然存在弊端：

- “过程中的不一致”
- 可能导致下游数据消费者出现脏读；

注： 动态过程不一致，主要出现在“输出结果非确定”的计算场景中，如，

输入： guid, event_id, event_cout

输出： guid, event_id, event_cout, **insert_time**（数据插入的时间或者随机数）

则重复写入 guid,event_id 相同的两次数据时，第一次的值和后面覆盖的值，是发生了变化的

多易教育

12.2. 两阶段事务写入

12.2.1. 核心流程

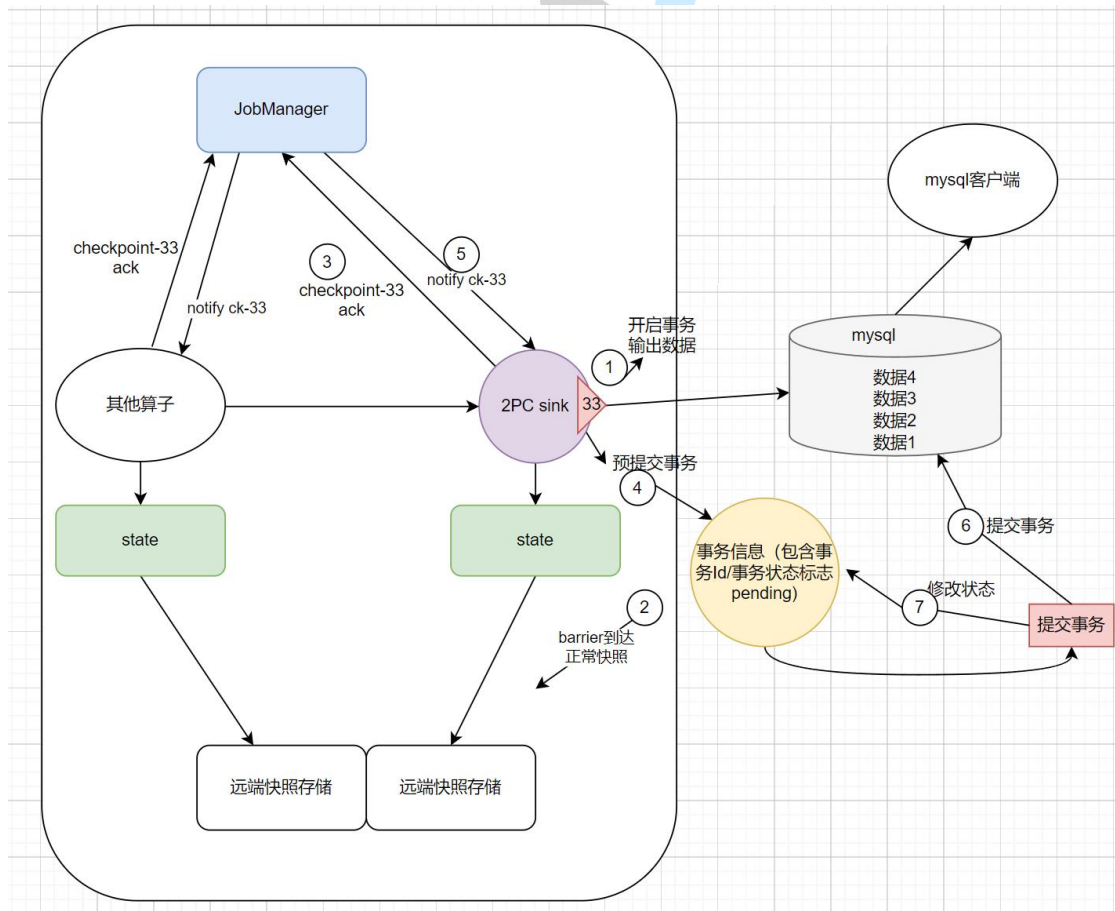
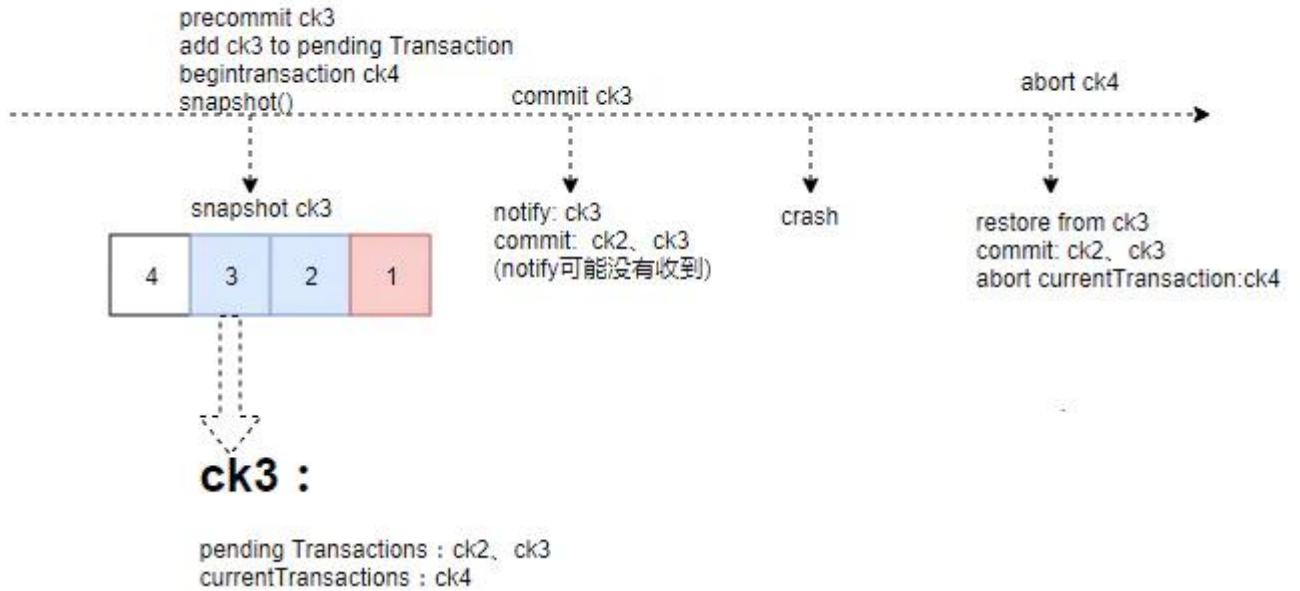
Flink 中的两阶段事务提交 sink，主要是利用了上述的“checkpoint 两阶段提交协议”和目标存储系统的事务支持机制（比如 mysql 等）；

Flink 中两阶段事务提交的核心过程如下：

- ✓ Sink 算子在一批数据处理过程中，先通过预提交事务开始对外输出数据
- ✓ 待这批数据处理完成(即收到了 checkpoint 信号)后,向 checkpoint coordinator 上报自身 checkpoint 完成信息
- ✓ checkpoint coordinator 收到所有算子任务的 checkpoint 完成信息后，再向各算子任务广播本次

checkpoint 全局完成信息

- ✓ 两阶段事务提交 sink 算子收到 checkpoint coordinator 的回调信息时，执行事务 commit 操作；



12.2.2. 代码示例

Flink 为开发者设计了两阶段事务提交的 sink 算子接口（TwoPhaseCommitSinkFunction）；用户通过实现该接口，即可获得一个实现了两阶段事务提交机制的 sink 算子；

代码示例如下：

```
/**
 * Example program that demonstrates the behavior of a 2-phase-commit (2PC) sink that writes
 * output to files.
 *
 * The 2PC sink guarantees exactly-once output by writing records immediately to a
 * temp file. For each checkpoint, a new temp file is created. When a checkpoint
 * completes, the corresponding temp file is committed by moving it to a target directory.
 *
 * The program includes a MapFunction that throws an exception in regular intervals to simulate
 * application failures.
 *
 * You can compare the behavior of a 2PC sink and the regular print sink in failure cases.
 *
 * - The TransactionalFileSink commits a file to the target directory when a checkpoint completes
 * and prevents duplicated result output.
 *
 * - The regular print() sink writes to the standard output when a result is produced and
 * duplicates result output in case of a failure.
 */
object TransactionSinkExample {

  def main(args: Array[String]): Unit = {

    // set up the streaming execution environment

    val env = StreamExecutionEnvironment.getExecutionEnvironment
```

```
// checkpoint every 10 seconds

env.getCheckpointConfig.setCheckpointInterval(10 * 1000)


// use event time for the application

env.setStreamTimeCharacteristic(TimeCharacteristic.EventTime)

// configure watermark interval

env.getConfig.setAutoWatermarkInterval(1000L)


// ingest sensor stream

val sensorData: DataStream[SensorReading] = env

    // SensorSource generates random temperature readings

    .addSource(new ResetableSensorSource)

    // assign timestamps and watermarks which are required for event time

    .assignTimestampsAndWatermarks(new SensorTimeAssigner)


// compute average temperature of all sensors every second

val avgTemp: DataStream[(String, Double)] = sensorData

    .timeWindowAll(Time.seconds(1))

    .apply((w, vals, out: Collector[(String, Double)]) => {

        val avgTemp = vals.map(_.temperature).sum / vals.count(_ => true)

        // format window timestamp as ISO timestamp string

        val epochSeconds = w.getEnd / 1000

        val tString = LocalDateTime.ofEpochSecond(epochSeconds, 0, ZoneOffset.UTC)

        .format(DateTimeFormatter.ISO_LOCAL_DATE_TIME)

        // emit record

        out.collect((tString, avgTemp))

    })

)
```

```
// generate failures to trigger job recovery

.map(new FailingMapper[(String, Double)](16)).setParallelism(1)

// OPTION 1 (comment out to disable)
// -----
// write to files with a transactional sink.
// results are committed when a checkpoint is completed.

val (targetDir, transactionDir) = createAndGetPaths

avgTemp.addSink(new TransactionalFileSink(targetDir, transactionDir))

// OPTION 2 (uncomment to enable)
// -----
// print to standard out without write-ahead log.
// results are printed as they are produced and re-emitted in case of a failure.

// avgTemp.print()
// // enforce sequential writing
// .setParallelism(1)

env.execute()
}

/** Creates temporary paths for the output of the transactional file sink. */
def createAndGetPaths: (String, String) = {

  val tempDir = System.getProperty("java.io.tmpdir")

  val targetDir = s"$tempDir/committed"

  val transactionDir = s"$tempDir/transaction"

  val targetPath = Paths.get(targetDir)

  val transactionPath = Paths.get(transactionDir)
```

```
if (!Files.exists(targetPath)) {
    Files.createDirectory(targetPath)
}

if (!Files.exists(transactionPath)) {
    Files.createDirectory(transactionPath)
}

(targetDir, transactionDir)
}
}

/**
 * Transactional sink that writes records to files and commits them to a target directory.
 *
 * Records are written as they are received into a temporary file. For each checkpoint, there is
 * a dedicated file that is committed once the checkpoint (or a later checkpoint) completes.
 */
class TransactionalFileSink(val targetPath: String, val tempPath: String)
    extends TwoPhaseCommitSinkFunction[(String, Double), String, Void](
        createTypeInfo[String].createSerializer(new ExecutionConfig),
        createTypeInfo[Void].createSerializer(new ExecutionConfig)) {

    var transactionWriter: BufferedWriter = _

    /**
     * Creates a temporary file for a transaction into which the records are
     * written.
     */
}
```

```
override def beginTransaction(): String = {

    // path of transaction file is constructed from current time and task index

    val timeNow = LocalDateTime.now(ZoneId.of("UTC"))

        .format(DateTimeFormatter.ISO_LOCAL_DATE_TIME)

    val taskIdx = this.getRuntimeContext.getIndexOfThisSubtask

    val transactionFile = s"$timeNow-$taskIdx"

    // create transaction file and writer

    val tFilePath = Paths.get(s"$tempPath/$transactionFile")

    Files.createFile(tFilePath)

    this.transactionWriter = Files.newBufferedWriter(tFilePath)

    println(s"Creating Transaction File: $tFilePath")

    // name of transaction file is returned to later identify the transaction

    transactionFile

}

/** Write record into the current transaction file. */

override def invoke(transaction: String, value: (String, Double), context: Context[_]): Unit = {

    transactionWriter.write(value.toString)

    transactionWriter.write('\n')

}

/** Flush and close the current transaction file. */

override def preCommit(transaction: String): Unit = {

    transactionWriter.flush()

    transactionWriter.close()

}
```

```
/** Commit a transaction by moving the pre-committed transaction file
 * to the target directory.
 */
override def commit(transaction: String): Unit = {
  val tFilePath = Paths.get(s"$tempPath/$transaction")

  // check if the file exists to ensure that the commit is idempotent.
  if (Files.exists(tFilePath)) {
    val cFilePath = Paths.get(s"$targetPath/$transaction")

    Files.move(tFilePath, cFilePath)
  }
}

/** Aborts a transaction by deleting the transaction file. */
override def abort(transaction: String): Unit = {
  val tFilePath = Paths.get(s"$tempPath/$transaction")

  if (Files.exists(tFilePath)) {
    Files.delete(tFilePath)
  }
}
}
```

12.2.3. 优缺点

要使用 `TwopahseCommitSinkFunction` 对外部系统具有如下要求

- 外部系统必须提供事务支持或者能够可以 Sink 去模拟事务（`BucketingSink` 的原子命名模拟保证了提交的原子性），在事务 `Committed` 之前不能对下游系统可见
- 在快照间隔内事务不能 `timeout`，否则无法以事务的方式提交输出。

- 事务必须在收到 job manager 发送的 global committed 的消息后，才能 committed。在 fail recovery 的时候，若恢复时间较长（载入大状态），若事务关闭（事务 timeout），该数据会丢失。
- 在 fail recovery 后，事务需要支持恢复之前 pending 的事务，并进行提交。（一些外部系统能够使用 transaction id 去 commit 或者 abort 之前的事务）
- 事务的提交必须是幂等的，因为在恢复时，会重新提交一遍 pending transaction，因此需要对同一个事务的 commit 是幂等的。

可以看到外部系统不但要支持事务，同时也要能支持根据事务 id 去恢复之前的事务。



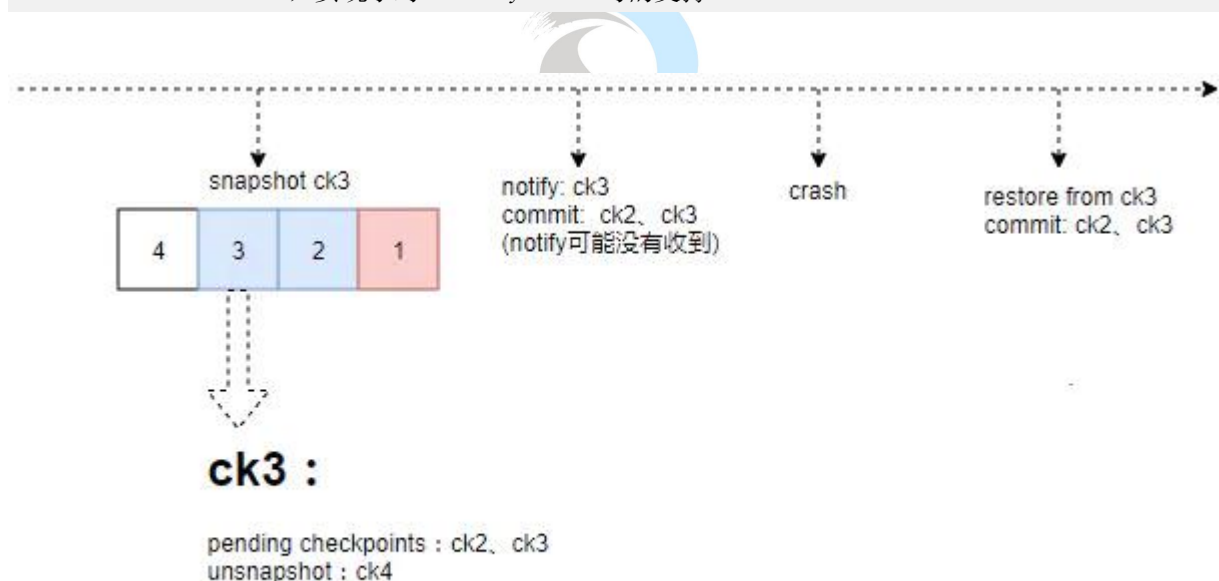
12.3. 预写日志 2pc 提交

12.3.1. 核心流程

- 把结果数据先当成状态保存，然后在收到 checkpoint 完成的通知时，一次性写入 sink 系统；
- 简单易于实现，由于数据提前在状态后端中做了缓存，所以无论什么 sink 系统，都能用这种方式一批搞定；
- `DataStream API` 提供了一个模板类: **`GenericWriteAheadSink`**，来实现这种事务性 sink；

抽象类 `GenericWriteAheadSink` 实现了缓存数据。在收到上游的消息时，会将消息存储在 `state` 中(保存于 `taskowned` 目录下)，收到全局一致快照完成的 `notify` 后，调用 `sendValues(Iterable<IN> values, long checkpointId, long timestamp)` 方法向下游系统发送 `global commit` 的消息。

Cassandra 的 connector `CassandraRowWriteAheadSink`、`CassandraTupleWriteAheadSink` 继承了 `GenericWriteAheadSink`，实现了对 `exactly once` 写的支持



12.3.2. 代码示例

继承 GenericWriteAheadSink 抽象类

```
package cn.doitedu.flink.java.demos;

import org.apache.commons.lang3.RandomUtils;

import org.apache.flink.api.common.eventtime.Watermark;

import org.apache.flink.api.common.typeutils.TypeSerializer;

import org.apache.flink.streaming.api.functions.sink.SinkFunction;

import org.apache.flink.streaming.runtime.operators.CheckpointCommitter;

import org.apache.flink.streaming.runtime.operators.GenericWriteAheadSink;

public class DoiteduWriteAheadSink extends GenericWriteAheadSink<String> {

    public TwoPcSink(CheckpointCommitter committer, TypeSerializer<String> serializer, String jobID) throws
Exception {

        super(committer, serializer, jobID);

    }

    @Override

    protected boolean sendValues(Iterable<String> values, long checkpointId, long timestamp) throws Exception
{

        for (String value : values) {

            if(value.equals("X") && RandomUtils.nextInt(1,5)%3==0) throw new
Exception("error.....");

            System.out.println(checkpointId + "," + value);

        }

        return true;

    }

}
```

实现 CheckpointCommitter 接口

```
package cn.doitedu.flink.java.demos;

import org.apache.commons.io.FileUtils;

import org.apache.commons.lang3.RandomUtils;

import org.apache.flink.api.common.functions.MapFunction;

import org.apache.flink.api.common.typeinfo.TypeInformation;

import org.apache.flink.streaming.api.CheckpointingMode;

import org.apache.flink.streaming.api.datastream.DataStreamSource;

import org.apache.flink.streaming.api.datastream.SingleOutputStreamOperator;

import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

import org.apache.flink.streaming.runtime.operators.CheckpointCommitter;

import java.io.File;

import java.nio.file.Files;

import java.nio.file.Path;

import java.nio.file.Paths;

import java.util.Locale;

public class Teacher {

    public static void main(String[] args) throws Exception {

        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

        env.enableCheckpointing(2000, CheckpointingMode.EXACTLY_ONCE);

        env.getCheckpointConfig().setCheckpointStorage("file:///d:/ckpt");

        DataStreamSource<String> source = env.socketTextStream("localhost", 9999);
```

```
SingleOutputStreamOperator<String> s1 = source.map(new MapFunction<String, String>() {  
  
    @Override  
  
    public String map(String value) throws Exception {  
  
        return value.toUpperCase();  
  
    }  
  
});  
  
s1.transform("x", TypeInformation.of(String.class), new DoiteduWriteAheadSink(new  
CheckpointCommitter() {  
  
    @Override  
  
    public void open() throws Exception {  
  
    }  
  
    @Override  
  
    public void close() throws Exception {  
  
    }  
  
    @Override  
  
    public void createResource() throws Exception {  
  
        Path path = Paths.get("d:/xxx" + this.jobId);  
  
        if(!Files.exists(path)) {  
  
            Files.createDirectory(path);  
  
        }  
  
    }  
  
    @Override  
  
    public void commitCheckpoint(int subtaskIdx, long checkpointID) throws Exception {  
  
        Path path = Paths.get("d:/xxx" + this.jobId + "/" + subtaskIdx);  
  
        if(!Files.exists(path)) {
```

```
        Files.createFile(path);

    }

    System.out.println("提交快照: " + subtaskIdx + ", " + checkpointID );

    Files.write(path, (checkpointID + "").getBytes());

}

@Override

public boolean isCheckpointCommitted(int subtaskIdx, long checkpointID) throws Exception {

    Path path = Paths.get("d:/xxx" + this.jobId + "/" + subtaskIdx);

    if(!Files.exists(path)) return false;

    String idStr = Files.readAllLines(path).get(0);

    return checkpointID <= Long.parseLong(idStr);

}

}, TypeInformation.of(String.class).createSerializer(env.getConfig(), "jobx")).setParallelism(1);

env.execute();

}

}
```

12.3.3. 优缺点

write ahead log 不能实现真正的 exactly once 传输，下列两种情况会导致数据重复

- sendValues()在向外部 commit 数据时 crash，若外部系统不支持原子写，则恢复后 replay 会造成数据重复。
- sendValues()方法成功完成，但程序在 CheckpointCommitter 在被调用前或者 CheckpointCommitter commit checkpointid 失败，Sink 在 recovery 时会重复提交。

要避免重复写，需要外部系统支持 upsert 操作的 key-value 存储或关系型数据库，后续的 commit 会覆盖之前 commit 失败的数据，达到最终一致性。与幂等性 Sink 不同的是，WAL sink 向外部系统提交的是处于 global commit 的数据，消除了不确定性带来的影响。

- 性能的影响，占用内存 buffer 数据，sendValues()时阻塞 Sink，降低影响吞吐。

13. flink 容错配置及测试

13.1. task 级失败重启恢复状态

Task 级别的故障重启，是系统自动进行的；

13.1.1. Task 失败的自动重启策略

- **Fixed Delay Restart Strategy**

固定延迟重启策略

- ✓ 会尝试一个给定的次数来重启 Job；
- ✓ 如果超过最大的重启次数（默认 Integer.MAX_VALUE 次），Job 最终将失败；
- ✓ 在连续两次重启尝试之间，重启策略会等待一个固定时间；

- **Exponential Delay Restart Strategy**

本策略：故障越频繁，两次重启间的惩罚间隔就越长

initialBackoff 重启间隔惩罚时长的初始值：1s

maxBackoff 重启间隔最大惩罚时长：60s

backoffMultiplier 重启间隔时长的惩罚倍数：2（每多故障一次，重启延迟惩罚就在 上一次的惩罚时长上 * 倍数）

resetBackoffThreshold 重置惩罚时长的平稳运行时长阈值（平稳运行达到这个阈值后，如果再故障，则故障重启延迟时间重置为了初始值：1s）

jitterFactor 取一个随机数，来加在重启时间点上，已让每次重启的时间点呈现一定随机性

job1:	9.51	$9.53+2*0.1$	9.57	
job2:	9.51	$9.53+2*0.15$	9.57	
job3:	9.51	$9.53+2*0.8$	9.57	

- **Failure Rate Restart Strategy**

失败率重启策略：

- ✓ 在 task 故障后会尝试重启 failover
- ✓ 但是超过失败率后，Job 会最终被认定失败
- ✓ 在两个连续的重启尝试之间，重启策略会等待一个固定的时间

- **No Restart Strategy**

无重启策略：

✓ 发生 task 级别故障，不尝试 failover，而是直接让 job 失败；

- **Fallback Restart Strategy**

回退策略：

✓ 就是直接使用配置文件中所配置的 failover 策略；

13.1.2. 重启策略的设置代码

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

env.setRestartStrategy(RestartStrategies.fixedDelayRestart(

    3, // number of restart attempts

    Time.of(10, TimeUnit.SECONDS) // delay

));
```

JUST DO IT
多易教育

13.1.3. Failover-strategy 策略

jobmanager.execution.failover-strategy: region

本参数的含义是： 当一个 task 故障，需要 restart 的时候，是 restart 整个 job 中的所有 task，还是只 restart 一部分 task；

- **Restart All Failover Strategy**

重启所有 task，来实现 failover

- **Restart Pipelined Region Failover Strategy**

本策略将 task 划分成无关联的 region。

当一个 task 失败时，该策略会计算出需要重启的最小 region 集，在一部分场景中，该策略相比 Restart All 会少重启一些 task。

region 是一个通过 pipelined 数据交换所连接的 task 集合； batch 数据交换则意味着 region 的边界；

- All data exchanges in a DataStream job or Streaming Table/SQL job are pipelined.
- All data exchanges in a Batch Table/SQL job are batched by default.
- The data exchange types in a DataSet job are determined by the ExecutionMode which can be set through ExecutionConfig.

region 重启必要性的计算策略:

- region 包含需要重启的失败的 task
- 结果分区被必须重启的 region 所依赖且不可用时, 产出结果分区的 region 也需要被重启;
- 如果一个需要被重启的 region, 则消费它数据的 region 也需要被重启; (这是为了确保数据一致性, 因为不确定的计算或分区, 可能造成不同的分区)

13.2. cluster 级失败重启恢复状态

13.2.1. save points 概念

- savepoint (保存点) 是基于 Flink 检查点机制的完整快照备份机制, 用来保存状态, 可以在另一个集群或者另一个时间点从保存的状态中将作业恢复回来;
- 适用于应用升级、集群迁移、Flink 集群版本更新、A/B 测试以及假定场景、暂停和重启、归档等场景;
- 保存点可以视为一个(算子 ID -> State) 的 Map, 对于每一个有状态的算子, Key 是算子 ID, Value 是算子的 State;

save points 相关配置 (flink-conf.yaml)

```
# Default savepoint target directory

state.savepoints.dir: hdfs:///flink/savepoints
```

13.2.2. savepoints 操作命令

IDEA 本地运行时 savepoints 恢复测试

```
Configuration conf = new Configuration();

// 指定想要从中恢复的检查点 (savepoints) 目录

conf.setString("execution.savepoint.path", "file:///D:/checkpoint/7ecbd4f9106957c42109bcde/chk-544");

StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(conf);
```

集群运行时，手动触发 savepoint

```
# 触发一次 savepoint

$ bin/flink savepoint :jobId [:targetDirectory]

# 为 yarn 模式集群触发 savepoint

$ bin/flink savepoint :jobId [:targetDirectory] -yid :yarnAppId

# 停止一个 job 集群并触发 savepoint

$ bin/flink stop --savepointPath [:targetDirectory] :jobId

# 从一个指定 savepoint 恢复启动 job 集群

$ bin/flink run -s :savepointPath [:runArgs]

# 删除一个 savepoint
```

```
$ bin/flink savepoint -d :savepointPath
```

集群运行时，指定目录恢复 savepoint

从一个指定 savepoint 恢复启动 job 集群

```
$ bin/flink run -s :savepointPath [:runArgs]
```

删除一个 savepoint

```
$ bin/flink savepoint -d :savepointPath
```

集群运行时，删除指定 savepoint

删除一个 savepoint

```
$ bin/flink savepoint -d :savepointPath
```

13.2.3. 从 savepoints 恢复注意要点

从保存点恢复作业并不简单，尤其是在作业变更(如修改逻辑、修复 bug) 的情况下， 需要考虑：

(1) 算子的顺序改变

如果对应的 UID 没变，则可以恢复，如果对应的 UID 变了恢复失败。

(2) 作业中添加了新的算子

如果是无状态算子，没有影响，可以正常恢复，如果有状态的算子，跟无状态的算子一样处理。

(3) 从作业中删除了一个有状态的算子

默认需要恢复保存点中所记录的所有算子的状态，如果删除了一个有状态的算子，从保存点回复的时候被删除的 OperatorID 找不到，所以会报错，可以通过在命令中添加
`-- allowNonReStoredSlale (short: -n )` 跳过无法恢复的算子 。

（4）添加和删除无状态的算子

如果手动设置了 UID 则可以恢复，保存点中不记录无状态的算子，如果是自动分配的 UID，那么有状态算子的可能会变(Flink 一个单调递增的计数器生成 UID, DAG 改版，计数器极有可能会变) 很有可能恢复失败。



13.2.4. 从 savepoint 恢复时，快照数据的重分配问题

Flink 程序，允许在某次重启时，修改程序的 JobGraph 图（比如改变算子顺序，改变算子并行度等）而且，在修改了程序的 JobGraph 图后，依然可以加载之前的状态快照数据，只不过，可能需要对之前的状态快照数据，在新的 JobGraph 下进行数据重分配；

- Operator state

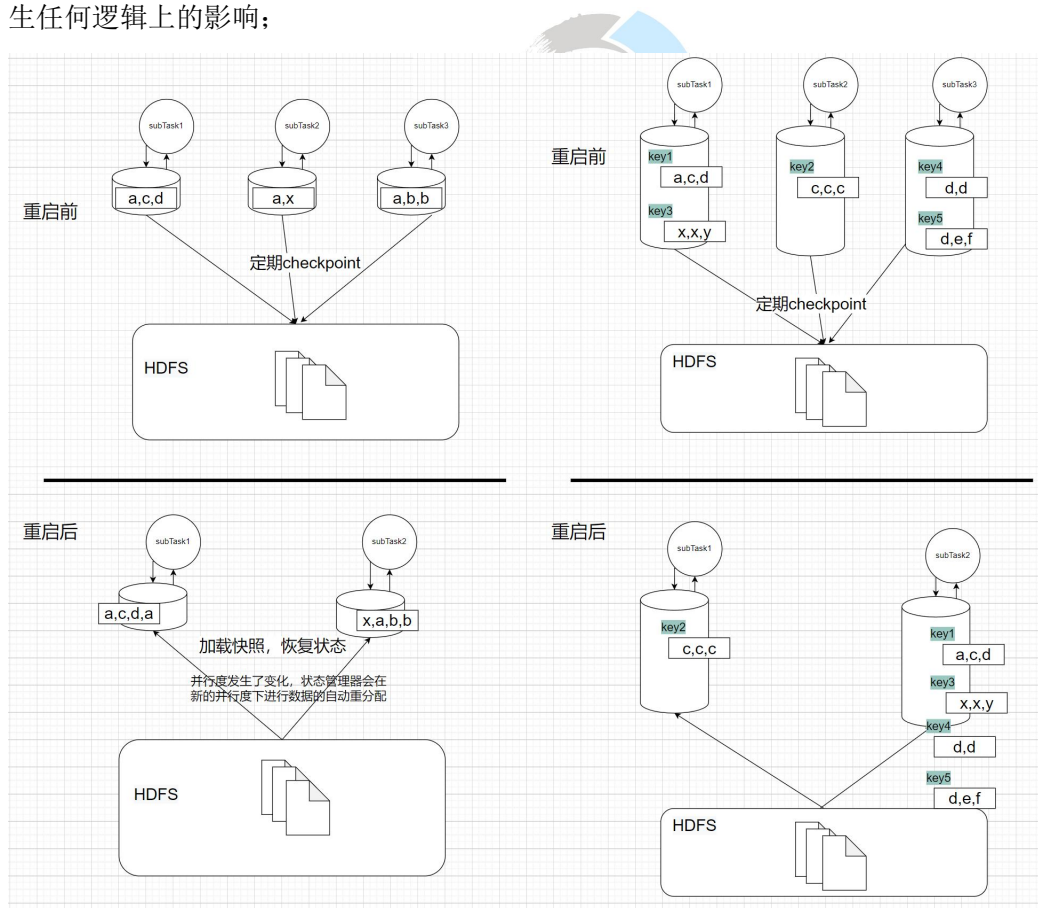
快照数据在重分配时，可能会对用户程序的计算逻辑产生不可预料的影响；

- ✓ UnionListState 用广播模式重分配；

- ✓ Liststate 用 round-robin 模式恢复；

- Keyed state

快照数据在重分配时，因为程序处理数据时接收数据的规律和状态的分配规律完全一致，所以不会产生任何逻辑上的影响；



13.3. 容错测试案例

在代码中制造一定概率的异常发生；

然后观察 task 自动重启、手动 job 重启时，数据从输入到输出的端到端一致性；

```
Configuration conf = new Configuration();

//conf.setString("execution.savepoint.path",
"file:///D:/checkpoint/7ecbd4f9cad68d43106957c42109bcde/chk-544");

StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(conf);

// 开启 checkpoint 机制

env.enableCheckpointing(500, CheckpointingMode.EXACTLY_ONCE);

env.getCheckpointConfig().setCheckpointStorage("file:///d:/checkpoint");

// 设置重启策略

env.setRestartStrategy(RestartStrategies.fixedDelayRestart(Integer.MAX_VALUE, 1000));

// 构造一个 kafka source

KafkaSource<String> source = KafkaSource.<String>builder()

    .setBootstrapServers("doit01:9092")

    .setTopics("b05")

    .setGroupId("x06")

    .setValueOnlyDeserializer(new SimpleStringSchema())

    .setProperty(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false")
```

```
// kafkaSource 的做状态 checkpoint 时，默认会向__consumer_offsets 提交一下状态中记录的偏移量

// 但是，flink 的容错并不优选依赖__consumer_offsets 中的记录，所以可以关闭该默认机制

.setProperty("commit.offsets.on.checkpoint","false") // 默认是 true


// kafkaSource 启动时，获取起始位移的策略设置，如果是 committedOffsets ，则是从之前所记录的偏移量开始

// 如果没有可用的之前记录的偏移量，则用策略 OffsetResetStrategy.LATEST 来决定

.setStartingOffsets(OffsetsInitializer.committedOffsets(OffsetResetStrategy.LATEST))

.build();


// 构造一个支持精确一次的 jdbcSink

SinkFunction<String> exactlyOnceSink = JdbcSink.exactlyOnceSink(

    "insert into t_once values (?)",

    new JdbcStatementBuilder<String>() {

        @Override

        public void accept(PreparedStatement preparedStatement, String s) throws SQLException {

            preparedStatement.setString(1, s);

        }

    },

    JdbcExecutionOptions.builder()

        .withMaxRetries(3)

        .withBatchSize(1)

        .build(),
```

```
JdbcExactlyOnceOptions.builder()

    .withTransactionPerConnection(true) // mysql 不支持同一个连接上存在并行的多个未完成的事务，
    必须把该参数设置为 true

    .build(),

    new SerializableSupplier<XADataSource>() {

        @Override

        public XADataSource get() {

            // XADataSource 就是 jdbc 连接，不过它是支持分布式事务的连接

            // 而且它的构造方法，不同的数据库构造方法不同

            MysqlXADataSource xaDataSource = new MysqlXADataSource();

            xaDataSource.setUrl("jdbc:mysql://doit01:3306/abc");

            xaDataSource.setUser("root");

            xaDataSource.setPassword("ABC123.abc123");

            return xaDataSource;

        }

    }

);

// 从 kafka 加载数据

DataStreamSource<String> s1 = env.fromSource(source, WatermarkStrategy.noWatermarks(), "kfk");
```

```
// 对数据进行运算，并设置一定的异常概率

s1.map(new MapFunction<String, String>() {

    @Override

    public String map(String s) throws Exception {

        if (s.equals("x") && 3 / RandomUtils.nextInt(0, 3) == 1) ;

        return s;

    }

});

// 将处理结果写入 mysql

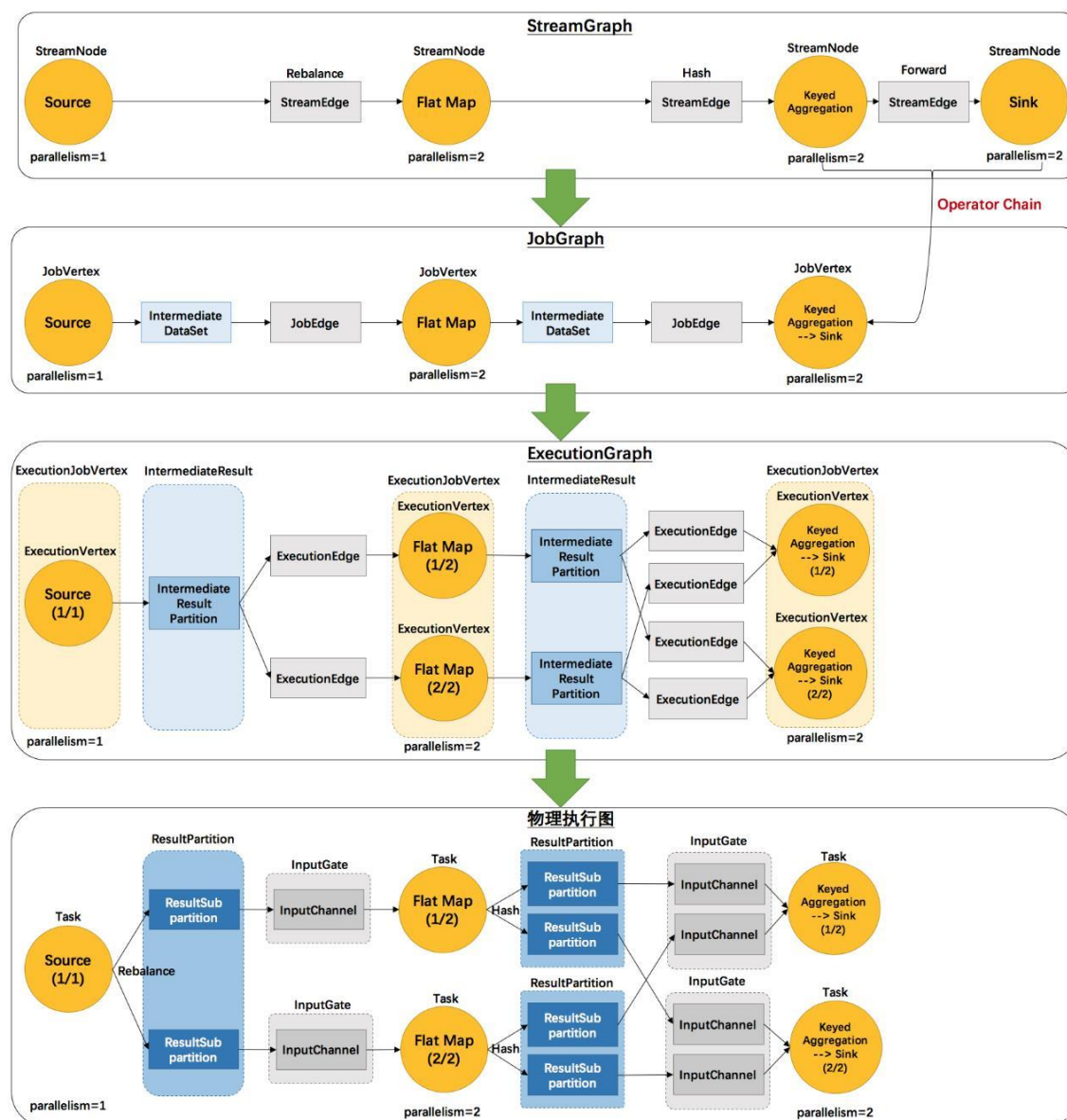
s1.addSink(exactlyOnceSink);

env.execute();
```

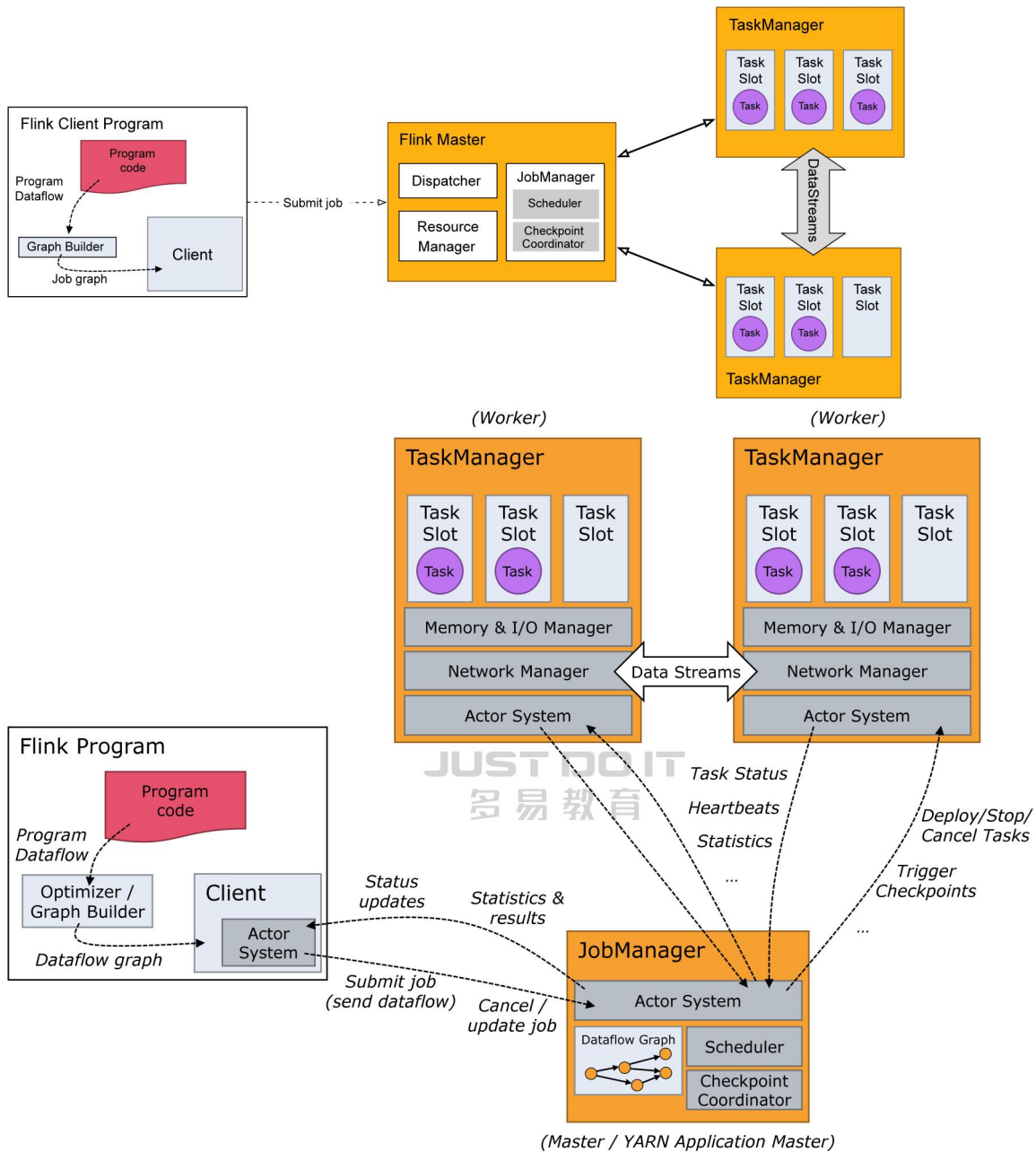
14. Flink 程序分布式部署运行

14.1. Job 执行流程

四层	说明	备注
StreamGraph	代码生成的最初的图	表示程序的拓扑结构。
JobGraph	将多个符合条件的节点，链接为一个节点	可减少数据在节点之间流动所需要的序列化/反序列化/传输消耗
ExecutionGraph	JobGraph的并行化版本	是调度层最核心的数据结构
PhysicalGraph	JobManager根据ExecutionGraph对Job进行调度后，在各个TaskManager上部署Task后形成的“图”	并不是一个具体的数据结构。



14.2. flink 集群运行时架构



14.3. flink standalone 集群

Flink 程序中如果要访问到hdfs，则需要添加2个jar包到flink的lib目录中

`flink-shaded-hadoop-3-uber-3.1.1.7.2.9.0-173-9.0.jar`

`commons-cli-1.4.jar`

Standalone session 集群模式的缺点：

- 资源利用弹性不够（资源总量是定死的；job退出后也不能立刻回收资源）
- 资源隔离度不够（所有job共享集群的资源）
- 所有job共用一个jobmanager，负载过大

将flink程序运行行为standalone集群模式，需要安装flink standalone集群：

集群搭建配置

- `conf/master` 配置JobManager地址
`doit01:8081`
- `conf/workers` 配置TaskManager机器地址
`doit01`
`doit02`
`doit03`
- `conf/flink-conf.yaml` 程序参数配置
`jobmanager.rpc.address: doit01`
`taskmanager.numberOfTaskSlots: 2`



集群启停命令

`bin/start-cluster.sh`

`bin/stop-cluster.sh`

应用提交

- 通过 web ui 提交 job

Uploaded Jars

doit29_flink-1.0.jar

2022-03-15, 17:25:52

cn.doitedu.flink.demos_16_TaskChainDemo

主类名

4 默认并行度

Savepoint Path 指定想要恢复的保存点 (快照点)

Submit

- 通过命令 `bin/flink run` 提交 job

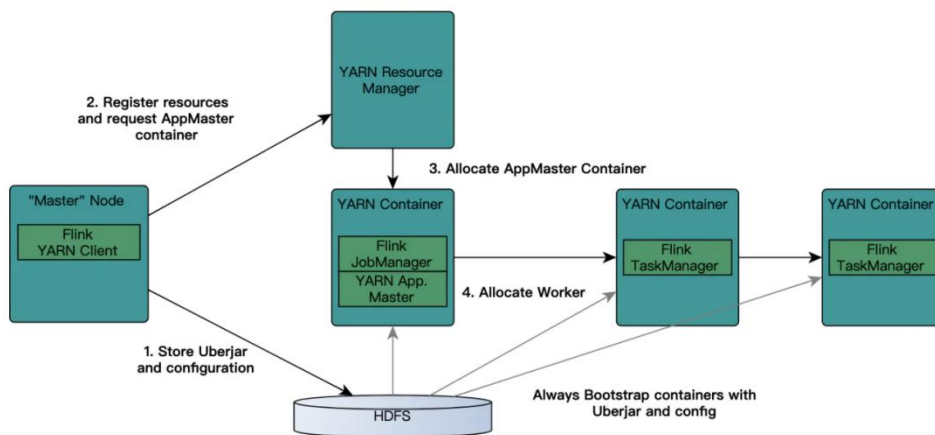
```
# 提交 standalone 模式的 job
# -c 主类名
# -p 并行度
# -s 从指定 savepoint 恢复
bin/flink run -t remote \
-c cn.doitedu.flink.java.demos._28_ToleranceSideToSideTest \
-p 5 \
-s hdfs://doit01:8020/eos_savepoint1/savepoint-5f1bc3-dde7a8627fff \
/root/flink_course-1.0.jar

# 触发 standalone 模式 job 做 savepoint
# -d : detach 模式, 客户端提交完 job 即退出
# -t remote : 表示 job 是 standalone 运行模式

bin/flink savepoint -t remote 5f1bc357bfd1e686e34b6cfce6d0cda9 hdfs://doit01:8020/eos_savepoint1
```

14.4. flink on yarn 概述

yarn 模式运行时示意图



基础逻辑

- flink on yarn, 本质上就是去 yarn 集群上申请容器, 来运行 flink 的 jobmanager+taskmanager 集群
- flink 的 job, 是 flink 集群内部的概念, 它对 yarn 是不可见的

14.5. Flink on yarn 的三种模式

Flink 程序可以运行以下 3 中模式:

- **Application Mode** 【生产中建议使用的模式】

每个 job 独享一个集群, job 退出集群则退出, 用户类的 main 方法在集群上运行

- **Per-Job Mode**

每个 job 独享一个集群, job 退出集群则退出, 用户类的 main 方法在 client 端运行;

(大 job, 运行时长很长, 比较合适; 因为每起一个 job, 都要去向 yarn 申请容器启动 jm,tm, 比较耗时)

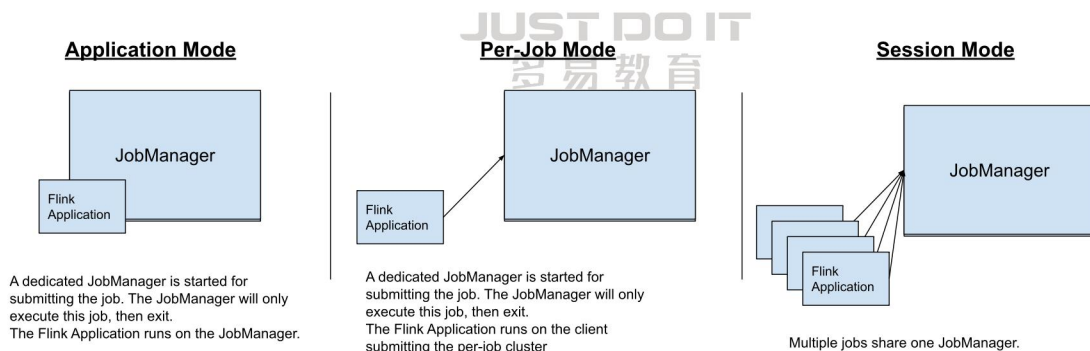
- **Session Mode.**

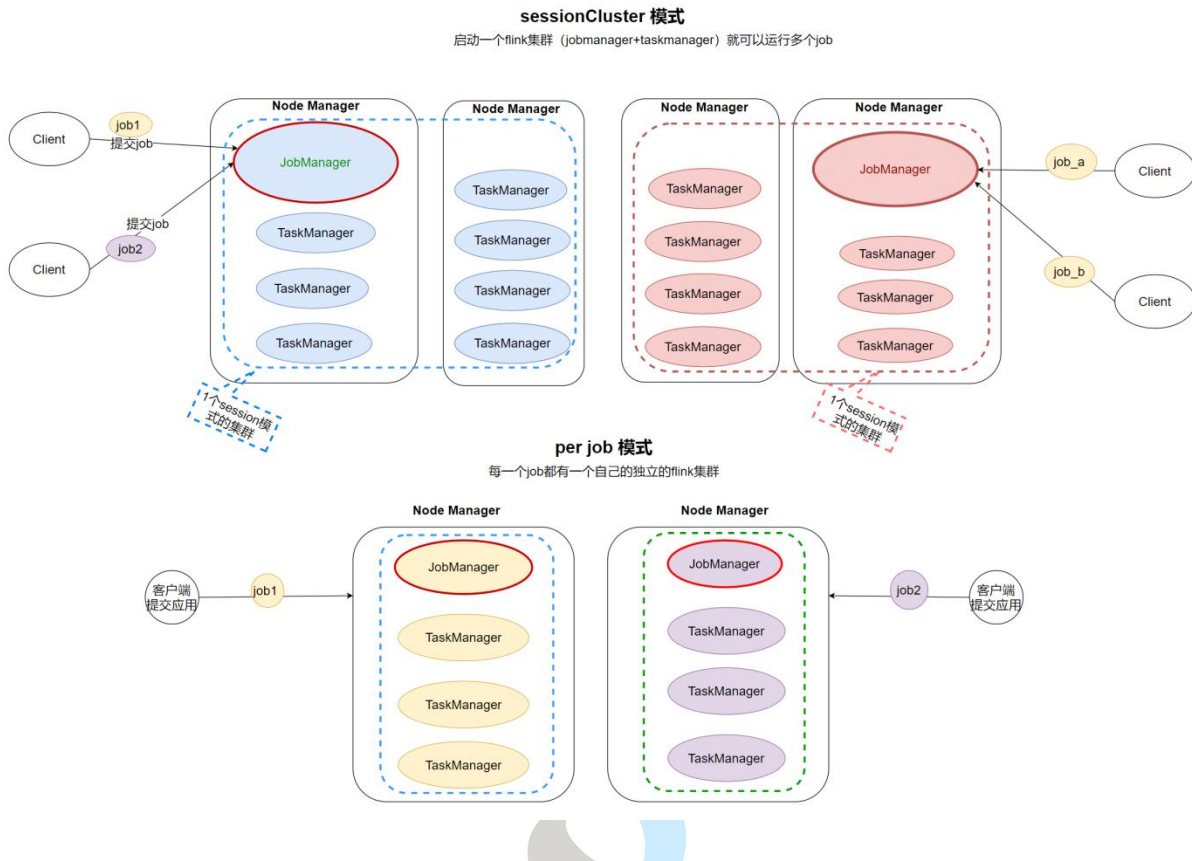
多个 job 共享同一个集群<jobmanager/taskmanager>、job 退出集群也不会退出, 用户类的 main 方法在 client 端运行;

(需要频繁提交大量小 job 的场景比较适用; 因为每次提交一个新 job 的时候, 不需要去向 yarn 注册应用)

上述 3 种模式的区别点在:

- 集群的生命周期和资源的隔离保证
- 用户类的 main 方法是运行在 client 端, 还是在集群端





14.5.1. Application Mode

In all the other modes, the application's `main()` method is executed on the client side. This process includes downloading the application's dependencies locally, executing the `main()` to extract a representation of the application that Flink's runtime can understand (i.e. the `JobGraph`) and ship the dependencies and the `JobGraph(s)` to the cluster. This makes the Client a heavy resource consumer as it may need substantial network bandwidth to download dependencies and ship binaries to the cluster, and CPU cycles to execute the `main()`. This problem can be more pronounced when the Client is shared across users.

14.5.2. Per-Job Mode

Aiming at providing better resource isolation guarantees, the Per-Job mode uses the available resource provider framework (e.g. YARN, Kubernetes) to spin up a cluster for each submitted job. This cluster is available to that job only. When the job finishes, the cluster is torn down and any lingering resources (files, etc) are cleared up. This provides better resource isolation, as a misbehaving job can only bring down its own TaskManagers. In addition, it spreads the load of book-keeping across multiple JobManagers, as there is one per job. For these reasons, the Per-Job resource allocation model is the preferred mode by many production reasons.

14.5.3. Session Mode

Session mode assumes an already running cluster and uses the resources of that cluster to execute any submitted application. Applications executed in the same (session) cluster use, and consequently compete for, the same resources. This has the advantage that you do not pay the resource

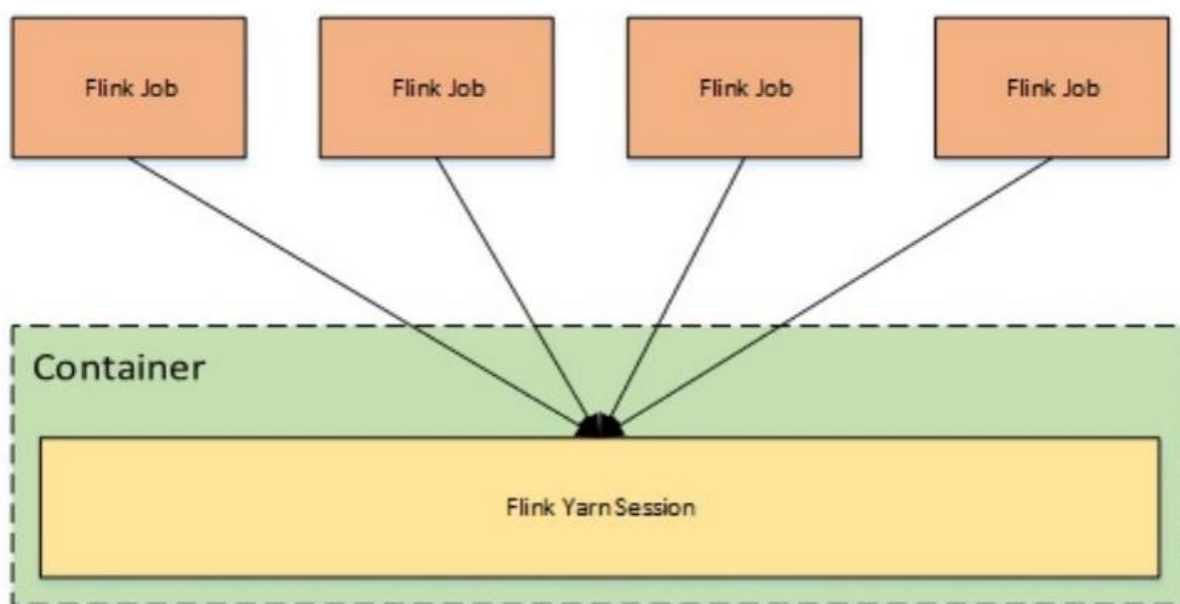
overhead of spinning up a full cluster for every submitted job. But, if one of the jobs misbehaves or brings down a TaskManager, then all jobs running on that TaskManager will be affected by the failure. This, apart from a negative impact on the job that caused the failure, implies a potential massive recovery process with all the restarting jobs accessing the filesystem concurrently and making it unavailable to other services. Additionally, having a single cluster running multiple jobs implies more load for the JobManager, who is responsible for the book-keeping of all the jobs in the cluster.

14.6. yarn session 模式提交

Yarn-Session 模式：所有**作业共享集群资源**，隔离性差，JM 负载瓶颈，main 方法在客户端执行。适合执行时间短，频繁执行的短任务，集群中的所有作业**只有一个 JobManager**，另外，**Job 被随机分配给 TaskManager**

特点：

Session-Cluster 模式需要先启动集群，**然后再提交作业**，接着会向 yarn 申请一块空间后，资源永远保持不变。如果资源满了，下一个作业就无法提交，只能等到 yarn 中的其中一个作业执行完成后，释放了资源，下个作业才会正常提交。所有作业共享 Dispatcher 和 ResourceManager；共享资源；适合规模小执行时间短的作业。



基本操作命令

提交命令: `bin/yarn-session.sh -help`

停止命令: `yarn application -kill application_1550836652097_0002`

具体操作步骤

(1) 先开辟资源启动 session 模式集群

```
# 老版本:

bin/yarn-session.sh -n 3 -jm 1024 -tm 1024

# -n --> 指定需要启动多少个 Taskmanager

# 新版本:

bin/yarn-session.sh -jm 1024 -tm 1024 -s 2 -m yarn-cluster -ynm hello -qu default

# -jm --> jobmanager memory

# -tm --> taskmanager memory

# -m yarn-cluster --> 集群模式 (yarn 集群模式)

# -s --> 规定每个 taskmanager 上的 taskSlot 数 (槽位数)

# -nm --> 自定义 application 名称

# -qu --> 指定要提交到的 yarn 队列
```

启动的服务进程

YarnSessionClusterEntrypoint (AppMaster, 即 JobManager)

注意: 此刻并没有 taskmanager, 也就是说, taskmanager 是在后续提交 job 时根据资源需求动态申请容器启动的;

(2) 向已运行的 session 模式集群提交 job

- 通过命令提交 job 到已运行的 yarn-session 集群

```
bin/flink run -d -yid application_1550579025929_62420 -p 4 -c
cn.doitedu.flink.java.demos._28_ToleranceSideToSideTest /root/flink_course-1.0.jar
```

- 通过已运行 yarn-session 集群的 webui 上提交 job

session 集群模式下的进程角色名称

- jobmanager 叫做 YarnSessionClusterEntrypoint
- taskmanager 叫做 YarnTaskExecutorRunner
- 客户端叫做 FlinkYarnSessionCli

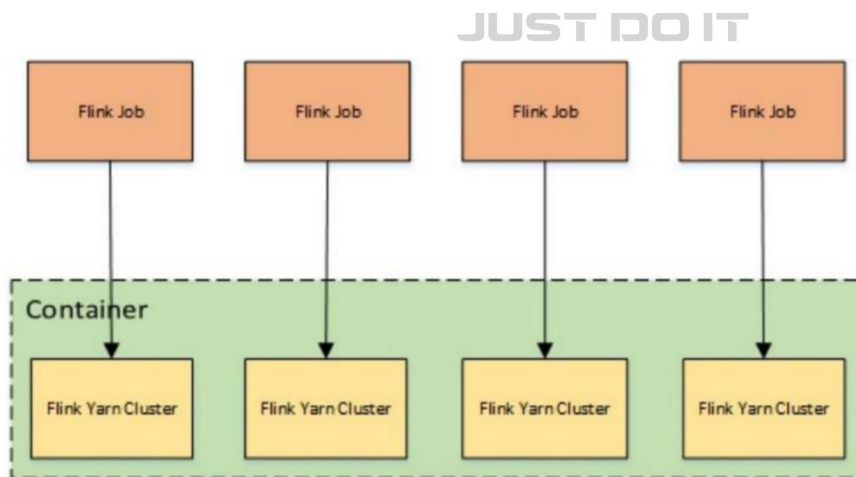
session 集群模式：更适用于有大量小的 job 需要频繁提交运行

14.7. yarn perJob 模式提交

Yarn-Per-Job 模式：每个作业单独启动集群，隔离性好，JM 负载均衡，main 方法在客户端执行。
在 per-job 模式下，每个 Job 都有一个 JobManager，每个 TaskManager 只有单个 Job。

特点：

一个任务会对应一个 Job，每提交一个作业会根据自身的情况，都会单独向 yarn 申请资源，直到作业执行完成，一个作业的失败与否并不会影响下一个作业的正常提交和运行。独享 Dispatcher 和 ResourceManager，按需接受资源申请；适合规模大长时间运行的作业。



提交命令 (申请容器启动 flink 集群以及提交 job，是合二为一的)

```
bin/flink run -m yarn-cluster -yjm 1024 -ynm sea -yqu default -ys 2 -ytm 1024 -p 4 -c
cn.doit._16_TaskChainDemo /root/doit29_flink-1.0.jar
```

14.8. yarn application 模式提交

```
[root@doit01 flink-1.14.3]# bin/flink run-application -t yarn-application \  
-yjm 1024 -ynm sea -yqu default -ys 2 \  
-ytm 1024 -p 4 \  
-c cn.doitedu.flink.java.demos._27_KafkaSinkYarn /root/flink_course-1.0.jar
```



15. Flink 工具库 CDC

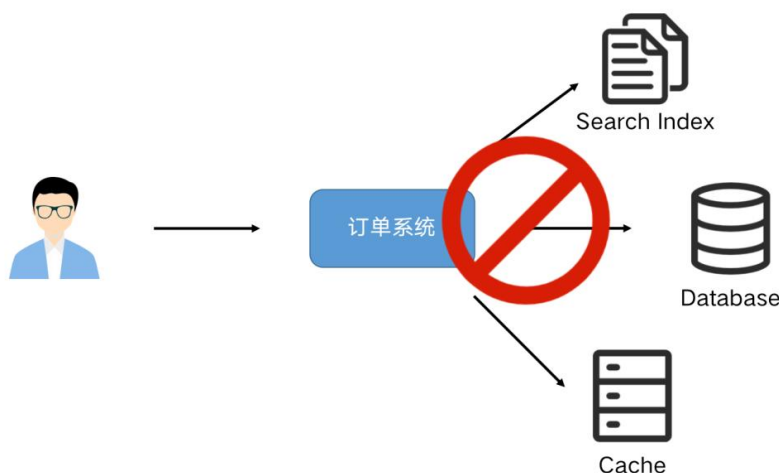
15.1. 什么是 CDC?

CDC 是 (Change Data Capture 变更数据获取) 的简称。

核心思想是, 监测并捕获数据库的变动 (包括数据 或 数据表的插入 INSERT、更新 UPDATE、删除 DELETE 等), 将这些变更按发生的顺序完整记录下来, 写入到消息中间件中以供其他服务进行订阅及消费。

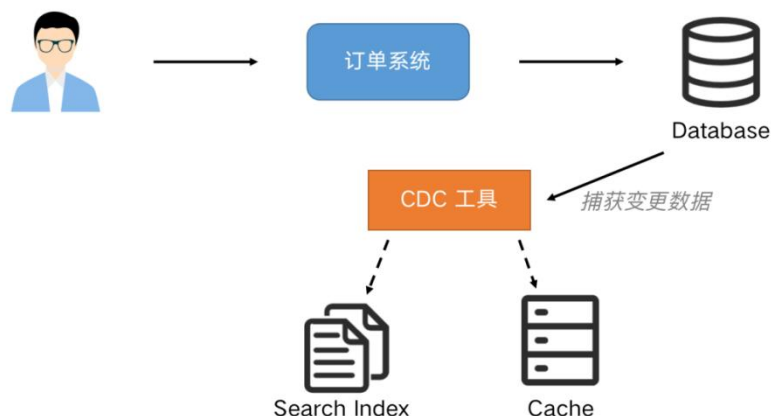
15.2. 传统的数据同步方案与 Flink SQL CDC 解决方案

业务系统经常会遇到需要更新数据到多个存储的需求。例如: 一个订单系统刚刚开始只需要写入数据库即可完成业务使用。某天 BI 团队期望对数据库做全文索引, 于是我们同时要写多一份数据到 ES 中, 改造后一段时间, 又有需求需要写入到 Redis 缓存中。



很明显这种模式是不可持续发展的, 这种双写到各个数据存储系统中可能导致不可维护和扩展, 数据一致性问题等, 需要引入分布式事务, 成本和复杂度也随之增加。我们可以通过 CDC (Change Data Capture) 工具进行解除耦合, 同步到下游需要同步的存储系统。通过这种方式提高系统的稳健性, 也

方便后续的维护。



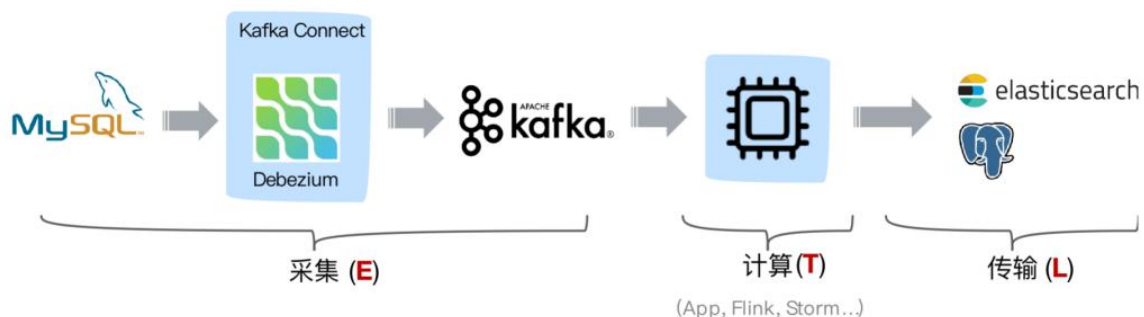
15.3. Flink SQL CDC 数据同步与原理解析

CDC 全称是 Change Data Capture，它是一个比较广义的概念，只要能捕获变更的数据，我们都可以称为 CDC。业界主要有基于查询的 CDC 和基于日志的 CDC，可以从下面表格对比他们功能和差异点。

	基于查询的 CDC	基于日志的 CDC
概念	每次捕获变更发起 Select 查询 进行全表扫描，过滤出查询之间变更的数据	读取数据存储系统的 log，例如 MySQL 里面的 binlog持续监控
开源产品	Sqoop, Kafka JDBC Source	Canal, Maxwell, Debezium
执行模式	Batch	Streaming
捕获所有数据的变化	✗	✓
低延迟，不增加数据库负载	✗	✓
不侵入业务（LastUpdated字段）	✗	✓
捕获删除事件和旧记录的状态	✗	✓
捕获旧记录的状态	✗	✓

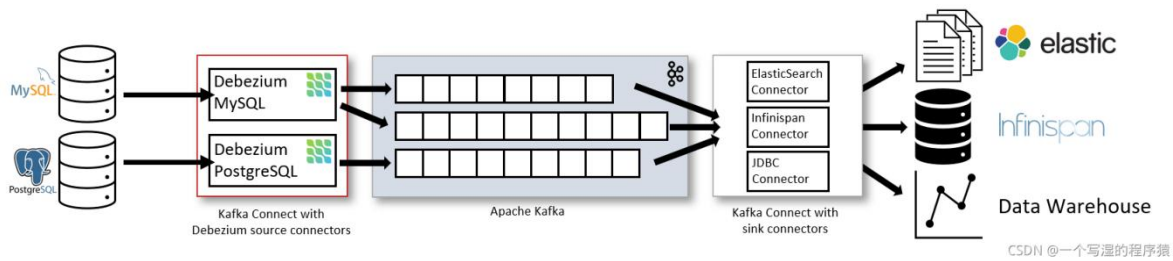
经过以上对比，我们可以发现基于日志 CDC 有以下这几种优势：

- 能够捕获所有数据的变化，捕获完整的变更记录。在异地容灾，数据备份等场景中得到广泛应用，如果是基于查询的 CDC 有可能导致两次查询的中间一部分数据丢失
- 每次 DML 操作均有记录无需像查询 CDC 这样发起全表扫描进行过滤，拥有更高的效率和性能，具有低延迟，不增加数据库负载的优势
- 无需入侵业务，业务解耦，无需更改业务模型
- 捕获删除事件和捕获旧记录的状态，在查询 CDC 中，周期的查询无法感知中间数据是否删除



在实时性、吞吐量方面占优，如果数据源是 MySQL、PostgreSQL、MongoDB 等常见的数据库实现，

建议使用 Debezium 来实现变更数据的捕获（下图来自 Debezium 官方文档）。如果使用的只有 MySQL，则可以用 Canal。

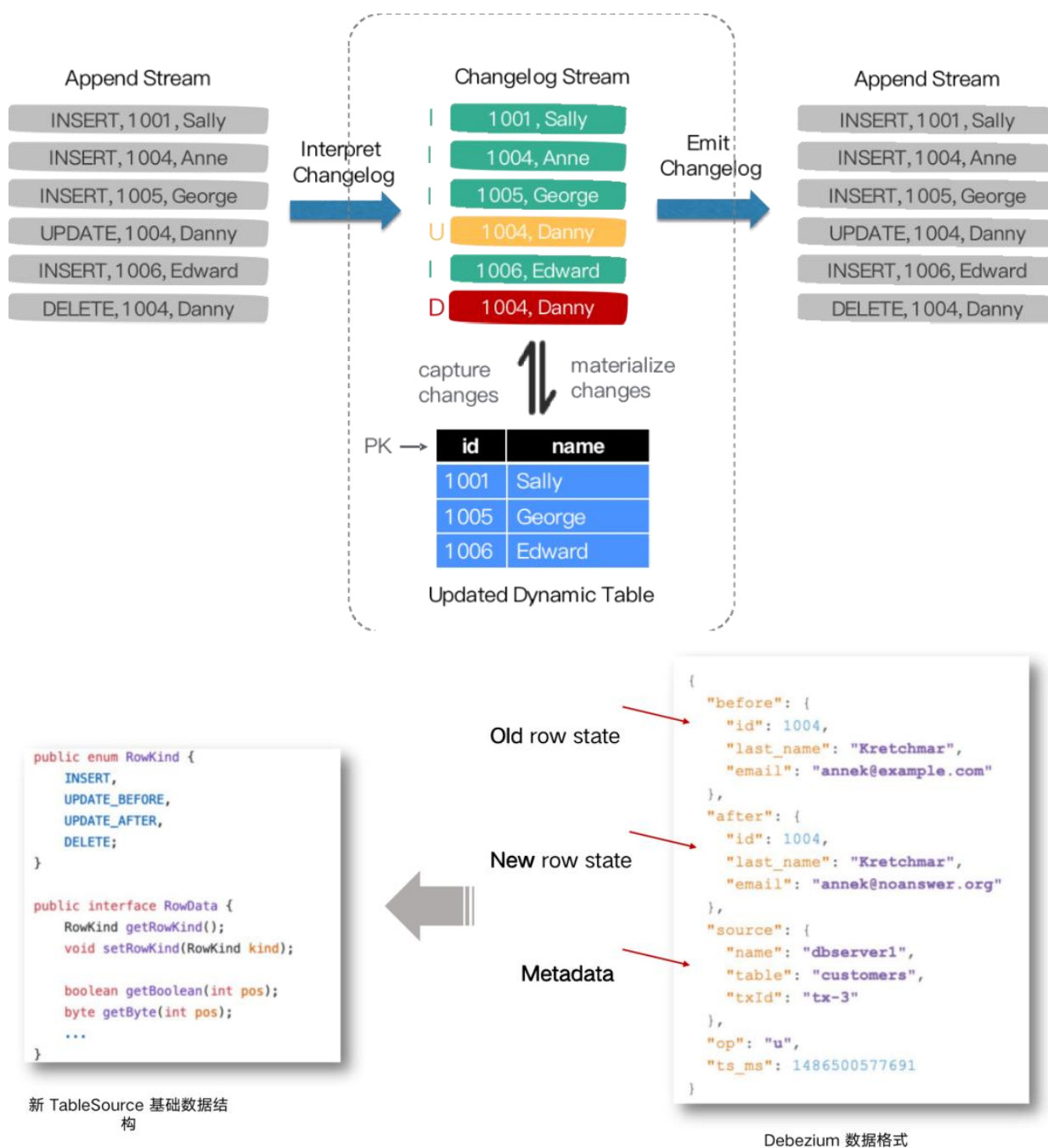


15.4. 基于日志的 CDC 方案介绍

从 ETL 的角度进行分析，一般采集的都是业务库数据，这里使用 MySQL 作为需要采集的数据库，通过 Debezium 把 MySQL Binlog 进行采集后发送至 Kafka 消息队列，然后对接一些实时计算引擎或者 APP 进行消费后把数据传输输入 OLAP 系统或者其他存储介质。

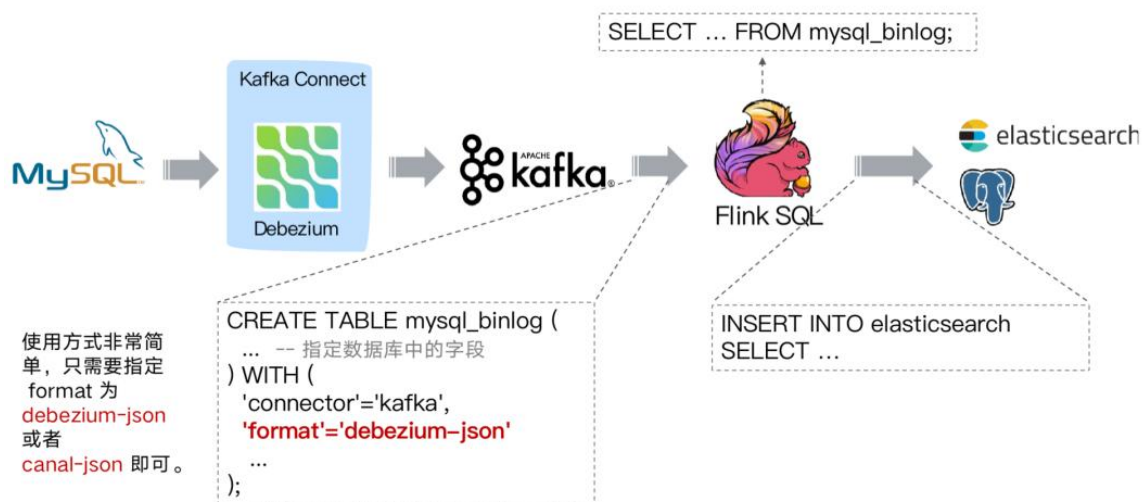
Flink 希望打通更多数据源，发挥完整的计算能力。我们生产中主要来源于业务日志和数据库日志，Flink 在业务日志的支持上已经非常完善，但是在数据库日志支持方面在 Flink 1.11 前还属于一片空白，这就是为什么要集成 CDC 的原因之一。

Flink SQL 内部支持了完整的 changelog 机制，所以 Flink 对接 CDC 数据只需要把 CDC 数据转换成 Flink 认识的数据，所以在 Flink 1.11 里面重构了 TableSource 接口，以便更好支持和集成 CDC。

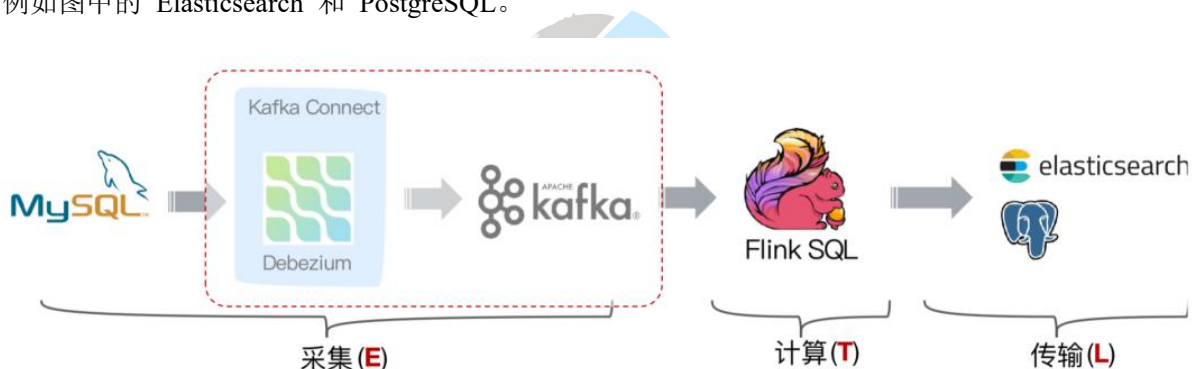


重构后的 TableSource 输出的都是 RowData 数据结构，代表了一行的数据。在 RowData 上面会有一个元数据的信息，我们称为 RowKind。RowKind 里面包括了插入、更新前、更新后、删除，这样和数据库里面的 binlog 概念十分类似。通过 Debezium 采集的 JSON 格式，包含了旧数据和新数据行以及原数据信息，op 的 u 表示是 update 更新操作标识符，ts_ms 表示同步的时间戳。因此，对接 Debezium JSON 的数据，其实就是将这种原始的 JSON 数据转换成 Flink 认识的 RowData。选择 Flink 作为 ETL 工具

当选择 Flink 作为 ETL 工具时，在数据同步场景，如下图同步结构：



通过 Debezium 订阅业务库 MySQL 的 Binlog 传输至 Kafka，Flink 通过创建 Kafka 表指定 format 格式为 **debezium-json**，然后通过 Flink 进行计算后或者直接插入到其他外部数据存储系统，例如图中的 Elasticsearch 和 PostgreSQL。



但是这个架构有个缺点，我们可以看到采集端组件过多导致维护繁杂，这时候就会想是否可以用 Flink SQL 直接对接 MySQL 的 binlog 数据呢，有没可以替代的方案呢？

答案是有的！经过改进后结构如下图：



社区开发了 **flink-cdc-connectors** 组件，这是一个可以直接从 MySQL、PostgreSQL 等数据库直接读

取全量数据和增量变更数据的 source 组件。目前也已开源，开源地址：

<https://github.com/ververica/flink-cdc-connectors>

flink-cdc-connectors 可以用来替换 Debezium+Kafka 的数据采集模块，从而实现 Flink SQL 采集+计算+传输（ETL）一体化，这样做的优点有以下：

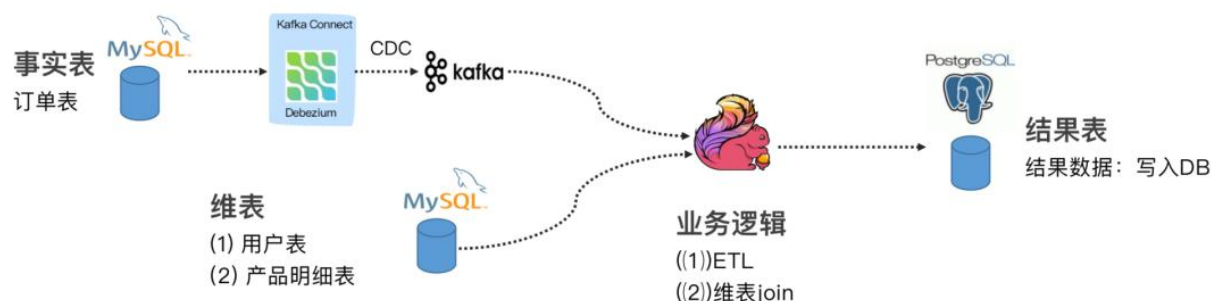
- 开箱即用，简单易上手
- 减少维护的组件，简化实时链路，减轻部署成本
- 减小端到端延迟
- Flink 自身支持 Exactly Once 的读取和计算
- 数据不落地，减少存储成本
- 支持全量和增量流式读取
- binlog 采集位点可回溯

15.5. 基于 Flink SQL CDC 的数据同步方案实践

下面给大家带来 3 个关于 Flink SQL + CDC 在实际场景中使用较多的案例。在完成实验时候，你需要 Docker、MySQL、Elasticsearch 等组件，具体请参考每个案例参考文档。

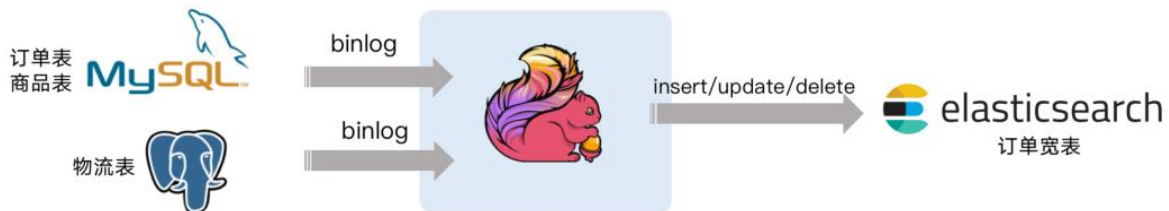
15.5.1. 案例 1 : Flink SQL CDC + JDBC Connector

这个案例通过订阅我们订单表（事实表）数据，通过 Debezium 将 MySQL Binlog 发送至 Kafka，通过维表 Join 和 ETL 操作把结果输出至下游的 PG 数据库。



15.5.2. 案例 2 : CDC Streaming ETL

模拟电商公司的订单表和物流表，需要对订单数据进行统计分析，对于不同的信息需要进行关联后续形成订单的大宽表后，交给下游的业务方使用 ES 做数据分析，这个案例演示了如何只依赖 Flink 不依赖其他组件，借助 Flink 强大的计算能力实时把 Binlog 的数据流关联一次并同步至 ES 。



例如如下的这段 Flink SQL 代码就能完成实时同步 MySQL 中 orders 表的全量+增量数据的目的。

```
CREATE TABLE orders (  
  order_id INT,  
  order_date TIMESTAMP(0),  
  customer_name STRING,  
  price DECIMAL(10, 5),  
  product_id INT,  
  order_status BOOLEAN  
) WITH (  
  'connector' = 'mysql-cdc',  
  'hostname' = 'localhost',  
  'port' = '3306',  
  'username' = 'root',  
  'password' = '123456',  
  'database-name' = 'mydb',  
  'table-name' = 'orders'  
);  
  
SELECT * FROM orders
```

15.5.3. 案例 3 : Streaming Changes to Kafka

下面案例就是对 GMV 进行天级别的全站统计。包含插入/更新/删除，只有付款的订单才能计算进入 GMV，观察 GMV 值的变化。



15.6. Flink SQL CDC 的更多应用场景

Flink SQL CDC 不仅可以灵活地应用于实时数据同步场景中，还可以打通更多场景提供给用户选择。

Flink 在数据同步场景中的灵活定位

- 如果已有 Debezium/Canal + Kafka 的采集层 (E), 可以使用 Flink 作为计算层 (T) 和传输层 (L)
- 也可用 Flink 替代 Debezium/Canal, 由 Flink 直接同步变更数据到 Kafka, Flink 统一 ETL 流程
- 如果不需要 Kafka 数据缓存, 可以由 Flink 直接同步变更数据到目的地, Flink 统一 ETL 流程

15.7. Flink SQL CDC : 打通更多场景

- 实时数据同步, 数据备份, 数据迁移, 数仓构建

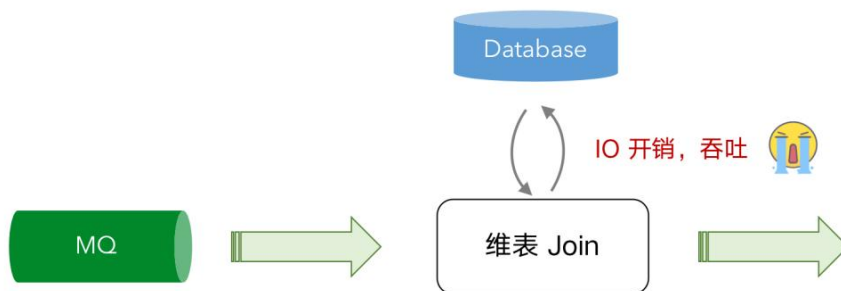
优势: 丰富的上下游 (E & L), 强大的计算 (T), 易用的 API (SQL), 流式计算低延迟

- 数据库之上的实时物化视图、流式数据分析
- 索引构建和实时维护
- 业务 cache 刷新
- 审计跟踪
- 微服务的解耦, 读写分离

- 基于 CDC 的维表关联

下面介绍一下为何用 CDC 的维表关联会比基于查询的维表查询快。

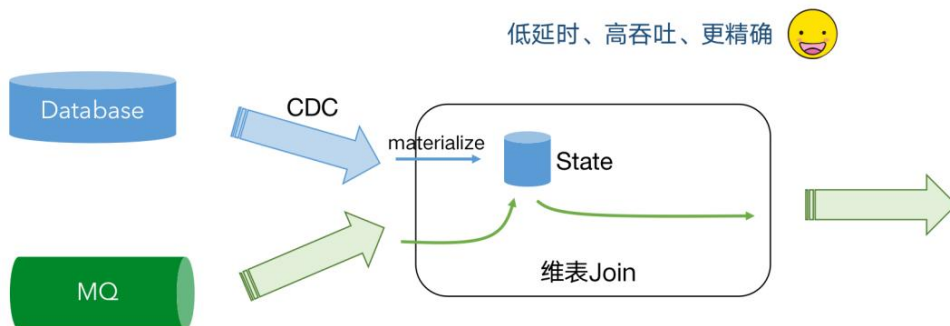
- 基于查询的维表关联



目前维表查询的方式主要是通过 Join 的方式，数据从消息队列进来后通过向数据库发起 IO 的请求，由数据库把结果返回后合并再输出到下游，但是这个过程无可避免的产生了 IO 和网络通信的消耗，导致吞吐量无法进一步提升，就算使用一些缓存机制，但是因为缓存更新不及时可能会导致精确性也没那么高。

JUST DO IT
多易教育

- 基于 CDC 的维表关联



我们可以通过 CDC 把维表的数据导入到维表 Join 的状态里面，在这个 State 里面因为它是一个分布式的 State，里面保存了 Database 里面实时的数据库维表镜像，当消息队列数据过来时候无需再次查询远程的数据库了，直接查询本地磁盘的 State，避免了 IO 操作，实现了低延迟、高吞吐，更精准。

16. flink 工具库 flinkcep



17. Flink 性能与监控

17.1. flink metrics

counter

```
public class MyMapper extends RichMapFunction<String, String> {

    private transient Counter counter;

    @Override

    public void open(Configuration config) {

        this.counter = getRuntimeContext()

            .getMetricGroup()

            .counter("myCounter");

    }

    @Override

    public String map(String value) throws Exception {

        this.counter.inc();

        return value;

    }

}
```

gauge

```
public class MyMapper extends RichMapFunction<String, String> {

    private transient int valueToExpose = 0;

    @Override

    public void open(Configuration config) {

        getRuntimeContext()

            .getMetricGroup()

            .gauge("MyGauge", new Gauge<Integer>() {

                @Override

                public Integer getValue() {

                    return valueToExpose;

                }

            });

    }

    @Override

    public String map(String value) throws Exception {

        valueToExpose++;

        return value;

    }

}
```

Histogram

```
public class MyMapper extends RichMapFunction<Long, Long> {

    private transient Histogram histogram;

    @Override

    public void open(Configuration config) {

        this.histogram = getRuntimeContext()

            .getMetricGroup()

            .histogram("myHistogram", new MyHistogram());

    }

    @Override

    public Long map(Long value) throws Exception {

        this.histogram.update(value);

        return value;

    }

}
```

Meter

```
public class MyMapper extends RichMapFunction<Long, Long> {

    private transient Meter meter;

    @Override

    public void open(Configuration config) {

        this.meter = getRuntimeContext()

            .getMetricGroup()

            .meter("myMeter", new MyMeter());

    }

    @Override

    public Long map(Long value) throws Exception {

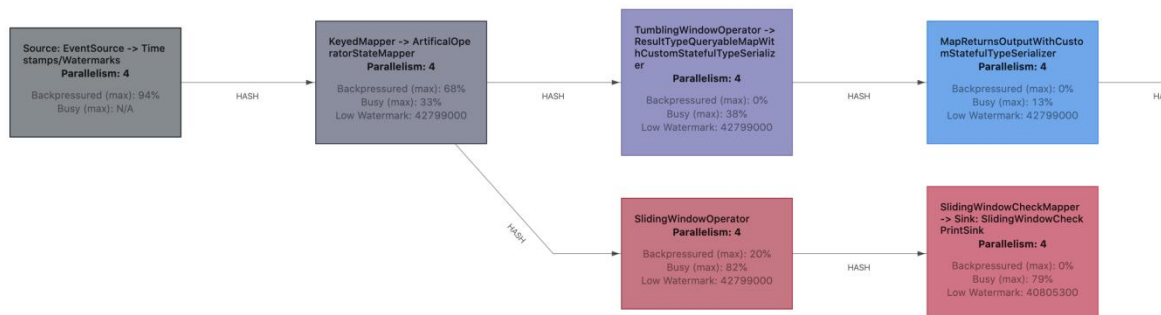
        this.meter.markEvent();

        return value;

    }

}
```

17.2. flink 反压机制



什么是反压

反压（backpressure）是实时计算应用开发中，特别是流式计算中，十分常见的问题。

反压意味着数据管道中某个节点成为瓶颈，“下游”处理速率跟不上“上游”发送数据的速率，而需要对“上游”进行限速。

由于实时计算应用通常使用消息队列来进行生产端和消费端的解耦，消费端数据源是 pull-based 的，所以反压通常是从某个节点传导至数据源并降低数据源（比如 Kafka consumer）的摄入速率。简单来说就是“下游”处理速率跟不上“上游”发送数据的速率，“下游”来不及消费，导致队列被占满后，“上游”的生产会被阻塞，最终导致数据源的摄入被阻塞；

反压会影响到两项指标: checkpoint 时长和 state 大小

（1）前者是因为 checkpoint barrier 是不会越过普通数据的，数据处理被阻塞也会导致 checkpoint barrier 流经整个数据管道的时长变长，因而 checkpoint 总体时间（End to End Duration）变长。

（2）后者是因为为保证 EOS（Exactly-Once-Semantics，准确一次），对于有两个以上输入管道的 Operator，checkpoint barrier 需要对齐（Alignment），接受到较快的输入管道的 barrier 后，它后面数据会被缓存起来但不处理，直到较慢的输入管道的 barrier 也到达，这些被缓存的数据会被放到 state 里面，导致 checkpoint 变大。

这两个影响对于生产环境的作业来说是十分危险的，因为 `checkpoint` 是保证数据一致性的关键，`checkpoint` 时间变长有可能导致 `checkpoint` 超时失败，而 `state` 大小同样可能拖慢 `checkpoint` 甚至导致 OOM（使用 `Heap-based StateBackend`）或者物理内存使用超出容器资源（使用 `RocksDBStateBackend`）的稳定性问题。

如何监控定位反压的发生及节点

要解决反压首先要做的是定位到造成反压的节点，这主要有两种办法：

1. 通过 Flink Web UI 自带的反压监控面板；
2. 通过 Flink Task Metrics。

（1）反压监控面板

Flink Web UI 的反压监控提供了 `SubTask` 级别的反压监控，原理是通过周期性对 `Task` 线程的栈信息采样，得到线程被阻塞在请求 `Buffer`（意味着被下游队列阻塞）的频率来判断该节点是否处于反压状态。

Detail	SubTasks	TaskManagers	Watermarks	Accumulators	BackPressure	Metrics	FlameGraph
Measurement: 3s ago Back Pressure Status: HIGH							
SubTask	Backpressured / Idle / Busy			Backpressure Status			
0	100% / 0% / 0%			HIGH			
1	100% / 0% / 0%			HIGH			
2	0% / 100% / 0%			OK			
3	0% / 100% / 0%			OK			
4	0% / 100% / 0%			OK			
5	49% / 51% / 0%			LOW			
6	97% / 3% / 0%			HIGH			
7	100% / 0% / 0%			HIGH			
8	100% / 0% / 0%			HIGH			
9	100% / 0% / 0%			HIGH			
10	100% / 0% / 0%			HIGH			

- OK: 0% <= back pressured <= 10%
- LOW: 10% < back pressured <= 50%
- HIGH: 50% < back pressured <= 100%

另外，还可以看到每个 `subtask` 背压、闲置、繁忙的具体时长

(2) Task Metrics

Flink 提供的 Task Metrics 是更好的反压监控手段

如果一个 Subtask 的发送端 Buffer 占用率很高，则表明它被下游反压限速了；

如果一个 Subtask 的接受端 Buffer 占用很高，则表明它将反压传导至上游。

17.3. flink 监控平台

17.4. flink 性能调优手段



JUST DO IT
多易教育

18. 附录

18.1. 两阶段提交协议（2PC）

两阶段提交协议（Two -Phase Commit, 2PC）是解决分布式事务问题最常用的方法，它可以保证在分布式事务中，要么所有参与进程都提交事务，要么都取消，即实现 ACID 中的 A（原子性）。

两阶段提交协议中有两个重要角色：

- 协调者（Coordinator）
- 参与者（Participant）

其中协调者只有一个，起到分布式事务的协调管理作用，参与者有多个。

两阶段提交阶段分为两个阶段：投票阶段（Voting）和 提交阶段（Commit）。

投票阶段:

(1) 协调者向所有参与者发送 `prepare` 请求和事务内容, 询问是否可以准备事务提交, 等待参与者的相应。

(2) 参与者执行事务中包含的操作, 并记录 `undo` 日志 (用于回滚) 和 `redo` 日志 (用于重放), 但不真正提交。

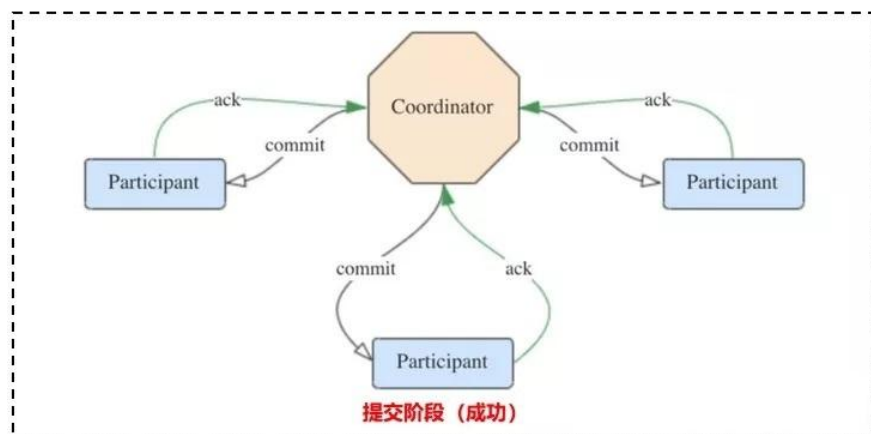
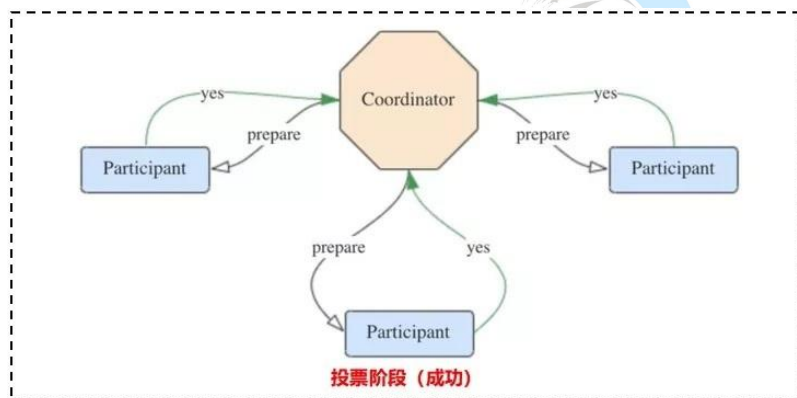
(3) 参与者向协调者返回事务操作的执行结果, 执行成功返回 `yes`, 失败返回 `no`。

提交阶段:

分为成功与失败两种情况。

一、若所有参与者都返回 `yes`, 说明事务可以提交:

- 协调者向所有参与者发送 `commit` 请求;
- 参与者收到 `commit` 请求后, 将事务真正提交上去, 并释放占用的事务资源, 向协调者返回 `ack`;
- 协调者收到所有参与者的 `ack` 消息, 事务成功完成, 如下图:

**二、若有参与者返回 `no` 或者超时未返回, 说明事务中断, 需要回滚:**

- 协调者向所有参与者发送 rollback 请求。
- 参与者收到 rollback 请求后，根据 undo 日志回滚到事务执行前的状态，释放占用的事务资源，并向协调者返回 ack。
- 协调者收到所有参与者的 ack 消息，事务回滚完成。

